МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»

Д. О. Кушнір

ПЕРИФЕРІЙНІ ПРИСТРОЇ, ІНТЕРФЕЙСИ ТА ДРАЙВЕРИ

Навчальний посібник

для студентів галузі знань F «Інформаційні технології»
спеціальності F7 «Комп'ютерна інженерія»

Рекомендувала Науково-методична рада
Національного університету «Львівська політехніка»

Львів
Видавництво Львівської політехніки
2026

УДК 004.35
К ___

Рецензенти:

Прізвище І. Б., науковий ступінь, вчене звання, посада, назва установи;
Прізвище І. Б., науковий ступінь, вчене звання, посада, назва установи;
Бочкарьов О. Ю., кандидат технічних наук, доцент, доцент кафедри електронних обчислювальних машин, Національний університет «Львівська політехніка»

Рекомендувала Науково-методична рада
Національного університету «Львівська політехніка»
як навчальний посібник для студентів спеціальності F7 «Комп'ютерна інженерія»
(протокол № __ від __ _____ 2026 р.)

К ___

Кушнір Д. О.

Периферійні пристрої, інтерфейси та драйвери : навч. посіб. / Д. О. Кушнір. – Львів : Видавництво Львівської політехніки, 2026. – 143 с.

ISBN 978-___-___-___-_

У навчальному посібнику розглянуто класифікацію периферійних інтерфейсів, топології з'єднання пристроїв, паралельні інтерфейси, послідовний асинхронний інтерфейс RS-232C (UART), універсальну послідовну шину USB та синхронні інтерфейси I²C і SPI. Наведено теоретичні відомості, практичні завдання з розроблення та дослідження програмного забезпечення для обміну даними через ці інтерфейси та їх інтеграції у складі вузла моніторингу, питання для самоперевірки, тестові завдання та тексти програм.

Для студентів спеціальності F7 «Комп'ютерна інженерія» галузі знань F «Інформаційні технології».

УДК 004.35

© Кушнір Д. О., 2026
© Національний університет «Львівська політехніка», 2026

ЗМІСТ

ВСТУП 5
1. Загальні відомості про периферійні інтерфейси 8
1.1. Класифікація периферійних інтерфейсів 8
1.2. Топології з'єднання периферійних пристроїв 10
1.3. Паралельні інтерфейси 12
1.4. Порівняння паралельних і послідовних інтерфейсів 15
1.5. Огляд інших послідовних інтерфейсів вбудованих систем 16
1.6. Драйвери периферійних пристроїв і рівні програмної підтримки 18
Питання для самоперевірки 22
Тестові завдання 23
2. Послідовний асинхронний інтерфейс RS-232C (UART) 25
2.1. Інтерфейс RS-232C 26
2.2. Керування потоком даних та апаратні засоби інтерфейсу 31
2.3. TTL-UART, RS-232C та RS-485 33
2.4. Протокол Modbus RTU та рівні взаємодії 36
2.5. Амплітудно-часова діаграма UART та кодування NRZI 38
2.6. Практичне завдання № 1. Дослідження програм передавача, приймача та моделі обміну даними інтерфейсу RS-232C 41
2.7. Практичне завдання № 2. Дослідження графічного представлення сигналів лінії зв'язку та кодування NRZI інтерфейсу USB 44
Питання для самоперевірки 49
Тестові завдання 50
3. Універсальна послідовна шина USB 52
3.1. Загальні характеристики та топологія шини 53
3.2. Обмін даними та логічна структура пристрою 54
3.3. Версії специфікації USB 56
3.4. З'єднувачі та кабелі USB 57
3.5. Флеш-пам'ять USB-накопичувачів 58
3.6. Практичне завдання № 3. Розроблення та дослідження елементів програмного драйвера інтерфейсу USB 59
Питання для самоперевірки 64
Тестові завдання 64
4. Синхронні послідовні інтерфейси I²C і SPI та інтеграція інтерфейсів 66
4.1. Шина I²C 67
4.2. Режими швидкості обміну шини I²C 71
4.3. Послідовний периферійний інтерфейс SPI 72
4.4. Дослідження магістралі логічним аналізатором 76
4.5. Вузол моніторингу та рівні програмного забезпечення 77
4.6. Практичне завдання № 4. Дослідження шини I²C та розроблення програмного драйвера периферійного давача 79

4.7. Практичне завдання № 5. Розроблення та дослідження інтегрованого вузла моніторингу комп'ютерної системи 84
Питання для самоперевірки 89
Тестові завдання 90
ТЕРМІНОЛОГІЧНИЙ СЛОВНИК 92
СПИСОК ЛІТЕРАТУРИ 97
ДОДАТОК А. ВАРІАНТИ ЗАВДАНЬ ТА ЕЛЕКТРОННІ РЕСУРСИ КУРСУ 100
А.1. Варіанти завдань 100
А.2. Електронні ресурси курсу 102
А.3. Підготовка програмного середовища 103
А.4. Вимоги до оформлення результатів і критерії оцінювання 104
ДОДАТОК Б. ТЕКСТИ ПРОГРАМ 105
Б.1. Програми до практичного завдання № 1 106
Б.2. Програми до практичного завдання № 2 112
Б.3. Програми до практичного завдання № 3 122
Б.4. Програми до практичного завдання № 4 129
Б.5. Програми до практичного завдання № 5 135
ДОДАТОК В. ВІДПОВІДІ ДО ТЕСТОВИХ ЗАВДАНЬ 138
ПРЕДМЕТНИЙ ПОКАЖЧИК 139
ІМЕННИЙ ПОКАЖЧИК 142

ВСТУП

Навчальний посібник «Периферійні пристрої, інтерфейси та драйвери» призначений для студентів третього курсу першого (бакалаврського) рівня вищої освіти галузі знань F «Інформаційні технології» спеціальності F7 «Комп'ютерна інженерія», які навчаються за освітньо-професійною програмою «Комп'ютерна інженерія». Посібник забезпечує вивчення однойменної навчальної дисципліни обсягом 4 кредити ЄКТС (120 год), з яких 60 год становлять аудиторні заняття (30 год лекцій, 15 год практичних і 15 год лабораторних занять), а 60 год – самостійна робота студентів; формою підсумкового контролю є екзамен. Посібник забезпечує практичну частину дисципліни – практичні та лабораторні заняття (30 год) і пов'язану з ними самостійну роботу студентів. Він містить п'ять практичних завдань, які охоплюють основні периферійні інтерфейси сучасних комп'ютерних систем, а також теоретичні відомості в обсязі, необхідному для їх виконання: класифікацію і топології периферійних інтерфейсів, паралельні інтерфейси, драйвери периферійних пристроїв, інтерфейси RS-232C, USB, I²C та SPI. Систематичний теоретичний курс дисципліни, що відповідає лекційним заняттям, викладено в навчальному посібнику [1], тому ці видання доповнюють одне одного. Питання для самоперевірки і тестові завдання можна використовувати для підготовки до навчальних занять і контрольних заходів.

Периферійна підсистема є невід'ємною складовою будь-якої комп'ютерної системи – від персонального комп'ютера до вбудованого вузла промислової автоматики чи системи моніторингу енергетичного обладнання. Саме через периферійні інтерфейси комп'ютерна система отримує дані від давачів, керує виконавчими пристроями, зберігає інформацію та обмінюється нею з іншими системами. Розвиток вбудованих систем, Інтернету речей і промислової автоматики зробив знання принципів роботи послідовних інтерфейсів і навички розроблення програмних драйверів однією з базових вимог до фахівця з комп'ютерної інженерії, що й зумовлює актуальність посібника.

Дисципліна спирається на знання, отримані під час вивчення дисциплін «Основи організації та функціонування комп'ютерів», «Програмування, ч. 1 (Основи алгоритмізації та програмування)» та «Теорія інформації та кодування»; зокрема, кодування сигналів NRZI, контроль парності та контрольні суми CRC, розглянуті в посібнику, безпосередньо продовжують теми останньої. Супутніми і наступними є дисципліни «Архітектура комп'ютерів» та «Програмне забезпечення мікропроцесорних систем», для яких посібник створює основу в частині периферійних підсистем, драйверів і обміну

даними мікроконтролерів з периферійними пристроями; набуті знання використовуються також у курсовому та дипломному проєктуванні. Матеріал посібника має міжпредметні зв'язки з електронікою (рівні сигналів, перетворювачі рівнів), операційними системами (драйвери, файли пристроїв, простір користувача та простір ядра) та комп'ютерними мережами (модель OSI, топології з'єднання, протоколи обміну).

Метою посібника є формування у студентів систематизованих знань про принципи побудови та функціонування периферійних інтерфейсів комп'ютерних систем і практичних навичок розроблення та відлагодження програмного забезпечення для обміну даними через ці інтерфейси.

Основні завдання посібника:

- ознайомити студентів з класифікацією периферійних інтерфейсів, топологіями з'єднання пристроїв, принципами роботи паралельних і послідовних інтерфейсів;
- розкрити принципи функціонування послідовного асинхронного інтерфейсу RS-232C (UART), універсальної послідовної шини USB та синхронних інтерфейсів I^2C і SPI, їх фізичні рівні, формати кадрів і структури транзакцій;
- навчити розробляти та відлагоджувати програми передавача і приймача, моделі транзакцій і програмні драйвери периферійних мікросхем;
- сформувати навички дослідження сигналів ліній зв'язку за допомогою амплітудно-часових діаграм і логічного аналізатора;
- показати принципи інтеграції кількох інтерфейсів у складі єдиного вузла та розподілу функцій між рівнями програмного забезпечення периферійної підсистеми.

Опрацювання матеріалу посібника сприяє формуванню таких компетентностей:

- здатність аналізувати та обирати периферійні інтерфейси для побудови комп'ютерних систем з урахуванням вимог до швидкості, дальності, кількості пристроїв і завадостійкості;
- здатність розробляти, відлагоджувати та документувати програмне забезпечення для обміну даними через послідовні інтерфейси, зокрема програмні драйвери периферійних пристроїв;
- здатність використовувати сучасні засоби моделювання та діагностики периферійних підсистем – віртуальні порти, симулятори мікроконтролерних систем, логічні аналізатори;
- здатність працювати з технічною документацією – стандартами, специфікаціями інтерфейсів і документацією на мікросхеми.

У результаті опрацювання посібника студент повинен знати класифікацію, характеристики та принципи роботи розглянутих інтерфейсів, формати кадрів і структури транзакцій, рівні програмного забезпечення периферійної підсистеми; уміти налаштовувати послідовні порти, розраховувати часові параметри обміну, розробляти та відлагоджувати програми обміну даними і драйвери периферійних мікросхем, досліджувати сигнали магістралі логічним аналізатором та інтегрувати кілька інтерфейсів у складі одного вузла.

Посібник складається з чотирьох розділів. Перший розділ містить загальні відомості про периферійні інтерфейси: їх класифікацію, топології з'єднання пристроїв, паралельні інтерфейси, порівняння паралельних і послідовних інтерфейсів, огляд інтерфейсів CAN і 1-Wire, а також відомості про драйвери периферійних пристроїв і рівні їх програмної підтримки. Наступні три розділи присвячено послідовним інтерфейсам: асинхронному інтерфейсу RS-232C (UART), універсальній послідовній шині USB та синхронним інтерфейсам I²C і SPI. Кожен із цих розділів містить теоретичні відомості та практичні завдання. Опис практичного завдання містить мету, постановку завдання, технологію виконання та покрокові вказівки з прикладами команд і очікуваних результатів. Кожен розділ завершують питання для самоперевірки та тестові завдання. Практичні завдання виконуються без спеціального обладнання – з використанням віртуальних послідовних портів, імітаційних моделей пристроїв і симулятора мікроконтролерів Wokwi; за наявності обладнання їх можна виконати і на реальних платах.

Опрацьовувати посібник рекомендовано послідовно: спочатку теоретичні відомості розділу, потім практичні завдання, наприкінці – питання для самоперевірки та тестові завдання. Варіанти практичних завдань, вказівки щодо підготовки програмного середовища, вимоги до оформлення результатів і критерії оцінювання наведено в додатку А, тексти програм – у додатку Б (посилання на відповідні лістинги подано безпосередньо в тексті завдань), відповіді до тестових завдань – у додатку В. Основні терміни наведено в термінологічному словнику, а пошук матеріалу полегшують предметний та іменний покажчики.

Посібник продовжує і суттєво оновлює лабораторний практикум з дисципліни, виданий у 2021 році [2]: на основі цього видання та навчального посібника [1] підготовлено частину теоретичних відомостей до розділів 2 і 3. Автор висловлює щиру подяку Я. С. Парамуду за багаторічну роботу над курсом, матеріали якого стали основою цього посібника.

1. Загальні відомості про периферійні інтерфейси

1.1. Класифікація периферійних інтерфейсів

Периферійний інтерфейс – це сукупність ліній зв'язку, електричних, логічних і конструктивних характеристик, а також правил (протоколу) обміну, що забезпечують взаємодію центральної частини комп'ютерної системи з периферійними пристроями [1, 24]. До складу інтерфейсу входять апаратні засоби (лінії зв'язку, з'єднувачі, приймачі-передавачі, контролер) та програмні засоби – драйвер, який приховує особливості апаратури від прикладних програм. Різноманіття периферійних пристроїв зумовлює велику кількість інтерфейсів, які класифікують за низкою ознак (табл. 1).

Таблиця 1

Класифікація периферійних інтерфейсів

Ознака	Різновиди	Приклади
Спосіб передавання бітів	послідовний, паралельний	RS-232C, USB, I²C, SPI, CAN; Centronics, ISA, IDE
Синхронізація	асинхронний, синхронний	UART, RS-232C; SPI, I²C, USB, PCI Express
Напрям передавання	симплексний, напівдуплексний, дуплексний	давач → мікроконтролер; I²C, RS-485, CAN; RS-232C, SPI
Топологія	точка-точка, шина, зірка, дерево	RS-232C; I²C, CAN; USB з концентратором; ієрархія USB, PCI Express
Розподіл ролей вузлів	провідний – підпорядковані, рівноправні	I²C, SPI, USB (хост – пристрій); CAN
Розміщення	внутрішньоплатний, внутрішньоблочний, зовнішній	I²C, SPI; PCI Express, SATA; USB, RS-232C, RS-485

За способом передавання бітів розрізняють послідовні інтерфейси, у яких біти слова передаються по черзі однією лінією або однією диференційною парою, та паралельні, у яких усі біти слова передаються одночасно окремими лініями. За синхронізацією інтерфейси поділяють на асинхронні, у яких приймач синхронізується за службовими бітами кожного символу (UART), та синхронні, у яких тактовий сигнал передається окремою лінією (I²C, SPI) або відновлюється з потоку даних (USB, PCI Express). За напрямом передавання розрізняють симплексний режим (дані передаються лише в одному напрямку), напівдуплексний (в обох напрямках по черзі однією лінією) та дуплексний (одночасно в обох напрямках окремими лініями).

Усі інтерфейси, які досліджуються в практичних завданнях посібника, є послідовними, але разом вони охоплюють основні ознаки класифікації: RS-

232C (розділ 2) – асинхронний дуплексний інтерфейс з топологією точка-точка; USB (розділ 3) – синхронна шина (у версії USB 2.0 – напівдуплексна) з топологією «ієрархічна зірка» та розподілом ролей «хост – пристрій»; I²C (розділ 4) – синхронна напівдуплексна магістраль з провідним і підпорядкованими пристроями.

1.2. Топології з’єднання периферійних пристроїв

Спосіб з’єднання пристроїв між собою – топологія – визначає кількість ліній зв’язку, спосіб адресації, масштабованість системи та її стійкість до відмов [3, 25]. У периферійних підсистемах застосовують такі основні топології.

- **Точка-точка** (point-to-point, P2P) – пряме з’єднання двох пристроїв виділеною лінією без проміжних вузлів і адресації. Уся пропускна здатність лінії належить двом вузлам, затримка мінімальна, колізії відсутні, апаратна реалізація найпростіша. Недолік – погана масштабованість: для кожного нового пристрою потрібні окремий порт і контролер. Приклади – UART, RS-232C, SATA, окрема лінія PCI Express.
- **Шина** (bus) – усі пристрої під’єднано паралельно до спільної лінії; у шинах з централізованим керуванням (I²C, RS-485 з протоколом Modbus) кожен пристрій має унікальну адресу, а провідний пристрій почергово звертається до підпорядкованих; у мультимайстерних шинах (CAN) доступ до шини визначає арбітраж. Топологія потребує мінімальної кількості проводів і дозволяє легко додавати нові вузли, але обрив спільної лінії виводить з ладу всю шину, а зі збільшенням кількості вузлів знижується швидкість обміну. На кінцях довгих ліній (RS-485, CAN) встановлюють узгоджувальні резистори-термінатори. Приклади – I²C, RS-485 (Modbus), CAN.
- **Зірка** (star) – кожен пристрій під’єднано окремим каналом до центрального вузла (концентратора, комутатора); прямий обмін між периферійними вузлами відсутній. Пошкодження одного кабелю не впливає на інші пристрої, проте центральний вузол є єдиною точкою відмови. Приклади – USB, Ethernet.
- **Послідовний ланцюжок** (daisy chain) – вихід одного пристрою під’єднано до входу наступного, і кожен пристрій ретранслює дані далі. Топологія економить порти головного контролера, але відмова одного пристрою перериває зв’язок з усіма наступними. Приклади – DisplayPort MST, Thunderbolt, каскадне з’єднання мікросхем SPI.
- **Дерево** (tree, ієрархічна зірка) – ієрархічне поєднання зірок із кореневим вузлом на вершині. Приклади – ієрархія концентраторів USB (до 127 пристроїв), комутатори PCI Express.
- **Кільце** (ring) – послідовне з’єднання в замкнений контур, у якому пакет рухається в одному напрямку; подвійне кільце забезпечує

резервування у разі обриву. Приклади – промислові мережі PROFINET MRP, EtherCAT.

– **Коміркова мережа** (mesh) – вузли з'єднано багатьма зв'язками, а дані передаються через сусідні вузли (multi-hop) з динамічною маршрутизацією в обхід непрацюючих вузлів. Мережа самовідновлюється, проте її протоколи складніші, а кожна ретрансляція додає затримку. Приклади – бездротові мережі Zigbee, BLE Mesh, Thread.

Порівняння топологій наведено в табл. 2. На рівні друкованої плати вбудованих систем переважають з'єднання точка-точка (UART) і шини I²C та SPI завдяки мінімальній кількості провідників; у персональних комп'ютерах – зірка й дерево (USB, PCI Express); в енергетиці та промисловій автоматиці – шина RS-485 [26, 27] з протоколом Modbus для інверторів і лічильників та коміркові бездротові мережі для давачів. У практичних завданнях посібника досліджуються топології точка-точка (RS-232C, UART – практичні завдання № 1, 2, 5), зірка (USB – практичне завдання № 3) та шина (I²C – практичні завдання № 4, 5).

Таблиця 2

Порівняння топологій з'єднання периферійних пристроїв

Топологія	Стійкість до відмов	Витрати кабелю	Типові інтерфейси
Точка-точка	висока (для однієї лінії)	низькі	UART, RS-232C, SATA, PCI Express
Шина	низька (обрив спільної лінії)	мінімальні	I²C, RS-485 (Modbus), CAN
Зірка	висока (залежить від центрального вузла)	високі	USB, Ethernet, Wi-Fi
Послідовний ланцюжок	низька (обрив ланцюга)	дуже низькі	DisplayPort MST, Thunderbolt, SPI (каскад)
Дерево	середня (залежить від гілки)	середні	USB (ієрархія концентраторів), PCI Express
Коміркова мережа	дуже висока (самовідновлення)	– (бездротова)	Zigbee, BLE Mesh, Thread

1.3. Паралельні інтерфейси

У паралельному інтерфейсі всі біти слова передаються одночасно окремими лініями зв’язку, тому за один цикл обміну передається 8, 16 або 32 біти [1, 24]. Паралельна шина складається з трьох груп ліній (рис. 1):

– шина даних (Data Bus) – двонаправлена сукупність ліній D0–D7 або D0–D15, якою за один цикл передається ціле слово між контролером і периферійним пристроєм;
– шина адреси (Address Bus) – однонаправлена шина для вибору комірки пам’яті або внутрішнього регістра периферійної мікросхеми; у простих пристроях її заміщують одна-дві лінії (A0, A1);
– шина керування (Control Bus) – стробувальні сигнали вибору кристала /CS (Chip Select), зчитування /RD (Read), записування /WR (Write) та вибору типу інформації «команда / дані» RS (D/C), які визначають дію на шині; риска перед назвою сигналу означає, що активним є низький рівень.



Рис. 1. Структура паралельної шини

Обмін паралельною шиною може бути синхронним або асинхронним. За синхронного обміну (стробування) контролер виставляє дані на шину і формує імпульс /WR або /RD, а периферійний пристрій фіксує дані фронтом цього імпульсу; тривалість імпульсу регламентує документація на мікросхему, а для повільних пристроїв контролер вставляє цикли очікування (wait states). Відповіді від периферійного пристрою при цьому не передбачено. За асинхронного обміну (квитування, handshake) контролер формує строб, а периферійний пристрій відповідає сигналом готовності (BUSY, ACK). Такий механізм гарантує надійний обмін з пристроями різної швидкодії; класичний приклад – паралельний порт принтера Centronics (LPT).

Для під’єднання периферійних мікросхем до мікроконтролерів поширені два протоколи паралельної шини. У протоколі Intel 8080 використовують окремі стробувальні сигнали зчитування /RD і записування /WR з активним низьким рівнем та сигнал вибору кристала /CS (рис. 2). У протоколі Motorola 6800 використовують один сигнал вибору напряму обміну R/W та окремий сигнал дозволу передавання E (Enable). Контролери графічних дисплеїв (ILI9341, ST7789, SSD1963) підтримують обидва протоколи, а лінією RS (Register Select) розрізняють, чи є байт на шині D0–D7 командою налаштування (RS = 0), чи даними зображення (RS = 1).

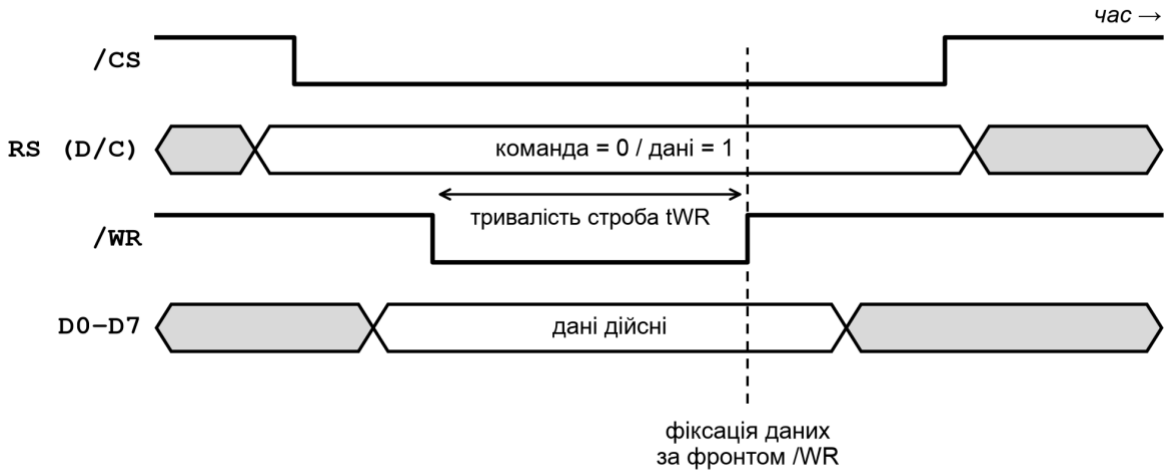


Рис. 2. Часова діаграма циклу записування паралельної шини типу Intel 8080

Паралельні інтерфейси потребують великої кількості виводів. Так, для під’єднання 8-бітного паралельного графічного дисплея потрібно щонайменше 16 контактів: 8 ліній даних D0–D7, 5 ліній керування (/CS, RS, /WR, /RD, /RST) та 3 лінії живлення і підсвічування (VCC, GND, LED), тобто половину контактів займають лише дані.

Історично паралельні інтерфейси розвивалися від принтерного порту Centronics (LPT) і внутрішньої шини ISA персональних комп’ютерів 1980-х років до паралельних шлейфів накопичувачів IDE (PATA, 40 і 80 провідників) та 32-бітної шини плат розширення PCI 1990-х років. У 2000-х роках у комп’ютерах їх витіснили швидкісні послідовні диференційні лінії SATA, PCI Express та USB. Нині паралельні шини незамінні на коротких відстанях у межах друкованої плати: для під’єднання зовнішньої пам’яті SRAM, NOR Flash, PSRAM через апаратні контролери (наприклад, FSMC/FMC у мікроконтролерах STM32), для швидкісних дисплеїв (паралельний порт LCD/Camera мікроконтролера ESP32-S3) та для сенсорів камер з інтерфейсом DVP (Digital Video Port: 8 ліній даних, піксельний тактовий сигнал PCLK і сигнали синхронізації HSYNC, VSYNC). Шина розрядністю 16 біт дозволяє передати

колір пікселя у форматі RGB565 за один цикл; кадр дисплея з роздільністю 320 × 240 пікселів займає 320 · 240 · 2 = 153 600 байт. Подальшим розвитком є компактні інтерфейси Octal SPI (xSPI) та HyperBus, які поєднують невелику кількість ліній даних з високою тактовою частотою.

1.4. Порівняння паралельних і послідовних інтерфейсів

Основні відмінності паралельних і послідовних інтерфейсів наведено в табл. 3.

Таблиця 3

Порівняння паралельних і послідовних інтерфейсів

Характеристика	Паралельний інтерфейс (8080, IDE)	Послідовний інтерфейс (SPI, PCI Express)
Кількість провідників	багато (від 8 до 40 і більше ліній)	мало (від 1 до 4 ліній або диференційних пар)
Кількість бітів за такт	8, 16 або 32	1 на лінію (у PCI Express лінії об’єднують x1…x16)
Вплив часового перекосу	критичний, обмежує частоту і довжину лінії	мінімальний або компенсується автоматично
Дальність зв’язку	від десятків сантиметрів (на платі) до кількох метрів (кабель)	значна (до десятків метрів і кілометрів)
Складність трасування плати	висока (потрібне вирівнювання довжин доріжок)	низька

Попри більшу кількість бітів за такт, підвищенню частоти паралельних шин перешкоджають фізичні явища:

- часовий перекіс між лініями (skew) – через різницю довжин доріжок і паразитні ємності біти одного слова надходять до приймача неодночасно, що на високих частотах спотворює слово;
- перехресні завади (crosstalk) – десятки паралельних ліній взаємодіють як трансформатори, і круті фронти сигналу на одній лінії наводять хибні імпульси на сусідніх;
- дефіцит виводів – 8- або 16-бітна шина займає значну частину виводів мікроконтролера, змушуючи обирати дорожчі корпуси з більшою кількістю виводів;
- шум комутації (ground bounce) – одночасне перемикання 16 або 32 виходів створює різкий кидок струму в колах живлення та спільного проводу мікросхеми.

Послідовні інтерфейси позбавлені більшості цих обмежень: в одній лінії або диференційній парі немає перекосу між бітами слова, а синхронізація за тактовим сигналом або за самим потоком даних дозволяє підвищувати частоту до гігагерців. Тому в сучасних комп’ютерних системах для зовнішніх і більшості внутрішніх з’єднань використовують послідовні інтерфейси, яким присвячено наступні розділи та практичні завдання посібника.

1.5. Огляд інших послідовних інтерфейсів вбудованих систем

Крім інтерфейсів, яким присвячено практичні завдання, у вбудованих і промислових системах поширені ще кілька послідовних інтерфейсів, принципи яких корисно знати для порівняння.

CAN (Controller Area Network) – послідовна магістральна шина, розроблена компанією Bosch для автомобільної електроніки, яку нині широко застосовують також у промисловій автоматиці, робототехніці та медичному обладнанні [1, 19]. Фізичний рівень CAN – диференційна пара CAN_H/CAN_L з узгоджувальними резисторами 120 Ом на обох кінцях лінії. Стан шини може бути домінантним (логічний 0) або рецесивним (логічна 1): якщо хоча б один вузол передає домінантний біт, на шині встановлюється домінантний стан. На відміну від RS-485 з протоколом Modbus, де обмін ініціює лише провідний пристрій, CAN є мультимайстерною шиною: передавання може розпочати будь-який вузол. Кадр CAN містить не адресу отримувача, а ідентифікатор повідомлення (11 або 29 бітів), який водночас визначає його пріоритет. Якщо кілька вузлів почали передавання одночасно, виконується побітовий неруйнівний арбітраж: вузол, який передає рецесивний біт, але бачить на шині домінантний, припиняє передавання, і повідомлення з меншим ідентифікатором, тобто з вищим пріоритетом, передається без втрат і затримок. Для підтримання синхронізації приймачів після п’яти однакових бітів передавач вставляє біт протилежного значення (bit stuffing). Класичний CAN 2.0 забезпечує швидкість до 1 Мбіт/с і до 8 байтів даних у кадрі, а CAN FD – до 64 байтів даних і вищу швидкість у фазі передавання даних (типово до 5–8 Мбіт/с). У мікроконтролерах ESP32 контролер CAN має назву TWAI, а в операційній системі Linux мережі CAN доступні через інтерфейс SocketCAN.

1-Wire – послідовний інтерфейс компанії Dallas Semiconductor (нині – частина Analog Devices), у якому для обміну даними і, за потреби, для живлення підпорядкованих пристроїв використовують одну сигнальну лінію та спільний провід. Провідний пристрій формує часові інтервали (слоти): тривалість імпульсу низького рівня визначає значення переданого біта, а підпорядкований пристрій відповідає, утримуючи лінію в низькому стані у визначений момент слоту. Кожна мікросхема має унікальний 64-бітний серійний номер, тому на одній лінії може працювати багато пристроїв, які провідний пристрій знаходить процедурою пошуку. Швидкість обміну невисока (близько 16 кбіт/с у стандартному режимі), проте мінімальна кількість проводів зробила 1-Wire поширеним для давачів температури (наприклад, DS18B20), електронних ідентифікаторів і мікросхем автентифікації акумуляторів та картриджів.

У спеціалізованих галузях використовують власні послідовні інтерфейси: MIL-STD-1553B – керовану мультиплексну шину авіаційної та військової техніки, у якій контролер шини почергово звертається до абонентів; ARINC 429 – інтерфейс бортового обладнання літаків; SpaceWire – інтерфейс космічних апаратів. Для більшості задач комп’ютерної інженерії достатньо знати їх призначення та місце в класифікації (табл. 1).

1.6. Драйвери периферійних пристроїв і рівні програмної підтримки

Драйвер периферійного пристрою – програмний компонент, який забезпечує взаємодію інших програм з апаратурою: він приховує особливості регістрової моделі, протоколу обміну та електричного інтерфейсу пристрою і надає уніфікований програмний інтерфейс. Завдяки драйверам прикладна програма звертається до послідовного порту, накопичувача чи давача однаковими засобами незалежно від того, яка саме мікросхема реалізує пристрій [1, 3].

Програмна підтримка периферійного пристрою має багаторівневу структуру (рис. 3). Найнижчий рівень утворюють вбудоване програмне забезпечення (firmware) і регістри самого пристрою; далі – апаратний контролер інтерфейсу (UART, контролер USB-хоста, контролер I²C чи SPI); над ним працює драйвер, який програмує контролер і виконує обмін. Операційна система надає доступ до драйвера через стандартний програмний інтерфейс – системні виклики та файли пристроїв; прикладні бібліотеки (pySerial, libusb, HID API) спрощують використання цього інтерфейсу, а на вершині структури знаходиться прикладна програма.



Рис. 3. Рівні програмної підтримки периферійного пристрою та їх відповідність практичним завданням посібника

Обмін драйвера з пристроєм організовують одним із трьох способів (табл. 4). Під час програмного опитування (polling) процесор періодично зчитує регістр стану пристрою і, виявивши готовність, передає наступну порцію

даних; спосіб найпростіший, але марно витрачає процесорний час. Під час обміну з перериваннями (interrupts) пристрій сам повідомляє процесор про готовність сигналом переривання, а драйвер опрацьовує подію в обробнику переривання; між подіями процесор виконує інші задачі. Під час прямого доступу до пам'яті (DMA, Direct Memory Access) контролер DMA пересилає блок даних між пристроєм і пам'яттю без участі процесора, а переривання сигналізує лише про завершення пересилання. Цей спосіб застосовують для швидкісних пристроїв – накопичувачів, мережевих адаптерів, дисплеїв, а також для інтерфейсу SPI під час передавання великих обсягів даних.

Таблиця 4

Способи обміну драйвера з периферійним пристроєм

Спосіб	Ініціатор передавання	Навантаження на процесор	Типове застосування
Програмне опитування	процесор (циклічна перевірка стану)	високе	прості пристрої, ініціалізація, мікроконтролери без ОС
Переривання	пристрій (сигнал готовності)	помірне	UART, клавіатури, давачі з сигналом готовності
Прямий доступ до пам'яті (DMA)	контролер DMA за завданням драйвера	мінімальне	накопичувачі, мережеві адаптери, дисплеї, SPI

У сучасних операційних системах драйвери виконуються в просторі ядра (kernel space) – привілейованому режимі з прямим доступом до апаратури, а прикладні програми – у просторі користувача (user space), де доступ до апаратури можливий лише через системні виклики. У Linux пристрої подаються файлами в каталозі /dev: послідовні порти – `/dev/ttyS*`, `/dev/ttyUSB*`, `/dev/ttyACM*`, шини I²C – `/dev/i2c-*`, пристрої SPI – `/dev/spidev*`. Прикладна програма відкриває такий файл (системний виклик open), читає й записує дані (read, write) і змінює параметри обміну спеціальним викликом ioctl. Драйвери Linux поділяють на символьні (послідовні порти, давачі), блокові (накопичувачі) і мережеві [20, 22]. У Windows драйвери розробляють на основі бібліотеки Windows Driver Frameworks (WDF), яка має версії для режиму ядра (KMDF) і режиму користувача (UMDF), а послідовні порти подаються іменами COM1, COM2 тощо [21].

Для поширених класів пристроїв USB операційні системи містять стандартні класові драйвери: HID (клавіатури, миші, ігрові маніпулятори), CDC ACM (віртуальні COM-порти мікроконтролерів з вбудованим інтерфейсом USB, наприклад ESP32-S3, RP2040, STM32), Mass Storage (накопичувачі), Audio і Video. Поширені USB-UART адаптери на мікросхемах FTDI, CH340 і CP2102 до класу CDC ACM не належать: для них використовують драйвери

виробників (у Linux – ftdi_sio, ch341, cp210x), і в системі вони подаються як `/dev/ttyUSB*`, тоді як пристрої CDC ACM – як `/dev/ttyACM*`. Пристрій, що відповідає специфікації класу, працює без встановлення окремого драйвера: операційна система обирає класовий драйвер за дескриптором інтерфейсу (підрозділ 3.2). Для пристроїв, які не належать до стандартних класів, драйвер можна реалізувати в просторі користувача за допомогою бібліотеки libusb, не розробляючи драйвера ядра.

У мікроконтролерних системах без операційної системи поділу на простір ядра і простір користувача немає: драйвер – це модуль прошивки, який безпосередньо звертається до регістрів контролера інтерфейсу або використовує бібліотеку абстрагування апаратури (HAL, Hardware Abstraction Layer) виробника, наприклад STM32 HAL чи ESP-IDF. У середовищі MicroPython роль такої бібліотеки виконує модуль machine з класами UART, I2C, SPI і Pin, а драйвер периферійної мікросхеми пишуть як звичайний модуль мовою Python – саме так побудовано драйвер давача `bmp180.py` практичних завдань № 4 і 5 [15]. Периферійні інтерфейси та способи програмного доступу до них на типових платформах наведено в табл. 5.

Таблиця 5

Периферійні інтерфейси та програмний доступ до них на типових платформах

Платформа	Клас	Інтерфейси	Програмний доступ
ПК (Windows, Linux)	універсальний комп’ютер	USB, PCI Express, SATA, Ethernet; UART через USB-UART адаптер	COMn, `/dev/ttyUSB*`, класові драйвери USB, libusb
Raspberry Pi	одноплатний комп’ютер з ОС Linux	USB, Ethernet, UART, I²C і SPI на роз’ємі GPIO	/dev/serial0 (ttyAMA0 або ttyS0 залежно від моделі), `/dev/i2c-*`, `/dev/spidev*`
ESP32	мікроконтролер з Wi-Fi і Bluetooth	UART, I²C, SPI, CAN (TWAI), USB у моделях ESP32-S2/S3	ESP-IDF, Arduino, MicroPython (модуль machine)
STM32	мікроконтролер	USART, I²C, SPI, CAN, USB device	бібліотеки STM32 HAL / LL, CMSIS

Практичні завдання посібника охоплюють різні рівні цієї структури (рис. 3). У практичних завданнях № 1–3 програми працюють у просторі користувача і звертаються до пристроїв через інтерфейс операційної системи та прикладні бібліотеки; у практичному завданні № 4 розробляється драйвер периферійної мікросхеми у прошивці мікроконтролера; у практичному завданні № 5 усі рівні – від драйвера давача до прикладної програми опрацювання телеметрії –

поєднуються в одному вузлі. Розроблення драйверів ядра операційної системи виходить за межі посібника, проте розуміння розподілу функцій між рівнями є необхідною основою для такої роботи.

Питання для самоперевірки

1. Дайте визначення периферійного інтерфейсу. Які апаратні та програмні засоби входять до його складу?
2. За якими ознаками класифікують периферійні інтерфейси? Наведіть приклади інтерфейсів для кожної ознаки.
3. Чим відрізняються симплексний, напівдуплексний і дуплексний режими обміну?
4. Чим синхронний інтерфейс відрізняється від асинхронного? Як синхронізується приймач у кожному з них?
5. Порівняйте топології точка-точка, шина та зірка за стійкістю до відмов і витратами кабелю.
6. Яку топологію має шина USB і чому її називають ієрархічною зіркою?
7. Назвіть групи ліній паралельної шини та поясніть їх призначення.
8. Чим відрізняються синхронний (стробування) і асинхронний (квитування) обмін паралельною шиною?
9. Чим відрізняються протоколи паралельної шини Intel 8080 і Motorola 6800? Для чого в контролерах дисплеїв використовують лінію RS?
10. Які фізичні явища обмежують тактову частоту паралельних шин?
11. Чому послідовні інтерфейси витіснили паралельні в сучасних комп’ютерах? Де паралельні шини застосовують досі?
12. Як виконується арбітраж на шині CAN і чому його називають неруйнівним?
13. Чим шина CAN відрізняється від RS-485 з протоколом Modbus за розподілом ролей вузлів?
14. Які особливості інтерфейсу 1-Wire дозволяють під’єднати кілька пристроїв до однієї лінії?
15. Яку роль виконує драйвер периферійного пристрою? Назвіть рівні програмної підтримки периферійного пристрою.
16. Порівняйте способи обміну з периферійним пристроєм: програмне опитування, переривання, прямий доступ до пам’яті.
17. Чим простір ядра відрізняється від простору користувача? Як прикладна програма в Linux звертається до послідовного порту?
18. Для чого операційні системи містять класові драйвери USB? Наведіть приклади класів пристроїв.
19. Чим драйвер периферійної мікросхеми в прошивці мікроконтролера відрізняється від драйвера ядра операційної системи?

Тестові завдання

Оберіть одну правильну відповідь. Відповіді до тестових завдань наведено в додатку В.

1. Який із наведених інтерфейсів є паралельним?
   а) USB
   б) Centronics
   в) I²C
   г) RS-485
2. У напівдуплексному режимі дані передаються:
   а) лише в одному напрямку
   б) в обох напрямках по черзі
   в) в обох напрямках одночасно
   г) лише від хоста до пристрою
3. Яка топологія має єдину точку відмови в центральному вузлі?
   а) точка-точка
   б) зірка
   в) коміркова мережа
   г) подвійне кільце
4. Сигнал /WR паралельної шини типу Intel 8080 – це:
   а) сигнал вибору кристала
   б) сигнал вибору «команда / дані»
   в) строб зчитування
   г) строб записування з активним низьким рівнем
5. Основний чинник, що обмежує тактову частоту паралельних шин:
   а) часовий перекіс між лініями
   б) відсутність тактового сигналу
   в) мала кількість ліній
   г) використання диференційних пар
6. Під час арбітражу на шині CAN передавання продовжує повідомлення:
   а) з більшим ідентифікатором
   б) з більшою кількістю байтів даних
   в) з меншим ідентифікатором
   г) вузла, який першим увімкнувся
7. Скільки сигнальних ліній, крім спільного проводу, використовує інтерфейс 1-Wire?
   а) чотири
   б) дві
   в) три

г) одну

8. Синхронним інтерфейсом є:
   а) RS-232C
   б) SPI
   в) UART
   г) TTL-UART

2. Послідовний асинхронний інтерфейс RS-232C (UART)

Розділ присвячено послідовному асинхронному інтерфейсу RS-232C та його логічній основі – старт-стопному обміну UART, який реалізують контролери послідовних портів комп’ютерів і всіх сучасних мікроконтролерів. У практичних завданнях розділу досліджуються програми передавача і приймача, формат кадру та часові діаграми сигналів лінії зв’язку.

2.1. Інтерфейс RS-232C

RS-232C – стандарт послідовного асинхронного обміну між кінцевим обладнанням даних (DTE, Data Terminal Equipment – термінал, комп’ютер) та апаратурою передавання даних (DCE, Data Circuit-terminating Equipment – модем, перетворювач). Стандарт розробила асоціація EIA (редакцію C прийнято в 1969 р.), чинна редакція має позначення TIA-232-F [4]. Він визначає електричні рівні сигналів, призначення ліній і контактів з’єднувачів та часові параметри обміну [1, 2]. У сучасних персональних комп’ютерах COM-порти майже витіснив USB, проте той самий принцип старт-стопного обміну реалізують контролери UART усіх мікроконтролерів, а сам інтерфейс RS-232C досі поширений у промисловому, вимірювальному та енергетичному обладнанні.

За класифікацією (підрозділ 1.1) RS-232C є послідовним, дуплексним і асинхронним інтерфейсом з топологією точка-точка (його також називають радіальним). Біти символу передаються по черзі однією сигнальною лінією; з’єднуються лише два пристрої (топологія точка-точка); для передавання і приймання використовуються окремі лінії TXD і RXD, тому обмін в обох напрямках може відбуватися одночасно; тактовий сигнал не передається, а приймач синхронізується за стартовим бітом кожного символу. Приклад використання з’єднувача послідовного порту показано на рис. 4.

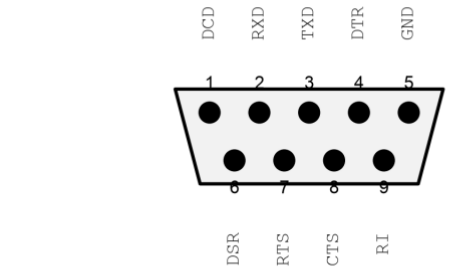


Рис. 4. З’єднувач DB9P послідовного порту (COM-порту) комп’ютера

Синхронізацію обміну забезпечує старт-стопний формат символу. Перед полем даних передається стартовий біт (логічний 0), який повідомляє приймач про початок символу; поле даних містить від 5 до 8 бітів; за ним може йти біт парності, а завершують символ один, півтора або два стопові біти (логічна 1). Приймач, виявивши перехід лінії до стартового біта, відлічує такти за власним генератором, налаштованим на ту саму швидкість, і зчитує кожен біт посередині його інтервалу. Службові біти займають щонайменше 20 % кадру,

тобто збільшують тривалість передавання кожного символу щонайменше на 25 % (підрозділ 2.5).

Символ зазвичай містить код ASCII. Наприклад, латинська літера A має семибітовий код 1000001; на рис. 5 зображено її передавання рівнями ТТЛ (*а*) та рівнями лінії RS-232C (*б*), де сигнал інвертовано. Для 8-бітових кодувань використовують 8-бітове поле даних. Під час контролю на непарність (odd) біт парності доповнює кількість одиниць у полі даних до непарного числа, під час контролю на парність (even) – до парного; так приймач виявляє помилку в одному біті (і загалом будь-яку непарну кількість помилок), проте не виявляє помилок у двох бітах.

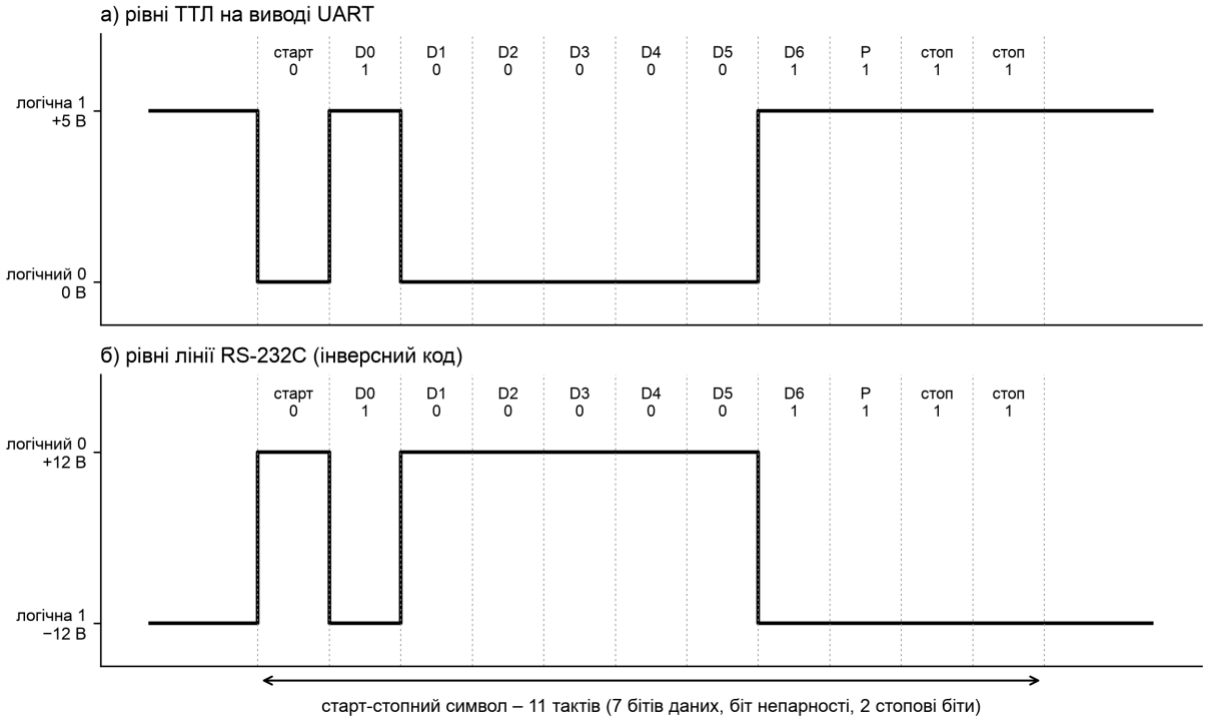
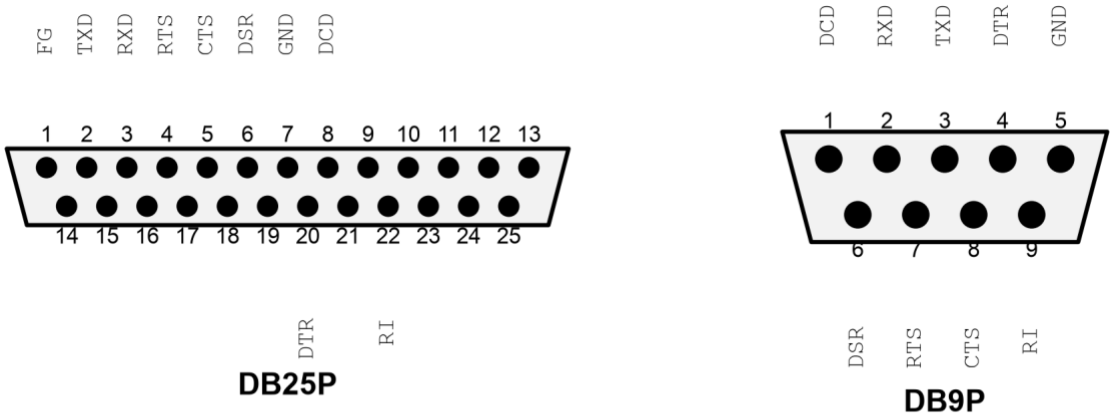


Рис. 5. Часові діаграми передавання символу «A» рівнями ТТЛ (а) та рівнями лінії RS-232C (б)

Стандарт передбачає 25-контактний (DB25) та 9-контактний (DB9) з’єднувачі (рис. 6). Більшість контактів DB25 у послідовному порту не використовується, тому в персональних комп’ютерах COM-порт зазвичай має з’єднувач DB9; контакт 1 з’єднувача DB25 інколи використовують для під’єднання захисного екрана кабелю. Призначення сигналів наведено в табл. 6.



Вигляд з боку контактів вилки; позначено задіяні сигнали інтерфейсу RS-232C

Рис. 6. Розташування контактів з’єднувачів DB25P та DB9P

Таблиця 6

Призначення контактів з’єднувачів DB25P та DB9P

Найменування	Контакт (з’єднувач DB25P)	Контакт (з’єднувач DB9P)	Напрям	Опис
DCD	8	1	IN	Carrier Detect (Визначення несучої)
RXD	3	2	IN	Receive Data (Дані, що приймаються)
TXD	2	3	OUT	Transmit Data (Дані, що передаються)
DTR	20	4	OUT	Data Terminal Ready (Готовність терміналу)
GND (SG)	7	5	-	System Ground (Сигнальна земля)
DSR	6	6	IN	Data Set Ready (Готовність даних)
RTS	4	7	OUT	Request to Send (Запит на відправку)
CTS	5	8	IN	Clear to Send (Готовність прийому)
RI	22	9	IN	Ring Indicator (Індикатор викликів)
FG	1	-	-	Захисне заземлення, екран

Сигнали інтерфейсу поділяють на дві групи. Шину даних утворюють лінії передавання TXD, приймання RXD та сигнальна земля GND (SG); шину керування – лінії RTS, CTS, DTR, DSR, DCD та RI, які використовують для апаратного керування потоком і керування модемом. Усі сигнали задіюють лише під час з’єднання комп’ютера з модемом. Якщо керування потоком

реалізоване програмно, достатньо трипровідного зв’язку (TXD, RXD, GND), а лінії керування на кожному з’єднувачі за потреби з’єднують перемичками, щоб програми, які перевіряють сигнали готовності, працювали коректно (рис. 7); для передавання лише в один бік вистачає ліній TXD і GND.

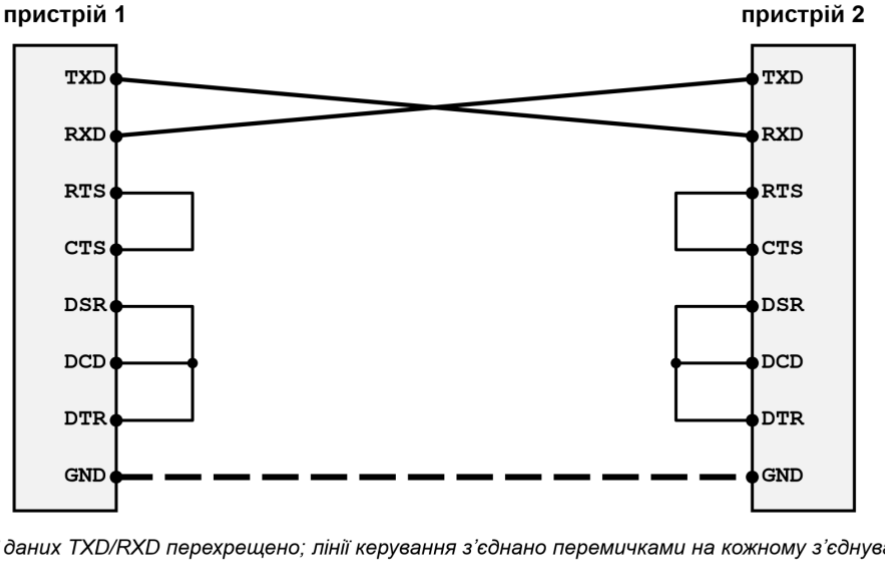


Рис. 7. Схема трипровідного (нуль-модемного) з’єднання з перемичками ліній керування

Формат символу (кількість бітів даних, тип контролю парності, кількість стопових бітів) і швидкість обміну задають під час налаштування порту, і вони мають бути однаковими на обох кінцях лінії (рис. 8). Тривалість одного такту обернена до швидкості обміну. Розбіжність частот генераторів передавача і приймача не повинна перевищувати кількох відсотків (для формату 8N1 – близько 5 % сумарно), інакше наприкінці символу приймач зчитуватиме біти не посередині їх інтервалів і виникатимуть помилки кадрування.

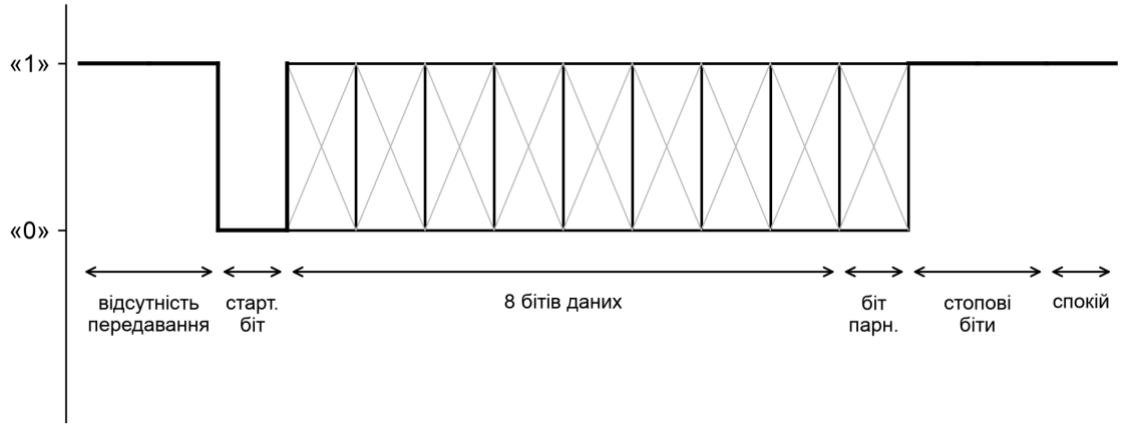


Рис. 8. Формат слова у логічних рівнях для RS-232C

На лінії RS-232C дані передаються в інверсному коді: логічній одиниці відповідає від’ємна напруга, логічному нулю – додатна (рис. 9). Передавач формує логічний нуль напругою від +5 до +15 В, а логічну одиницю – від −5 до −15 В; приймач розпізнає рівні вже від ±3 В, а діапазон від −3 до +3 В є зоною невизначеності. Запас між рівнями передавача і порогами приймача забезпечує завадостійкість лінії довжиною до 15 м. Типова напруга на лінії становить ±12 В, а за нестабілізованого живлення може сягати ±25 В.

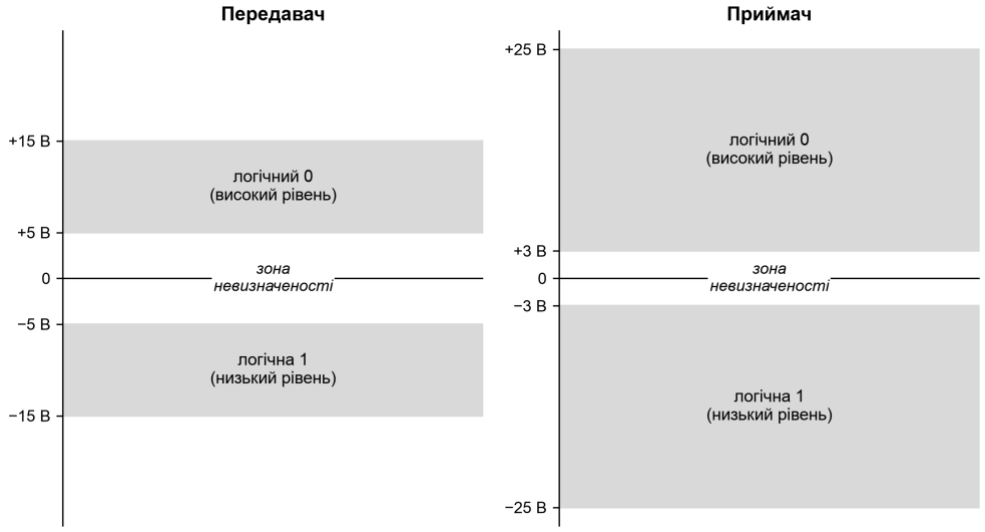


Рис. 9. Рівні сигналів RS-232C на передавальному і приймальному кінцях лінії зв’язку

Стандартні швидкості обміну становлять 50, 75, 110, 150, 300, 600, 1200, 2400, 4800, 9600, 19 200, 38 400, 57 600 та 115 200 біт/с. У персональних комп’ютерах архітектури IBM PC послідовні порти мали стандартні ресурси: COM1 – адреси 3F8h…3FFh і переривання IRQ4, COM2 – адреси 2F8h…2FFh і переривання IRQ3; порти COM3 (3E8h…3EFh) та COM4 (2E8h…2EFh) зазвичай використовували ті самі переривання, що й COM1 і COM2. Сучасні операційні системи приховують ці ресурси за драйвером і подають порти як пристрої COMn (Windows) або `/dev/ttyS*`, `/dev/ttyUSB*` (Linux).

2.2. Керування потоком даних та апаратні засоби інтерфейсу

Якщо приймач не встигає опрацьовувати дані, передавач потрібно тимчасово зупинити. Для цього застосовують керування потоком даних (flow control) – апаратне або програмне [1, 2].

Апаратне керування потоком (hardware flow control) використовує лінії RTS (Request to Send – запит на передавання) і CTS (Clear to Send – готовність до приймання). Передавач починає передавати черговий символ лише за активного сигналу CTS; символ, передавання якого вже розпочалося, завершується, тому цілісність даних зберігається. Реакція на зміну стану приймача мінімальна, проте потрібні додаткові лінії зв'язку. Якщо апаратне управління не використовують, лінії RTS і CTS на з'єднувачі з'єднують перемичкою, щоб передавач «бачив» постійну готовність приймача.

Програмне керування потоком (software flow control) обходиться лініями даних: приймач, буфер якого заповнюється, надсилає передавачу символ XOFF (13h), а коли знову готовий до приймання – символ XON (11h). Час реакції збільшується щонайменше на час передавання цього символу, тому приймач повинен мати запас буфера; крім того, коди XON і XOFF не можна передавати в самих даних без додаткового кодування. Завдяки економії ліній зв'язку програмне керування потоком широко застосовують на практиці.

Перетворення паралельного коду процесора в послідовний потік бітів і зворотне перетворення виконує універсальний асинхронний приймач-передавач UART (Universal Asynchronous Receiver/Transmitter). У персональних комп'ютерах історично використовували сумісні на рівні регістрів мікросхеми i8250, 16450 та 16550A; остання має буфери FIFO на 16 байтів для передавання і приймання, що зменшує кількість переривань процесора. Сьогодні вузли UART вбудовано в усі мікроконтролери (ESP32, STM32) і в мікросхеми перетворювачів USB-UART.

Програмну модель UART 16550A утворюють регістри, які займають вісім суміжних адрес відносно базової адреси порту; доступ до частини з них визначає біт DLAB (біт 7 регістра керування лінією LCR). Через ці регістри задають дільник частоти для потрібної швидкості обміну, кількість бітів даних (5–8), тип контролю парності та кількість стопових бітів, а також зчитують стан приймача і передавача. Для налаштування швидкості драйвер встановлює DLAB = 1, записує дільник частоти в регістри DLL і DLM, після чого скидає DLAB = 0 і задає формат кадру в регістрі LCR; лише після цього порт переходить у режим основної роботи – передавання і приймання даних через регістри THR і RBR. Саме ці параметри налаштовує драйвер послідовного порту; у практичних завданнях ними керують засобами бібліотеки pySerial [14].

Вузол UART формує на своїх виводах логічні рівні ТТЛ (0 В і +5 В) або рівні КМОН 3,3 В (LVTTL), тоді як лінія RS-232C використовує двополярні рівні близько ±12 В. Для узгодження цих рівнів застосовують перетворювачі, розглянуті в підрозділі 2.3.

2.3. TTL-UART, RS-232C та RS-485

Сам вузол UART мікроконтролера або мікросхеми послідовного порту формує на виводах TXD/RXD рівні ТТЛ: логічний нуль – близько 0 В, логічна одиниця – напруга живлення логіки (+5 В, а в сучасних мікроконтролерах ESP32, STM32 – +3,3 В). Такий варіант інтерфейсу називають TTL-UART. Від RS-232C він відрізняється лише електричними рівнями та полярністю сигналу: часова структура кадру – стартовий біт, поле даних, біт парності, стопові біти – залишається тією самою (рис. 5).

Для з'єднання TTL-UART з лінією RS-232C застосовують перетворювач рівнів, наприклад мікросхему MAX232, яка з однополярного живлення +5 В формує двополярну напругу близько ±10 В та інвертує сигнал; для живлення від 3,3 В використовують її аналог MAX3232 (3,0–5,5 В). Пряме з'єднання виводів TXD/RXD мікроконтролера з контактами з'єднувача DB9 неприпустиме: напруга ±12 В лінії RS-232C пошкоджує вхід логіки, розрахований на 0…+3,3 В.

У промислових системах та енергетиці (інвертори, системи керування акумуляторами BMS, лічильники електроенергії) поряд з RS-232C широко застосовують інтерфейс RS-485. Він використовує диференційну пару ліній A і B: логічний стан визначається різницею потенціалів між лініями, а не напругою відносно спільного проводу, завдяки чому синфазні завади придушуються, а довжина лінії може сягати 1200 м. До однієї пари під'єднують кілька пристроїв (типово до 32, а з приймачами-передавачами зниженого навантаження – до 128), тобто RS-485 є магістральним інтерфейсом [5]. Порівняння фізичних рівнів послідовного асинхронного обміну наведено в табл. 7.

Таблиця 7

Порівняння фізичних рівнів послідовного асинхронного обміну

Параметр	TTL-UART	RS-232C	RS-485
Рівні сигналу	0 В / +3,3 В або +5 В	±(5…15) В, інверсний код	диференційна напруга між A і B ±(1,5…6) В
Режим обміну	дуплексний (окремі TX, RX)	дуплексний (окремі TXD, RXD)	зазвичай напівдуплексний (одна пара A/B); чотирипровідний варіант – дуплексний
Дальність	до 0,5–1 м (у межах плати)	до 15 м	до 1200 м
Завадостійкість	низька	середня	висока
Топологія	точка-точка	точка-точка	магістраль, до 32–128 пристроїв

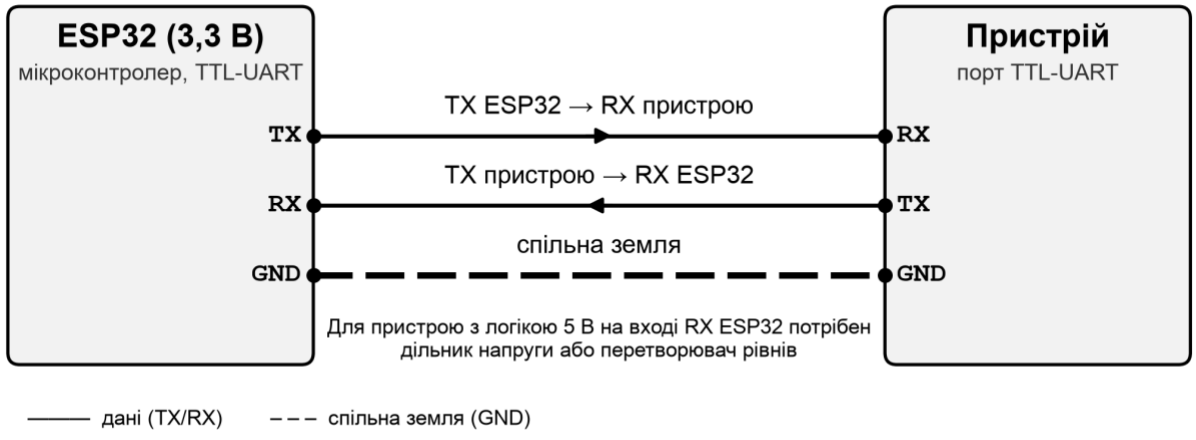
Оскільки пара A/B одна на всю магістраль, передавач кожного вузла більшість часу вимкнений (перебуває у високоімпедансному стані) і вмикається лише на час передавання кадру. Керує цим окремий сигнал DE/RE (Driver

Enable / Receiver Enable), який мікроконтролер переводить в активний стан перед передаванням і знімає одразу після передавання останнього стопового біта (сигнали DE з активним високим рівнем і /RE з активним низьким рівнем з’єднують разом). Сигнал слід знімати за ознакою завершення роботи зсувного регістра передавача (TEMT, transmission complete), а не за ознакою спорожнення буфера (THRE), інакше останній байт буде обрізано [5, 26]. Помилка в керуванні цим сигналом – типова причина того, що вузол передає дані, але не приймає відповіді. Вибір перетворювача залежно від типу порту пристрою наведено в табл. 8, схеми під’єднання мікроконтролера ESP32 – на рис. 10, 11, 12.

Таблиця 8

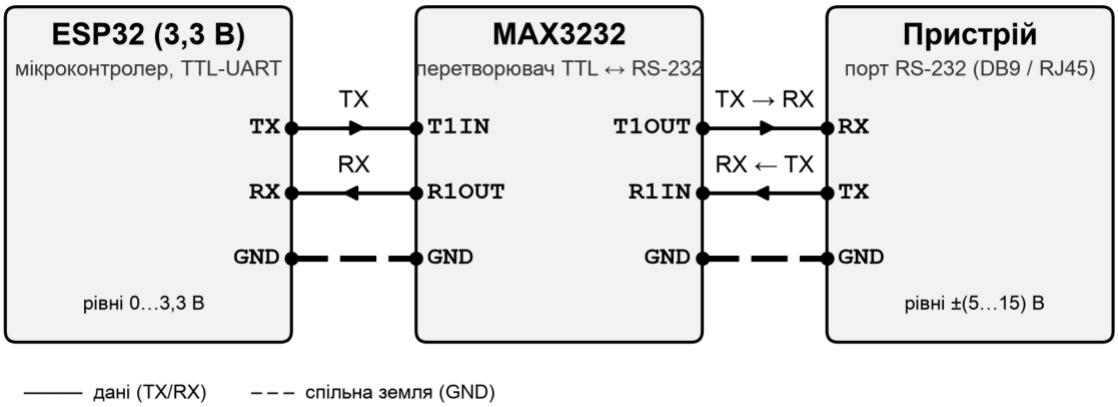
Перетворювачі рівнів для під’єднання мікроконтролера до порту пристрою

Порт пристрою	Що потрібно між портом і виводами мікроконтролера
TTL-UART	пряме з’єднання TX, RX, GND; за логіки 5 В – дільник напруги або перетворювач рівнів на вході RX мікроконтролера з живленням 3,3 В
RS-232C	перетворювач рівнів MAX3232 (аналог MAX232 з живленням від 3,0 до 5,5 В)
RS-485	приймач-передавач MAX485 / MAX3485 і додатковий вивід GPIO для керування сигналом DE/RE



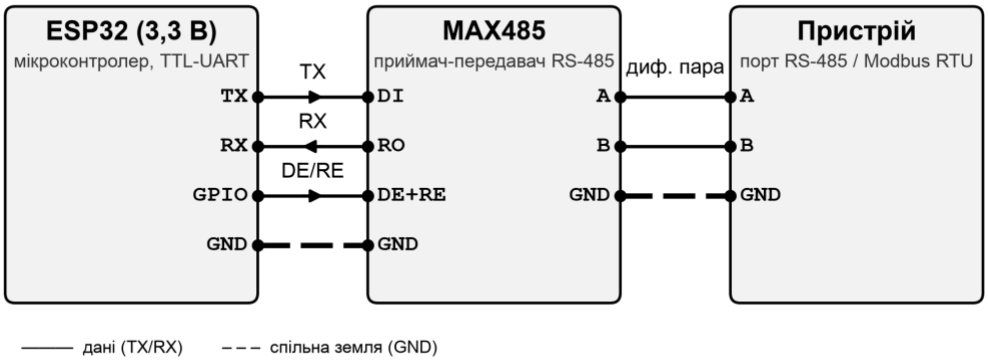
Лінії TX/RX перехрещують, землю з’єднують. Типовий формат PI30/Voltronic – 2400 біт/с, 8N1.

Рис. 10. Пряме під’єднання мікроконтролера ESP32 до порту TTL-UART



Повний дуплекс, сигнал DE/RE не потрібен. Пряме з'єднання виводів ESP32 з лінією RS-232 неприпустиме.

Рис. 11. Під'єднання мікроконтролера ESP32 до порту RS-232C через перетворювач рівнів MAX3232



Напівдуплекс: напрям обміну задає сигнал DE/RE з виводу GPIO. Типовий формат Modbus RTU – 9600 біт/с, 8E1 або 8N1.

Рис. 12. Під'єднання мікроконтролера ESP32 до магістралі RS-485 через приймач-передавач MAX485

Для TTL-UART і RS-232C лінії TX/RX перехрещують: вихід передавача одного вузла з'єднують із входом приймача іншого; спільний провід GND з'єднують обов'язково. Типові швидкості обміну реальних інверторів – 2400 біт/с (протокол PI30/Voltronic) та 9600 біт/с (Modbus RTU). Найчастішою причиною нечитабельних символів у порту є неузгоджена швидкість обміну або неправильне перетворення рівнів, а не несправність пристрою.

У практичних завданнях посібника фізичний перетворювач рівнів не потрібен. USB-UART адаптер (на мікросхемах FTDI, CH340, CP2102) містить перетворювач USB↔UART і виводить назовні саме рівні ТТЛ, а операційна система подає його як віртуальний COM-порт (COM3, /dev/ttyUSB0). Симулятор Wokwi у практичних завданнях № 4–5 моделює той самий TTL-UART, спрямовуючи його вивід у Serial Monitor.

2.4. Протокол Modbus RTU та рівні взаємодії

Поверх послідовного каналу RS-232C або RS-485 у промисловій автоматиці найчастіше використовують прикладний протокол Modbus RTU. Провідний пристрій (master) надсилає запит підпорядкованому пристрою (slave) з адресою від 1 до 247, а той повертає відповідь. Перший байт кадру – адреса пристрою, другий – код функції, далі – дані, наприкінці – контрольна сума CRC16 [6, 7]. Наприклад, запит

```
01 03 00 6B 00 01 F5 D6
```

означає: пристрою з адресою 1 (01) виконати функцію 03 (зчитування регістрів збереження) для одного (00 01) регістра з адресою 0x006B (00 6B); останні два байти F5 D6 – контрольна сума CRC16, передана молодшим байтом уперед. Зміст регістра визначає документація пристрою – наприклад, це може бути напруга акумулятора інвертора. Ні контролер UART, ні приймач-передавач MAX485 не «знають» змісту цих байтів: для них це лише вісім символів UART (за стандартом Modbus RTU – у форматі 8E1), переданих імпульсами напруги. Сусідні кадри Modbus RTU розділяють паузою тривалістю щонайменше 3,5 символу.

Щоб розмежувати відповідальність апаратних і програмних компонентів, послідовний канал зручно розглядати за моделлю взаємодії відкритих систем OSI. Для з’єднання «точка-точка» або магістралі з фіксованою адресацією основні функції виконують три рівні – фізичний, канальний і прикладний, а функції сеансового рівня і рівня представлення (таймаути, перетворення форматів) зазвичай реалізує прикладна програма (табл. 9). Мережевий і транспортний рівні не потрібні, оскільки немає маршрутизації, а цілісність кадру перевіряє контрольна сума.

Таблиця 9

Рівні моделі OSI для послідовного каналу RS-232C / RS-485

Рівень OSI	Компонент	Зона відповідальності
7. Прикладний	прошивка мікроконтролера, Modbus RTU, протокол PI30	зміст команд: запит телеметрії, розбирання відповіді
5–6. Сеансовий, представлення	таймаути очікування відповіді, перетворення ASCII / Hex, порядок байтів	фіксація втрати зв’язку, форматування чисел
3–4. Мережевий, транспортний	відсутні	маршрутизація на виділеній лінії не потрібна
2. Канальний	апаратний контролер UART, таймери пауз між кадрами	формування кадру 8N1, CRC16, керування DE/RE, виявлення помилок парності, кадрування та переповнення буфера
1. Фізичний	приймачі-передавачі	електричні рівні, захист від

Рівень OSI	Компонент	Зона відповідальності
	MAX3232, MAX485, проводи TX/RX, A/B, GND	електростатичного розряду

Приймач-передавач MAX3232 або MAX485 – суто компонент фізичного рівня: він не має регістрів і буферів, не аналізує даних, а лише перетворює рівні напруги. Помилки фізичного рівня (обрив спільного проводу, переплутані лінії A і B, пошкоджений перенапругою вхід 3,3 В) неможливо виправити програмно – їх виявляють мультиметром, осцилографом або логічним аналізатором. Тому під час відлагодження послідовного каналу дотримуються правила «знизу вгору»: спочатку перевіряють фізичний рівень (наявність спільної землі, правильні рівні сигналів), далі канальний (однакову швидкість обміну і формат кадру, перехрещення TX/RX) і лише потім прикладний (правильність команд і адрес регістрів).

2.5. Амплітудно-часова діаграма UART та кодування NRZI

Амплітудно-часова діаграма UART – послідовність логічних рівнів на лінії передачі протягом формування старт-стопного кадру кожного символу. У спокої (idle) лінія перебуває в стані логічної 1. Початок символу відмічається стартовим бітом (логічний 0), далі передаються біти даних (5–8), за потреби – біт парності, після чого – один або кілька стопових бітів (логічна 1). Між сусідніми символами може бути пауза довільної тривалості, протягом якої лінія перебуває в стані спокою; стандарт UART такої паузи не вимагає. Програми курсу для наочності вставляють між символами паузу тривалістю один такт.

Позначення формату кадру (8N1, 7E1, 8N2 тощо) визначає кількість біт даних, наявність і тип контролю парності та кількість стопових бітів. Тривалість одного такту обернено пропорційна швидкості обміну:

$$T_{такт} = 1 / \text{baudrate}$$

Час передавання повідомлення оцінюють через кількість символів, кількість тактів на символ (з урахуванням паузи між символами) і величину $T_{такт}$. На рис. 13 зображено приклад амплітудно-часової діаграми UART.

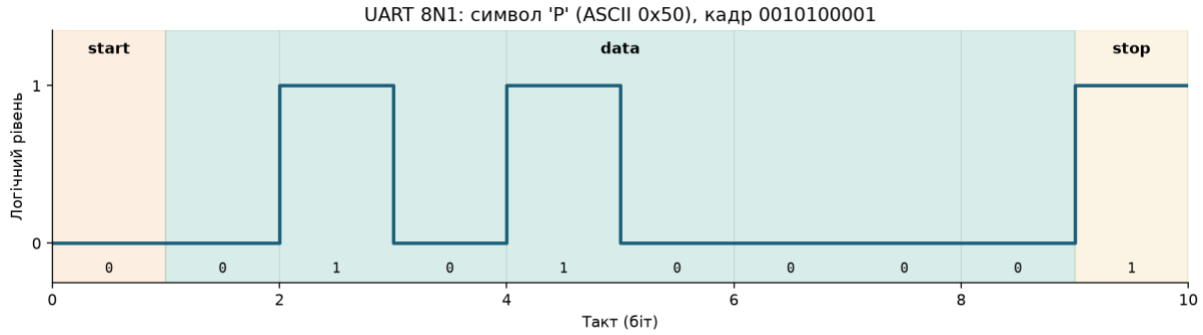


Рис. 13. Приклад амплітудно-часової діаграми UART

Слід розрізняти бодову і бітову швидкість. Бодова швидкість (baud rate) – кількість змін стану лінії за секунду; оскільки в UART один стан лінії кодує один біт, швидкість 9600 бод відповідає 9600 біт/с. Корисна швидкість передавання даних менша, бо частину тактів займають службові біти: у форматі 8N1 на кожний байт припадає 10 тактів (стартовий біт, 8 бітів даних, стоповий біт), тобто службові біти становлять 20 % кадру (тривалість передавання зростає на 25 % відносно корисних даних), і за швидкості 9600 біт/с передається близько 960 байт/с. Формат 9600 8N1 є найпоширенішим налаштуванням за замовчуванням для мікроконтролерів і багатьох промислових пристроїв, хоча специфікація Modbus RTU передбачає 11-бітовий символ – формат 8E1, а без контролю парності – 8N2 [7].

Біти поля даних передаються в лінію, починаючи з молодшого (LSB first). Для зручності читання на діаграмах і в програмах курсу (рис. 13, 16) біти даних символу зображено від старшого до молодшого, як у двійковому записі коду символу; фізичний порядок бітів на лінії – зворотний.

Слід розрізняти логічні рівні в моделі сигналу (0 і 1) та електричні рівні RS-232C на фізичній лінії (інверсний код, діапазони напруг на передавальному та приймальному кінцях), розглянуті вище.

Для послідовних високошвидкісних шин, зокрема інтерфейсу USB (USB), застосовують лінійне кодування, відмінне від старт-стопного кадру UART. У USB 2.0 в усіх режимах (Low-Speed, Full-Speed і High-Speed) використовується схема NRZI (Non-Return to Zero Inverted) [8]:

– при передачі логічного 0 рівень сигналу на лінії змінюється;
– при передачі логічної 1 рівень сигналу не змінюється.

Довга послідовність одиниць не створює переходів на лінії, що ускладнює підтримання синхронізації. Тому застосовують bit stuffing: після шести послідовних одиниць у потік вставляють біт 0, який забезпечує зміну рівня (рис. 14, 15). Наприклад, послідовність бітів 11111111 після вставлення набуває вигляду 111111011. Для кодів латинських літер A–Z (41h–5Ah) шести одиниць поспіль не буває, тому в прикладах практичного завдання № 2 вставлення біта не відбувається; щоб побачити його дію, достатньо закодувати, наприклад, байт FFh. Розглянута модель є спрощеною: у реальному пакеті USB кодуванню NRZI підлягають також поле синхронізації SYNC, ідентифікатор PID і контрольна сума, а біти кожного байта передаються молодшим уперед (рис. 15).



Рис. 14. Ілюстрація bit stuffing (вставлення біта 0 після шести послідовних одиниць)

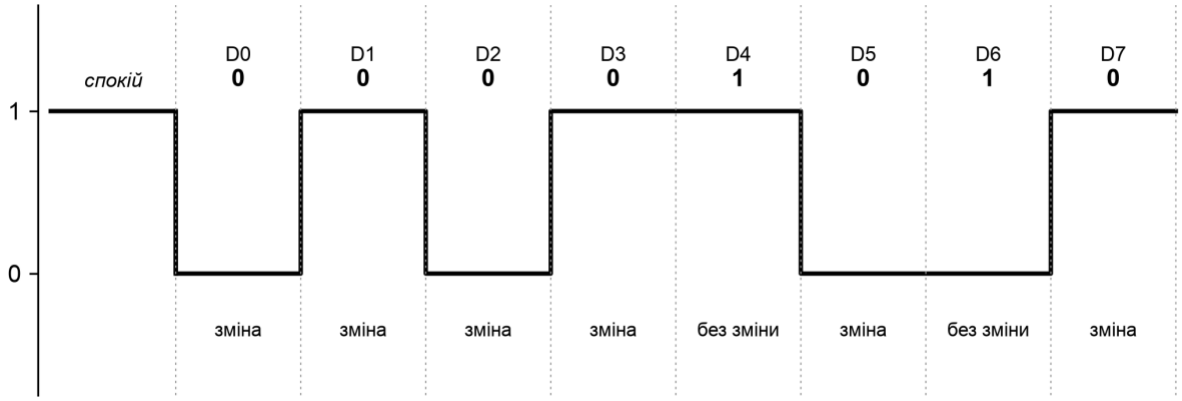


Рис. 15. Кодування NRZI бітів даних символу «P» (50h) у фізичному порядку передавання – молодшим бітом уперед (логічний 0 – зміна рівня, логічна 1 – рівень без зміни)

При аналізі одного інформаційного символу спочатку розглядають його бітове представлення (наприклад, 8-бітове подання коду ASCII), далі – відповідний кадр UART (зі стартовим бітом, бітом парності та стоповими бітами за обраним форматом) і окремо – кодування NRZI для бітів даних (з урахуванням bit stuffing за потреби).

Для USB 3.x SuperSpeed застосовують інші схеми кодування (8b/10b у USB 3.0, 128b/132b у новіших версіях), а не класичний NRZI USB 2.0.

2.6. Практичне завдання № 1. Дослідження програм передавача, приймача та моделі обміну даними інтерфейсу RS-232C

Мета завдання: опанування студентом технології та процесу налаштування передавального та приймального портів, дослідження програм передавача і приймача пакетних даних та реалізації моделі обміну заданим повідомленням через послідовний асинхронний інтерфейс RS-232C (COM-порт / UART).

Постановка завдання. Конкретний пакет даних та відповідне повідомлення студенту визначає викладач (переважно прізвище студента латиницею, наприклад, IVANOV). За варіантом задається швидкість обміну даними; формат кадру в цьому завданні сталий – 8N1.

Технологія виконання завдання. У завданні досліджується логічний рівень інтерфейсу – обмін кадрами UART через драйвер послідовного порту операційної системи; електричні рівні RS-232C не використовуються, оскільки обмін виконується через віртуальні порти. У ході виконання цього практичного завдання необхідно дослідити роботу програм передавача (TX) та приймача (RX) і виконати живий обмін байтами через UART. Приклади реалізації:

Передавач: `host/uart_host.py` (лістинг Б.1). Відкриває послідовний порт засобами бібліотеки pySerial [14], формує повідомлення (прізвище студента латиницею) і за допомогою функції send_message() передає його в лінію зв'язку, завершуючи пакет символом повернення каретки `\r`. За наявності ключа `--wait-ack` передавач після відправлення очікує від приймача підтвердження у вигляді рядка ACK:… і перевіряє коректність відповіді.

Приймач: `host/uart_device_emu.py` (лістинг Б.2). Зчитує з порту вхідний рядок до символу `\r`, виводить у журнал прийняті дані у текстовому та шістнадцятковому вигляді, а також формує й надсилає назад підтвердження у форматі ACK:<повідомлення>`\n`, чим завершує цикл обміну «запит – підтвердження».

У цьому практичному завданні формат кадру UART є сталим – 8N1, де 8 – кількість бітів даних; N – біт парності відсутній; 1 – один стоповий біт.

Кроки виконання

Перед виконанням завдання потрібно підготувати програмне середовище згідно з підрозділом А.3 (додаток А); усі команди наведено для активованого віртуального середовища Python. У прикладах використано прізвище PETRENKO і baudrate 9600. Під час виконання практичного завдання потрібно підставити власне прізвище (латиницею A–Z) та параметри свого варіанту.

Крок 1. Визначення вихідних даних. За таблицею варіантів (табл. А.1) і правилом повідомлення студент фіксує: номер варіанту, baudrate, формат кадру та повідомлення – власне прізвище латиницею. Це саме повідомлення має бути на стороні передавача (`--message`) і в прийнятому кадрі приймача.

Крок 2. Перевірка передавача (TX). Перед повним обміном доцільно перевірити, що програма `host/uart_host.py` коректно формує та записує кадр у порт. Для цього використовують віртуальний порт `loop://`:

```
$ python -m host.uart_host --message "PETRENKO" --baud 9600 --port loop://
```

Очікуваний вивід консолі:

```
TX text: 'PETRENKO'
Надіслано байт: 9
TX hex: 50 45 54 52 45 4e 4b 4f 0d
RX text (loopback): 'PETRENKO'
Verify: OK
```

У рядку TX hex останній байт 0d – символ `\r` (завершення пакета).

Крок 3. Створення віртуальної пари портів (міст). Для живого шляху байтів між передавачем і приймачем на одному ПК створюють віртуальну пару COM. На Linux/macOS у першому терміналі запускають і залишають працюючою програму `host/uart_pty_pair.py` (лістинг Б.3):

```
$ python -m host.uart_pty_pair
```

Типово утворюються порти `/tmp/comA` і `/tmp/comB`.

На Windows використовують com0com (наприклад, COM5 ↔ COM6).

Крок 4. Запуск приймача (RX). У другому терміналі запускають емулятор пристрою на одній половині пари:

```
$ python -m host.uart_device_emu --port /tmp/comB
```

Програма очікує вхідний кадр, що завершується символом `\r`. На віртуальній парі портів швидкість обміну фізично не впливає на передавання, проте для відповідності варіанту її задають ключем `--baud` (наприклад, `--baud 19200`) в обох програмах – так само, як на реальному обладнанні.

Крок 5. Запуск передавача (TX) з очікуванням підтвердження. У третьому терміналі запускають host на другій половині пари з ключем `--wait-ack`:

```
$ python -m host.uart_host --message "PETRENKO" --port /tmp/comA --wait-ack
```

Очікуваний вивід передавача:

```
TX text: 'PETRENKO'
Надіслано байт: 9
TX hex: 50 45 54 52 45 4e 4b 4f 0d
RX text: 'ACK:PETRENKO'
Verify: OK
```

Очікуваний вивід приймача:

```
--- exchange ---
RX text: PETRENKO
RX bytes (9): 50 45 54 52 45 4e 4b 4f 0d
TX ACK: ACK:PETRENKO
TX bytes (13): 41 43 4b 3a 50 45 54 52 45 4e 4b 4f 0a
Verify: OK
--------------
```

Успішний обмін підтверджується тим, що байти RX bytes збігаються з TX hex передавача, а передавач отримує ACK:PETRENKO і друкує Verify: OK.

Крок 6. Порівняння і аналіз результатів. Проаналізувати отримані результати і оформити звіт.

2.7. Практичне завдання № 2. Дослідження графічного представлення сигналів лінії зв'язку та кодування NRZI інтерфейсу USB

Мета завдання: опанування студентом особливостей передавання пакета даних через лінію зв'язку у графічному представленні сигналів для послідовного асинхронного інтерфейсу RS-232C (COM-порта / UART) та особливостей формування посимвольних даних у схемі кодування NRZI через інтерфейс USB.

Постановка завдання. Конкретний пакет даних та відповідне повідомлення студенту визначає викладач згідно з таблицею варіантів (табл. А.1). Повідомлення – прізвище студента латиницею (літери A–Z). Задаються швидкість обміну даними (baudrate) та формат кадру UART (8N1, 7E1 або 8N2).

Технологія виконання завдання. У практичному завданні № 1 формат лінії на стороні host було зафіксовано як 8N1, а акцент зроблено на живому обміні байтами між передавачем і приймачем. У практичному завданні № 2 досліджується вже амплітудно-часове представлення сигналу: для заданого повідомлення будуються діаграми UART (старт-біт, біти даних, біт парності за наявності, стоп-біт(и), пауза між символами) та діаграми кодування NRZI, що застосовується у USB 2.0.

Для побудови діаграм використовують програмні засоби курсу: модулі `encoding/uart_plot.py` (лістинг Б.4), `encoding/usb_nrzi.py` (лістинг Б.5), а також довідковий графічний інтерфейс `host/signal_gui.py` (лістинг Б.6) із збереженням рисунків у форматі PNG. Результатом виконання завдання є дві діаграми для всього повідомлення, ручний розрахунок загального часу передавання повідомлення по UART та посимвольний розбір одного символу прізвища (кадр UART і відповідне кодування NRZI).

Кроки виконання

Крок 1. Визначення вихідних даних. За варіантом студент фіксує повідомлення (прізвище), baudrate і формат кадру. Приклад варіанту: повідомлення PETRENKO, швидкість 9600 бод, формат 8N1.

Крок 2. Побудова амплітудно-часової діаграми UART. У кореневому каталозі проєкту виконати команду за варіантом, наприклад:

```
$ python -m encoding.uart_plot --message "PETRENKO" --baud 9600 --format 8N1
```

Програма відкриває графік matplotlib для всього повідомлення (рис. 16) та виводить розрахований час передачі:

```
Розрахований час передачі: 0.009167 с
```

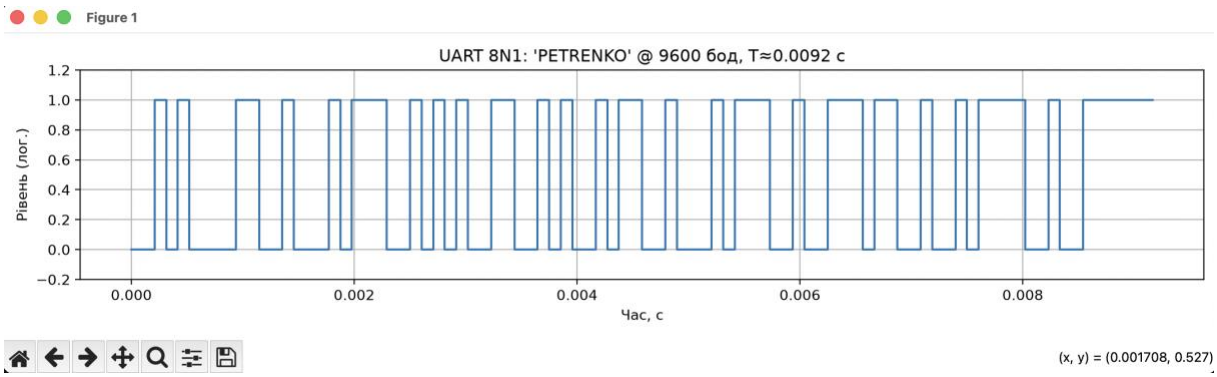


Рис. 16. Амплітудно-часова діаграма UART

Діаграму зберігають у PNG для звіту. На рисунку (або в підписі до нього) обов'язково позначають для одного символу: старт-біт, біти даних, парність (якщо є), стоп-біт(и).

Альтернативно можна використати `host/signal_gui.py` для задання потрібних параметрів через графічний інтерфейс:

```
$ python -m host.signal_gui
```

На рис. 17 зображено приклад роботи програми.

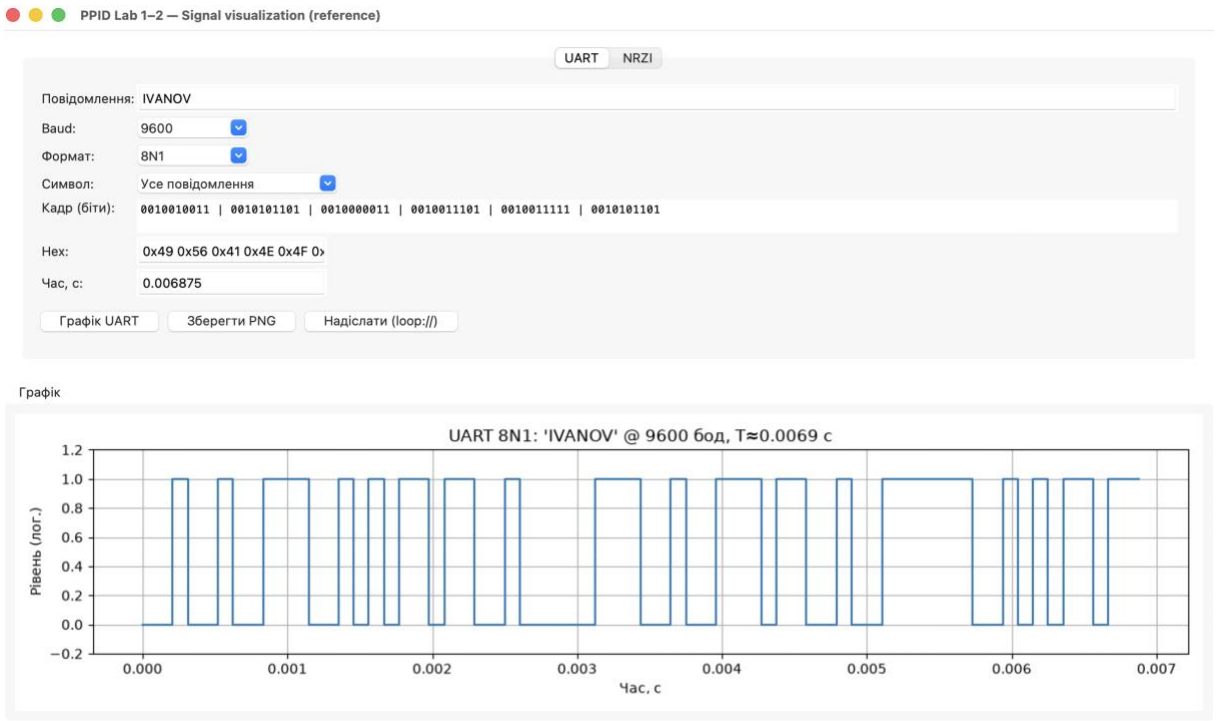


Рис. 17. Формування амплітудно-часової діаграми UART для заданого варіанту

Крок 3. Побудова діаграми кодування NRZI. У кореневому каталозі проєкту виконати команду за варіантом, наприклад:

```
$ python -m encoding.usb_nrzi --message "PETRENKO"
```

Очікуваний фрагмент виводу консолі:

```
Біти (raw): 0101000001000101010101000101001001000101010011100100101101001111
Після bit stuffing:
0101000001000101010101000101001001000101010011100100101101001111
NRZI рівні: [1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1,
1, 0,
 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0,
0, 1,
 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0]
```

Перший елемент списку «NRZI рівні» – початковий рівень лінії (стан спокою), тому список на одне значення довший за кількість бітів. Програма також відкриває графік matplotlib з діаграмою NRZI для всього повідомлення (рис. 18).

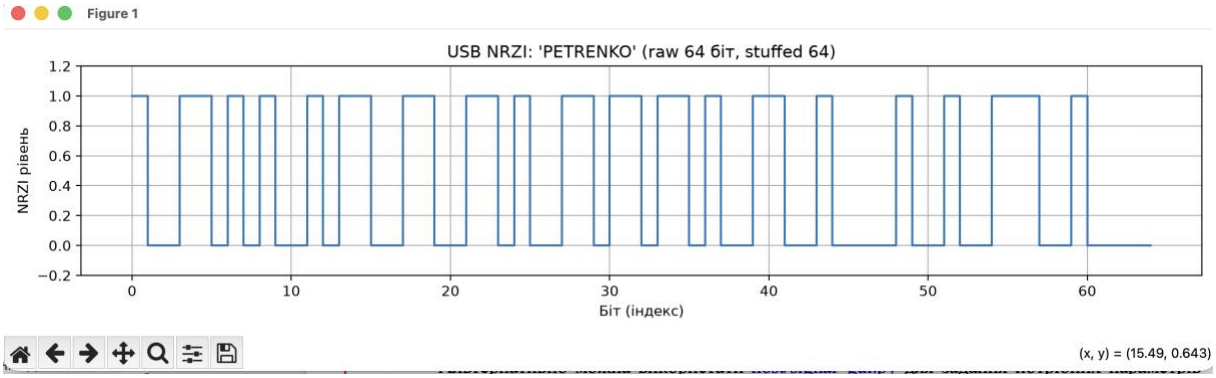


Рис. 18. Амплітудно-часова діаграма NRZI

Альтернативно можна використати `host/signal_gui.py` для задання потрібних параметрів через графічний інтерфейс:

```
$ python -m host.signal_gui
```

На рис. 19 зображено приклад роботи програми.

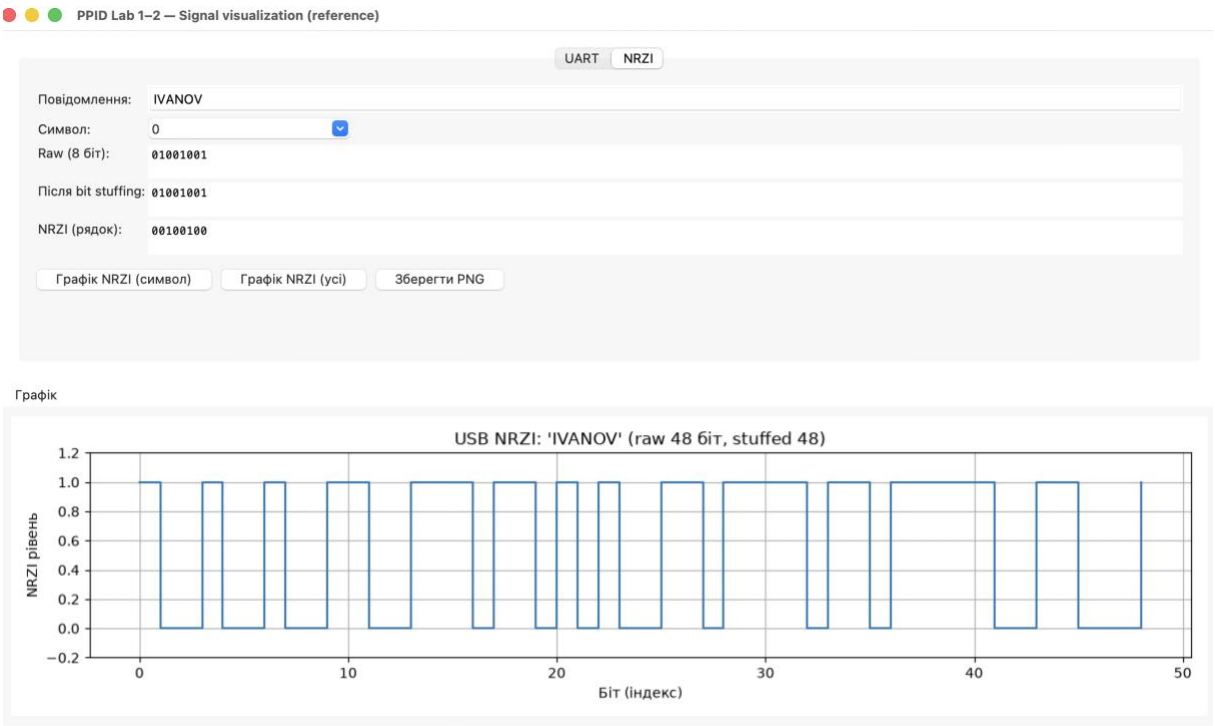


Рис. 19. Формування амплітудно-часової діаграми NRZI для заданого варіанту

Крок 4. Розрахунок загального часу передавання повідомлення (UART).

Тривалість одного такту:

$$T_{такт} = 1 / baudrate$$

Для формату 8N1: бітів на символ = 1 (старт) + 8 (дані) + 1 (стоп) = 10; з паузою 1 такт між символами, яку вставляє програма uart_plot, – 11 тактів на символ (за безперервного передавання без паузи – 10 тактів).

Приклад для PETRENKO (8 символів, 9600, 8N1):

$$T_{такт} = 1 / 9600 \approx 0{,}00010417 \text{ с}$$

$$T_{символ} = 11 \times T_{такт}$$

$$T_{повідомлення} = 8 \times 11 \times T_{такт} \approx 0{,}00917 \text{ с}$$

Отриманий результат зіставляють із виводом програми (0,009167 с) і фіксують у звіті збіг (або пояснюють розбіжність). Для порівняння обчислюють також час безперервного передавання без пауз: $8 \times 10 \times T_{такт} \approx 0{,}00833$ с.

Крок 5. Посимвольний розбір кожної букви прізвища.

Для першої літери прізвища (наприклад, P, ASCII 0x50 = 01010000) складають кадр UART за форматом варіанту та таблицю ролей бітів (табл. 10).

Приклад кадру 8N1: 0 + 01010000 + 1 = 0010100001 (біти даних записано від старшого до молодшого; фізично в лінію вони передаються у зворотному порядку – 0 + 00001010 + 1).

Таблиця 10

Ролі бітів у кадрі UART 8N1 для символу «P»

№ у кадрі	Біт	Роль
1	0	старт
2–9	01010000	дані
10	1	стоп

Далі для тих самих 8 біт даних (без start/stop) застосовують правило NRZI: логічний 0 – зміна рівня, логічна 1 – рівень не змінюється; за потреби враховують bit stuffing після шести одиниць підряд. Результат звіряють із виводом `host/signal_gui.py` (рис. 20).

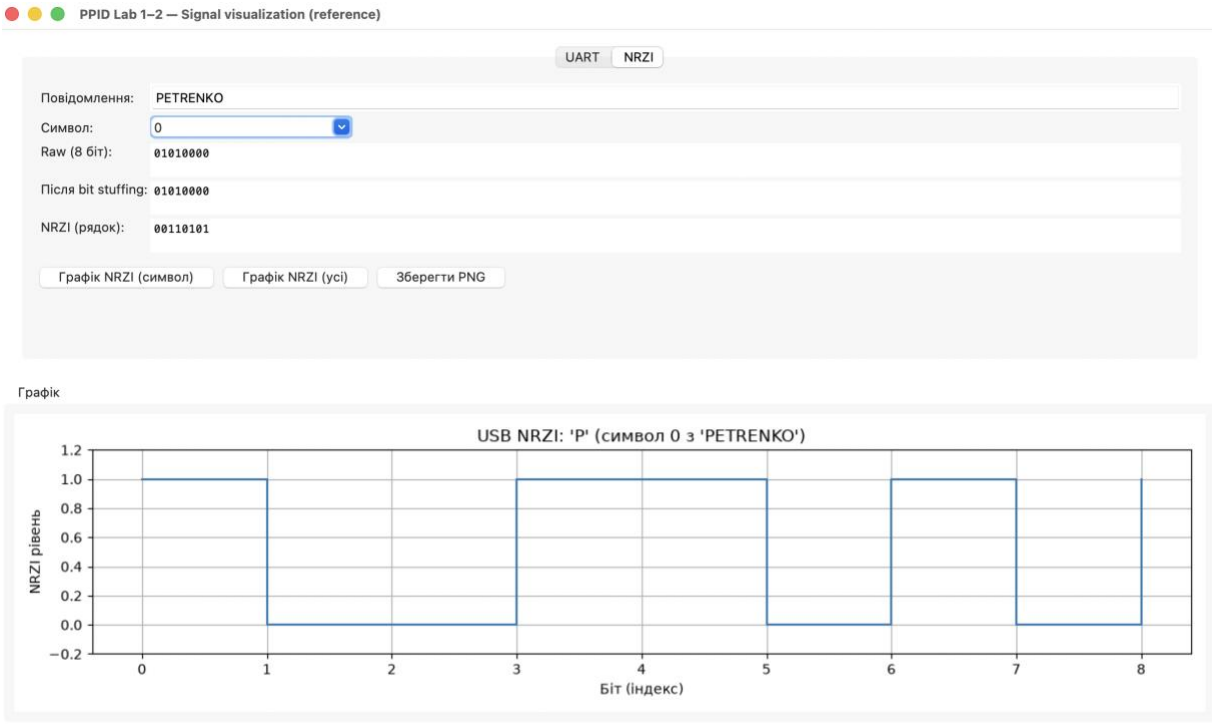


Рис. 20. Посимвольний NRZI-аналіз літери «P» прізвища PETRENKO

Крок 6. Оформлення результатів. Проаналізувати отримані результати та сформувати звіт. У звіті необхідно показати знімки екрана виконання програм, посимвольний розбір кожної літери прізвища і перевірку результатів.

Питання для самоперевірки

1. Навести основні характеристики інтерфейсу RS-232C.
2. Навести призначення сигналів інтерфейсу RS-232C (зокрема TXD, RXD, GND).
3. Навести формат даних інтерфейсу RS-232C / UART (старт-біт, біти даних, парність, стоп-біт).
4. Навести формат слова передавального та приймального порту; пояснити позначення форматів 8N1, 7E1, 8N2.
5. Навести особливості основних режимів функціонування передавального та приймального портів.
6. Пояснити, яким способом порт переводиться з режиму налаштування в режим основної роботи.
7. Пояснити основні принципи розроблення схеми алгоритму первинного налаштування передавача та приймача.
8. Навести основні елементи схеми алгоритму обміну даними між передавачем і приймачем (відкриття порту, передача повідомлення, прийом кадру, формування підтвердження).
9. Навести основні етапи відлагодження програм передавача та приймача.
10. Пояснити роботу програм передавача (uart_host) і приймача (uart_device_emu); навіщо потрібна віртуальна пара COM і ключ `--wait-ack`.
11. Чим відрізняється самоперевірка на `loop://` від повного обміну між передавачем і приймачем?
12. Навіщо baudrate і формат кадру мають збігатися на обох сторонах лінії?
13. Чим TTL-UART відрізняється від RS-232C за рівнями сигналу та полярністю? Навіщо потрібен перетворювач рівнів (MAX232) і чому не можна з’єднувати виводи мікроконтролера напряму з роз’ємом DB9?
14. Чим RS-485 відрізняється від RS-232 за топологією та дальністю? Навіщо поверх UART часто використовують протокол Modbus RTU?
15. Яким сигналом керують напрямом обміну на напівдуплексній магістралі RS-485 і до чого призводить помилка в керуванні ним?
16. Які рівні моделі OSI задіяні в послідовному каналі «точка-точка» і в якій послідовності їх перевіряють під час відлагодження?
17. Навести співвідношення швидкості обміну інформацією та тривалості одного такту для інтерфейсу RS-232C.

18. Навести сигнальні рівні кодування логічної одиниці та логічного нуля для передавача та приймача.
19. Пояснити, яким способом можна розрахувати загальний час передавання заданого повідомлення.
20. Навести основні характеристики інтерфейсу USB у частині, потрібній для розуміння кодування лінії.
21. Навести особливості системи кодування NRZI.
22. Пояснити призначення процедури bit stuffing у NRZI.
23. Навести основні етапи побудови амплітудно-часової діаграми UART для заданого повідомлення.
24. Навести основні етапи побудови діаграми NRZI для заданого повідомлення.
25. Пояснити роботу програм (uart_plot, usb_nrzi / signal_gui) для заданого пакету даних.
26. Чим відрізняється логічний рівень у програмній моделі від електричного рівня на лінії RS-232?
27. Для якої версії USB застосовується NRZI, а яке кодування використовує SuperSpeed (USB 3.x)?
28. Чим бодова швидкість відрізняється від бітової? Які накладні витрати має формат 8N1?
29. У якому порядку біти поля даних передаються в лінію UART?

Тестові завдання

Оберіть одну правильну відповідь. Відповіді до тестових завдань наведено в додатку В.

1. Логічна одиниця на лінії RS-232C передається напругою:
   а) від −5 до −15 В
   б) від +5 до +15 В
   в) 0 В
   г) +3,3 В
2. Скільки тактів займає символ у форматі 8N1 без паузи між символами?
   а) 10
   б) 9
   в) 8
   г) 11
3. Позначення формату кадру 7E1 означає:
   а) 7 стопових бітів
   б) 7 символів у кадрі

в) 7 бітів даних без контролю парності, 1 стоповий біт
г) 7 бітів даних, контроль на парність, 1 стоповий біт

4. Сигнал CTS використовується для:
   а) апаратного керування потоком
   б) програмного керування потоком
   в) живлення пристрою
   г) передавання тактового сигналу
5. Символ XOFF програмного керування потоком має код:
   а) 11h
   б) 0Ah
   в) 0Dh
   г) 13h
6. Для під’єднання мікроконтролера з логікою 3,3 В до порту RS-232C використовують:
   а) MAX485
   б) пряме з’єднання
   в) дільник напруги
   г) MAX3232
7. Сигнал DE/RE приймача-передавача MAX485 керує:
   а) напрямом обміну
   б) швидкістю обміну
   в) контролем парності
   г) живленням лінії
8. У кодуванні NRZI логічний 0 передається:
   а) збереженням рівня сигналу
   б) зміною рівня сигналу
   в) імпульсом половинної тривалості
   г) завжди високим рівнем

3. Універсальна послідовна шина USB

Розділ присвячено універсальній послідовній шині USB – найпоширенішому інтерфейсу під’єднання периферійних пристроїв до комп’ютера – та розробленню елементів програмного драйвера для неї: моделі транзакції, переліку пристроїв і запису даних на накопичувач.

3.1. Загальні характеристики та топологія шини

Універсальна послідовна шина USB (Universal Serial Bus) – стандартний інтерфейс під’єднання периферійних пристроїв до комп’ютера, який забезпечує гаряче під’єднання, автоматичну конфігурацію пристроїв (Plug&Play) та їх живлення по кабелю [1, 8]. USB 2.0 є послідовною напівдуплексною шиною: дані передаються в обох напрямках по черзі однією диференційною парою D+/D− (у режимах SuperSpeed USB 3.x для передавання і приймання використовують окремі пари, і обмін є повнодуплексним).

Шина має топологію «ієрархічна зірка» (tiered star). У її корені знаходиться хост – контролер комп’ютера з вбудованим кореневим концентратором (root hub), до портів якого під’єднують пристрої та зовнішні концентратори (hubs). Концентратор має кілька низхідних портів (типово чотири або сім), а кількість послідовно з’єднаних концентраторів між хостом і пристроєм обмежена п’ятьма через затримки поширення сигналу. Хост може адресувати до 127 пристроїв: адреси 1–127 присвоюються під час конфігурації, а адресу 0 має лише щойно під’єднаний пристрій. Фізичний пристрій може бути складеним, тобто містити кілька функцій, – наприклад, клавіатура з вбудованим маніпулятором.

Увесь обмін на шині ініціює хост: він по черзі звертається до пристроїв, надсилаючи пакети-маркери (token packets), а пристрої лише відповідають. Безпосереднього обміну між периферійними пристроями немає – всі дані проходять через хост. Про під’єднання або від’єднання пристрою хост дізнається від концентратора, після чого виконує опитування шини (bus enumeration): зчитує дескриптори пристрою, присвоює йому унікальну адресу та обирає драйвер. Після від’єднання пристрою його адреса звільняється.

Під час під’єднання пристрій повідомляє хосту про свою швидкість підтягувальним резистором 1,5 кОм між лінією даних і джерелом 3,3 В: до лінії D− під’єднує резистор пристрій Low-Speed, до лінії D+ – пристрій Full-Speed; пристрої High-Speed спочатку під’єднуються як Full-Speed, а потім узгоджують вищу швидкість з хостом [8]. Порт USB 2.0 надає периферійному пристрою живлення +5 В зі струмом до 500 мА після конфігурації (до конфігурації – до 100 мА; порт USB 3.x – до 900 мА), що дозволяє живити малопотужні пристрої без окремого джерела.

Для з’єднання пристроїв без комп’ютера (наприклад, фотокамери з принтером чи смартфона з флеш-накопичувачем) у 2001 р. прийнято доповнення до специфікації USB 2.0 – USB On-The-Go (OTG). Пристрій з підтримкою OTG може виконувати роль хоста або периферійного пристрою, а роль визначається під час з’єднання.

3.2. Обмін даними та логічна структура пристрою

Обмін на шині організовано кадрами тривалістю 1 мс (у режимі High-Speed – мікрокадрами тривалістю 125 мкс). Кожен кадр починається пакетом SOF (Start of Frame), а в його межах хост виконує транзакції з різними пристроями. Специфікація визначає чотири типи передач даних (табл. 11), які відрізняються гарантіями доставки та часу відгуку: керуючі передачі використовують для конфігурації пристроїв (максимальний розмір пакета – 8, 16, 32 або 64 байти, для пристроїв Low-Speed – 8 байтів), групові – для передавання великих обсягів даних, передачі переривань – для пристроїв введення, ізохронні – для потоків реального часу.

Апаратну частину хоста реалізує контролер шини. Перші специфікації контролерів – UHCI (Universal Host Controller Interface, Intel), за якої більшу частину функцій керування шиною виконує драйвер, та OHCI (Open Host Controller Interface, Compaq, Microsoft, National Semiconductor) з апаратною реалізацією більшості функцій. Сучасні комп’ютери використовують контролери EHCI (USB 2.0) та xHCI (USB 3.x і новіші версії), причому xHCI обслуговує всі режими швидкості.

Логічну структуру пристрою USB описують кінцеві точки (endpoints) і дескриптори. Кінцева точка – це буфер у контролері пристрою, який є джерелом або приймачем даних. Кінцева точка 0 обов’язкова і використовується для конфігурації та службових команд; точки 1–15 призначені для прикладних даних. Напрям визначають відносно хоста: IN – до комп’ютера, OUT – від комп’ютера. Під час опитування шини (enumeration) хост зчитує ієрархію дескрипторів: дескриптор пристрою (клас пристрою, ідентифікатори виробника та продукту VID/PID), дескриптор конфігурації (профіль живлення), дескриптор інтерфейсу (функціональний клас – HID, Mass Storage, CDC) та дескриптори кінцевих точок (тип передачі, максимальний розмір пакета). Саме за класом інтерфейсу та парою VID/PID операційна система обирає драйвер пристрою. Типи передач даних узагальнено в табл. 11.

Таблиця 11

Типи передач даних USB

Тип передачі	Призначення	Особливості
Керуюча (control)	ініціалізація та конфігураційні команди через кінцеву точку 0	гарантована доставка з підтвердженням
Групова (bulk)	великі обсяги даних: накопичувачі, принтери, сканери	контроль CRC з повторенням, передається у вільні інтервали шини
Переривання (interrupt)	регулярне опитування пристроїв введення: миші, клавіатури	гарантований максимальний час відгуку

Тип передачі	Призначення	Особливості
Ізохронна (isochronous)	потоки реального часу: аудіо, відео	фіксована смуга пропускання, без повторного передавання при помилках

Типова транзакція складається з трьох фаз. У фазі маркера хост надсилає пакет Token з ідентифікатором PID OUT, IN або SETUP, адресою пристрою та номером кінцевої точки. У фазі даних передавач надсилає пакет Data з ідентифікатором PID DATA0 або DATA1, корисним навантаженням і контрольною сумою CRC16; чергування DATA0/DATA1 дозволяє приймачу виявити повторно переданий пакет. У фазі квитування приймач відповідає пакетом Handshake: ACK – дані прийнято, NAK – пристрій тимчасово не готовий, STALL – помилка або непідтримуваний запит. В ізохронних передачах фаза квитування відсутня, а у відповідь на маркер IN пристрій, не готовий до передавання, одразу надсилає NAK без фази даних. Саме таку послідовність пакетів формує навчальна програма практичного завдання № 3.

3.3. Версії специфікації USB

Першу версію специфікації – USB 1.0 – опубліковано в 1996 р. Її розробив консорціум компаній (Compaq, DEC, IBM, Intel, Microsoft, NEC, Northern Telecom), щоб замінити різнотипні низькошвидкісні порти комп'ютера – RS-232C, Centronics (LPT), PS/2 – єдиним інтерфейсом [8]. Специфікація визначила два режими швидкості: Low-Speed (1,5 Мбіт/с) для пристроїв введення – клавіатур, мишей, джойстиків – та Full-Speed (12 Мбіт/с) для принтерів, аудіопристроїв і накопичувачів. Довжина кабелю становить до 5 м для режиму Full-Speed і до 3 м для режиму Low-Speed. Версія USB 1.1 (1998 р.) усунула недоліки першої версії, пов'язані переважно з концентраторами, і саме вона набула масового поширення.

Версія USB 2.0 (2000 р.) додала режим High-Speed зі швидкістю 480 Мбіт/с, зберігши сумісність з пристроями Low-Speed і Full-Speed. Концентратори USB 2.0 ізолюють повільні пристрої від швидкісного трафіку, тому пристрої з різними швидкостями можуть працювати на одній шині. Реальна швидкість обміну з накопичувачами помітно нижча за номінальну через накладні витрати протоколу та затримки між запитом хоста і відповіддю пристрою.

Подальшим розвитком стандарту стали специфікації USB 3.x (режими SuperSpeed 5, 10 та 20 Гбіт/с) та USB4 (до 40 Гбіт/с, у версії USB4 2.0 – до 80 Гбіт/с) [9]. Для режимів SuperSpeed у кабелі передбачено додаткові диференційні пари, а сигнали кодують за схемами 8b/10b (USB 3.0), 128b/132b (USB 3.1 і новіші версії), а в режимі USB4 Gen 4 – трирівневе кодування PAM3, замість NRZI. USB 3.x зворотно сумісний з USB 2.0 та USB 1.1. Версії специфікації та режими швидкості узагальнено в табл. 12.

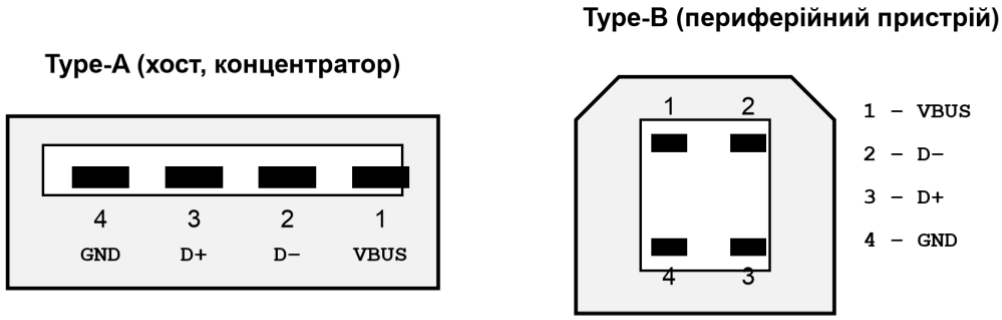
Таблиця 12

Версії специфікації та режими швидкості USB

Версія	Режим	Максимальна швидкість	Типові пристрої
USB 1.x, 2.0	Low-Speed	1,5 Мбіт/с	миші, клавіатури, джойстики
USB 1.x, 2.0	Full-Speed	12 Мбіт/с	аудіопристрої, принтери, мікроконтролери
USB 2.0	High-Speed	480 Мбіт/с	флеш-накопичувачі, веб-камери
USB 3.x / USB4	SuperSpeed	від 5 до 40 Гбіт/с і більше	зовнішні SSD, відеозахоплення, DisplayPort

3.4. З’єднувачі та кабелі USB

Специфікація визначає не лише електричні характеристики шини, а й конструкцію з’єднувачів та розведення сигналів на контактах. Для USB 1.x і 2.0 стандартизовано з’єднувачі типів A та B (рис. 21). З’єднувач типу A встановлюють на стороні хоста або концентратора, типу B – на стороні периферійного пристрою, що унеможливлює з’єднання двох хостів між собою. Сучасний з’єднувач USB Type-C має 24 симетрично розташовані контакти, тому його можна вставляти будь-якою стороною; лінії CC1/CC2 використовують для визначення орієнтації кабелю та ролей пристроїв (хост чи периферійний пристрій), а також для узгодження живлення за специфікацією USB Power Delivery (до 240 Вт) [10, 23].



Вигляд з боку гнізда; 1 – VBUS (+5 В), 2 – D−, 3 – D+, 4 – GND

Рис. 21. Нумерація контактів з’єднувачів USB типів A та B

Кабель USB 2.0 містить чотири провідники (табл. 13).

Таблиця 13

Призначення провідників кабелю USB

Номер контакту	Позначення	Колір провідника
1	VBUS	червоний
2	D-	білий
3	D+	зелений
4	GND	чорний

Провідник VBUS (+5 В) і спільний провід GND призначені для живлення периферійних пристроїв, а провідники D+ і D− утворюють диференційну пару даних. Логічний стан лінії визначається різницею потенціалів між D+ і D− (понад 0,2 В), а не напругою кожного провідника відносно GND, що забезпечує стійкість до завад. Провідники даних у кабелі виконують витою парою, а кабель для пристроїв Full-Speed і High-Speed – екранованим [8].

3.5. Флеш-пам’ять USB-накопичувачів

Найпоширенішим пристроєм класу Mass Storage є USB флеш-накопичувач, з яким працює програма практичного завдання № 3. Він містить мікросхему енергонезалежної флеш-пам’яті типу NAND та контролер, який реалізує інтерфейс USB і перетворює команди класу Mass Storage на операції з пам’яттю. Інформація у флеш-пам’яті зберігається у вигляді заряду плаваючих затворів транзисторів-комірок, тому не втрачається без живлення. Комірка може зберігати один (SLC) або кілька (MLC, TLC) бітів залежно від кількості розрізнюваних рівнів заряду. Записування виконується сторінками, а стирання – цілими блоками, і кількість циклів перезапису кожного блока обмежена, тому контролер накопичувача рівномірно розподіляє записи між блоками (wear leveling) [1].

Для операційної системи накопичувач є блоковим пристроєм з файловою системою (зазвичай FAT32 або exFAT), тож прикладна програма працює з ним через звичайні файлові операції. Саме тому запис файла на накопичувач у практичному завданні № 3 є роботою прикладної програми, а не розробленням драйвера пристрою: драйвер класу Mass Storage вже входить до складу операційної системи.

3.6. Практичне завдання № 3. Розроблення та дослідження елементів програмного драйвера інтерфейсу USB

Мета завдання: опанування студентом принципів обміну даними через інтерфейс USB – моделі транзакції передавання повідомлення, сканування інтерфейсних портів (у навчальній моделі – імітаційний перелік пристроїв) та налаштування передавання даних на рівні моделі пакетів і файлової системи.

Постановка завдання. Конкретний пакет даних та відповідне повідомлення студенту визначає викладач згідно з таблицею варіантів (табл. А.1). Повідомлення – прізвище латиницею (літери A–Z, наприклад IVANOV). Імітаційний пристрій – зі стовпця «Mock USB» таблиці варіантів.

Технологія виконання завдання. Драйвер передавача має бути налаштований на характеристики, індивідуально задані студенту. Насамперед це довжина інформаційного повідомлення (прізвище) та імітаційний пристрій варіанту. Спочатку потрібно визначитися зі структурою транзакції. Якщо передавачем є контролер комп'ютера (host), транзакція включає три пакети: маркерний (Token), даних (Data) та підтвердження (Handshake). Маркерний пакет визначає тип і напрям транзакції (OUT), адресу пристрою та номер кінцевої точки. Кількість байт у полі даних пакета Data має бути не меншою за кількість байт заданого повідомлення; у навчальній реалізації `host/usb_transaction.py` (лістинг Б.7) передбачено мінімум 8 байт payload – коротше прізвище доповнюється нульовими байтами (це умовність навчальної моделі: у реальному USB дані довільної довжини діляться на пакети розміром до wMaxPacketSize з чергуванням DATA0/DATA1, а пакети нульової довжини також допустимі).

Для створення драйвера необхідно розробити відповідну схему алгоритму (побудова кадрів транзакції → сканування переліку пристроїв → вибір пристрою → формування / відкриття тексту повідомлення → запис). Розроблення схеми алгоритму є одним із показників поглибленого розуміння логіки взаємодії пристроїв у периферійній підсистемі на основі інтерфейсу USB.

У курсі програмний драйвер реалізується трьома пов'язаними модулями. Приклади реалізації:

Модель транзакції: `host/usb_transaction.py` (лістинг Б.7). Побудова байтових кадрів OUT: Token → Data → Handshake (hex-дамп).

Сканування портів: `host/usb_scan.py` (лістинг Б.8). Виведення імітаційного переліку з `fixtures/usb_devices.json` (лістинг Б.9) – аналог lsusb.

Діалоговий передавач: `host/usb_gui.py` (лістинг Б.10). Виконання «Сканування», вибір пристрою, перегляд властивостей, відкриття файлу або введення тексту, запис `message.txt`.

На рис. 22 наведено типи з’єднувачів USB (A/B; оглядово USB-C) – для порівняння призначення контактів у коротких теоретичних відомостях звіту.

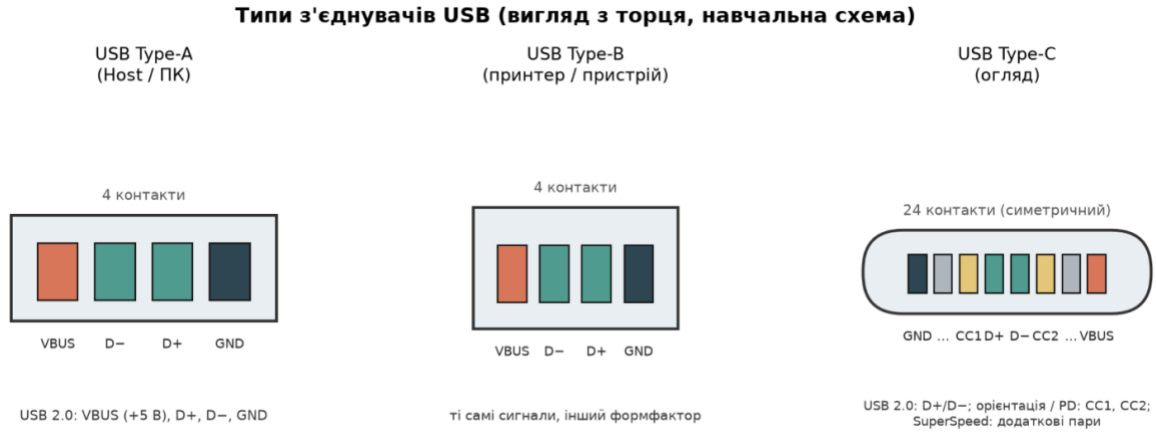


Рис. 22. Типи з’єднувачів USB

На рис. 23 наведено приклад вікна програмного драйвера передавання даних через USB-інтерфейс.

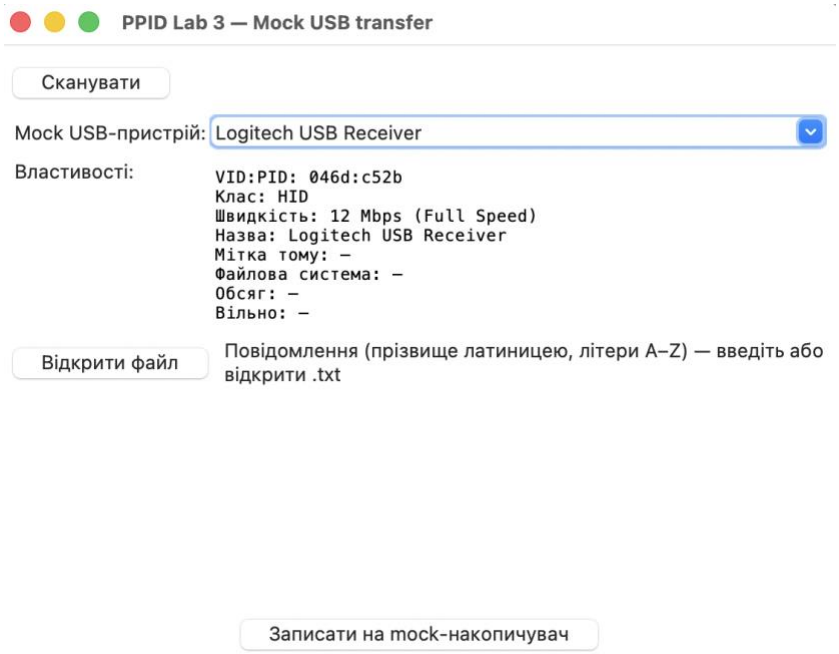


Рис. 23. Вікно програмного драйвера передавання даних через USB-інтерфейс

Список пристроїв у демонстраційній програмі завжди зчитується з імітаційних даних у форматі JSON – це не реальні USB-порти ПК. Реалізація реального зчитування і запису даних через інтерфейс USB – це додаткова задача у цьому практичному завданні для отримання максимальної оцінки.

Кроки виконання

У прикладах використано прізвище PETRENKO і імітаційний пристрій SanDisk Cruzer (Mass Storage) (як типовий Mass Storage з таблиці варіантів). Під час виконання практичного завдання потрібно підставити власне прізвище (латиницею A–Z) та параметри свого варіанту (номер варіанту, імітаційний пристрій зі стовпця «Mock USB»).

Крок 1. Визначення вихідних даних. За таблицею варіантів (табл. А.1) і правилом повідомлення студент фіксує: номер варіанту, імітаційний пристрій (mock_usb_name), та повідомлення – власне прізвище латиницею. Це саме повідомлення має бути в `--message` для usb_transaction, у полі тексту usb_gui і в файлі `message.txt` після запису.

Спочатку потрібно визначитися зі структурою транзакції. Якщо передавачем є контролер комп'ютера (host), транзакція включає три пакети: маркерний (Token), даних (Data) та підтвердження (Handshake). Кількість байт у полі даних має бути не меншою за довжину повідомлення; у навчальній моделі usb_transaction мінімум payload – 8 байт (коротше прізвище доповнюється нулями). У звіті також порівняти контакти USB-A та оглядово USB-C.

Крок 2. Побудова моделі транзакції (Token → Data → Handshake). Програма `host/usb_transaction.py` формує байтові кадри OUT-транзакції для заданого повідомлення (навчальна модель USB 2.0, не передача по фізичній шині):

```
$ python -m host.usb_transaction --message "PETRENKO"
```

Очікуваний вивід консолі:

```
Фази: TOKEN → DATA → HANDSHAKE
Пакет 1: e1 81 00
Пакет 2: c3 50 45 54 52 45 4e 4b 4f e7 f4
Пакет 3: d2
```

Де пакет 1 – Token (PID OUT); пакет 2 – Data (PID DATA0 c3, далі ASCII-байти прізвища 50 45 54 52 45 4e 4b 4f = PETRENKO, потім CRC); пакет 3 – Handshake (PID ACK d2). Кількість байт у полі даних для PETRENKO – 8. Ті самі ASCII-байти з'являлися у практичних завданнях № 1–2 (TX hex / діаграми). Модель є навчальною і спрощеною: поле CRC5 маркерного пакета не обчислюється (за стандартом пакет OUT до адреси 1 і кінцевої точки 1 має вигляд e1 81 58), контрольна сума CRC16 пакета даних обчислюється без фінальної інверсії (за стандартом для PETRENKO – 18 0b), а поля синхронізації SYNC і кінця пакета EOP не показано. Байт 81h містить 7-бітову адресу пристрою (1) і молодший біт номера кінцевої точки (1), передані молодшим бітом уперед. Пакет квитування ACK надсилає пристрій, а не хост.

Крок 3. Сканування інтерфейсних портів (імітаційний перелік). Програма `host/usb_scan.py` виводить навчальний список пристроїв з `fixtures/usb_devices.json` (аналог lsusb, але не з реальної шини ПК):

```
$ python -m host.usb_scan
```

Очікуваний вивід (фрагмент; у звіті виділити свій рядок варіанту):

```
Mock USB enumeration (fixtures/usb_devices.json):
У продакшені: lsusb (Linux) або Get-PnpDevice (Windows)

046d:c52b HID @ 12 Mbps (Full Speed) — Logitech USB Receiver
0781:5567 Mass Storage @ 480 Mbps (High Speed) — SanDisk Cruzer (Mass Storage)
303a:1001 CDC @ 12 Mbps (Full Speed) — Espressif USB JTAG/serial (CDC)
1d6b:0002 Hub @ 480 Mbps (High Speed) — Linux Foundation 2.0 root hub
8087:0026 Wireless @ 12 Mbps (Full Speed) — Intel Bluetooth (USB)
```

Коротко зіставити з реальним переглядом ОС (lsusb / Диспетчер пристроїв): у практичному завданні список зчитується з JSON.

Крок 4. Діалоговий драйвер передавача (usb_gui). Запустити графічну програму:

```
$ python -m host.usb_gui
```

Послідовність дій у вікні:

1. Натиснути «Сканувати» – оновити імітаційний перелік.
2. У списку (ComboBox) обрати пристрій варіанту (наприклад, SanDisk Cruzer (Mass Storage)).
3. Переглянути панель «Властивості» (для Mass Storage типово):

```
VID:PID: 0781:5567
Клас: Mass Storage
Швидкість: 480 Mbps (High Speed)
Назва: SanDisk Cruzer (Mass Storage)
Мітка тому: PPID_USB
Файлова система: FAT32
Обсяг: 16 ГБ
Вільно: 15.2 ГБ
```

4. У поле повідомлення ввести прізвище, наприклад PETRENKO (або відкрити .txt-файл з цим текстом).
5. Натиснути «Записати на mock-накопичувач» – створюється файл `message.txt` у тимчасовому каталозі `ppid_usb_*`. Справжній запис файлів можливий лише на пристрої класу Mass Storage; для варіантів з пристроями інших класів (HID, Hub, CDC, Wireless) програма-модель також створює файл, а у звіті слід пояснити, чим клас вашого пристрою відрізняється від Mass Storage і чому на нього неможливо записати файл.
6. Перевірити вміст файлу командою `cat /path_to/message.txt` – має бути виведено PETRENKO.

7. У звіті навести знімок екрана вікна програми і результат виконання.
8. *(Для максимальної оцінки.)* Реалізувати роботу з реальними пристроями USB:

– отримати перелік реальних пристроїв USB з їх дескрипторами (VID/PID, клас) засобами бібліотеки pyusb (libusb) і порівняти його з імітаційним переліком;
– вставити USB-накопичувач;
– знайти знімні (removable) томи ОС (Windows – знімні диски; macOS – /Volumes/; Linux – /media/$USER або /run/media/$USER);
– зафіксувати мітку тому, формат і розмір обраного носія;
– записати `message.txt` з прізвищем на USB-накопичувач і перевірити результат.

Крок 5. Порівняння і аналіз результатів. Проаналізувати отримані результати і оформити звіт.

Питання для самоперевірки

1. Навести основні характеристики інтерфейсу USB.
2. Навести призначення контактів роз’ємів USB.
3. Навести особливості структур транзакцій інтерфейсу USB.
4. Навести основні етапи відлагодження драйвера та прикладних програм практичного завдання.
5. Пояснити роботу написаного програмного драйвера.
6. Пояснити структуру транзакцій інтерфейсу USB.
7. Пояснити основні принципи розроблення схеми алгоритму сканування інтерфейсних портів.
8. Пояснити основні принципи розроблення схеми алгоритму налаштування інтерфейсних портів для передавання даних.
9. Чим імітаційний перелік пристроїв відрізняється від lsusb або Диспетчера пристроїв?
10. Чим роз’єм USB-C відрізняється від USB-A за кількістю контактів і ролями (host/device)?
11. Чим запис на реальний USB-накопичувач відрізняється від запису в тимчасовий каталог імітаційного накопичувача `ppid_usb_*`?

Тестові завдання

Оберіть одну правильну відповідь. Відповіді до тестових завдань наведено в додатку В.

1. Топологія шини USB:
   а) кільце
   б) спільна шина
   в) ієрархічна зірка
   г) коміркова мережа
2. Максимальна кількість пристроїв, які може адресувати хост USB:
   а) 7
   б) 32
   в) 255
   г) 127
3. Обов’язкова кінцева точка пристрою USB, через яку виконується конфігурація:
   а) 127
   б) 1
   в) 15
   г) 0

4. Тип передачі USB, у якому дані не передаються повторно при помилках:
   а) ізохронна
   б) групова
   в) переривання
   г) керуюча
5. Пакет квитування, яким пристрій повідомляє про тимчасову неготовність:
   а) ACK
   б) NAK
   в) STALL
   г) SETUP
6. Швидкість режиму High-Speed USB 2.0:
   а) 1,5 Мбіт/с
   б) 480 Мбіт/с
   в) 12 Мбіт/с
   г) 5 Гбіт/с
7. Пристрій Full-Speed під’єднує підтягувальний резистор 1,5 кОм до лінії:
   а) D−
   б) GND
   в) VBUS
   г) D+
8. Кількість контактів з’єднувача USB Type-C:
   а) 4
   б) 9
   в) 24
   г) 12

4. Синхронні послідовні інтерфейси I²C і SPI та інтеграція інтерфейсів

Матеріал розділу призначено для поглибленого вивчення особливостей, характеристик синхронної послідовної шини I²C, одного із найбільш поширених інтерфейсів підключення периферійних мікросхем, та отримання навиків розроблення для цього інтерфейсу програмних драйверів, які є складовими компонентами багатьох комп'ютерних систем.

4.1. Шина I²C

Інтерфейс I²C за класифікацією послідовний, магістральний, напівдуплексний синхронний інтерфейс. Він реалізовує стандарт обміну даними I²C (Inter-Integrated Circuit – міжмікросхемне з'єднання), який розроблено корпорацією Philips (нині NXP Semiconductors) у 1982 році для зв'язку між інтегральними мікросхемами у межах однієї друкованої плати [11]. На відміну від інтерфейсу RS-232C, орієнтованого на зв'язок комп'ютера із зовнішнім периферійним пристроєм, шина I²C призначена насамперед для внутрішньоблочного обміну між мікроконтролером та периферійними мікросхемами: давачами, годинниками реального часу, мікросхемами пам'яті, індикаторами. Відповідно вивчення цього інтерфейсу є обґрунтованим для розуміння побудови периферійної підсистеми вбудованих комп'ютерних систем.

Термін «**послідовний**» означає, що передавання даних виконується по одиночному провіднику, а біти передаються послідовно один за другим. У певний момент часу в одному напрямку передається тільки один біт даних.

Термін «**магістральний**» означає, що інтерфейс забезпечує взаємодію багатьох пристроїв, під'єднаних до однієї пари ліній зв'язку. Цим шина I²C принципово відрізняється від радіального інтерфейсу RS-232C, який забезпечує взаємодію тільки двох пристроїв. Розрізнення пристроїв на магістралі виконується не окремими лініями зв'язку, а адресою, яка передається на початку кожної транзакції.

Термін «**напівдуплексний**» означає, що інтерфейс забезпечує можливість передавання даних між пристроями в обох напрямах, проте не одночасно, оскільки для даних використовується лише одна сигнальна лінія.

Термін «**синхронний**» означає, що разом з даними по окремій лінії зв'язку передається тактовий сигнал, який формує пристрій-ініціатор обміну. Приймач визначає моменти зчитування бітів за цим тактовим сигналом, а не за власним генератором, як це відбувається в асинхронному інтерфейсі RS-232C. Відповідно шина I²C не потребує попереднього узгодження швидкості обміну між передавачем та приймачем і не використовує стартового, стопового бітів та біта парності.

Шина I²C використовує дві двонапрямлені лінії зв'язку: лінію даних SDA (Serial Data) та лінію тактового сигналу SCL (Serial Clock). Обидві лінії є спільними для всіх пристроїв магістралі. Структурна схема шини з одним провідним та кількома підпорядкованими пристроями показана на рис. 24.

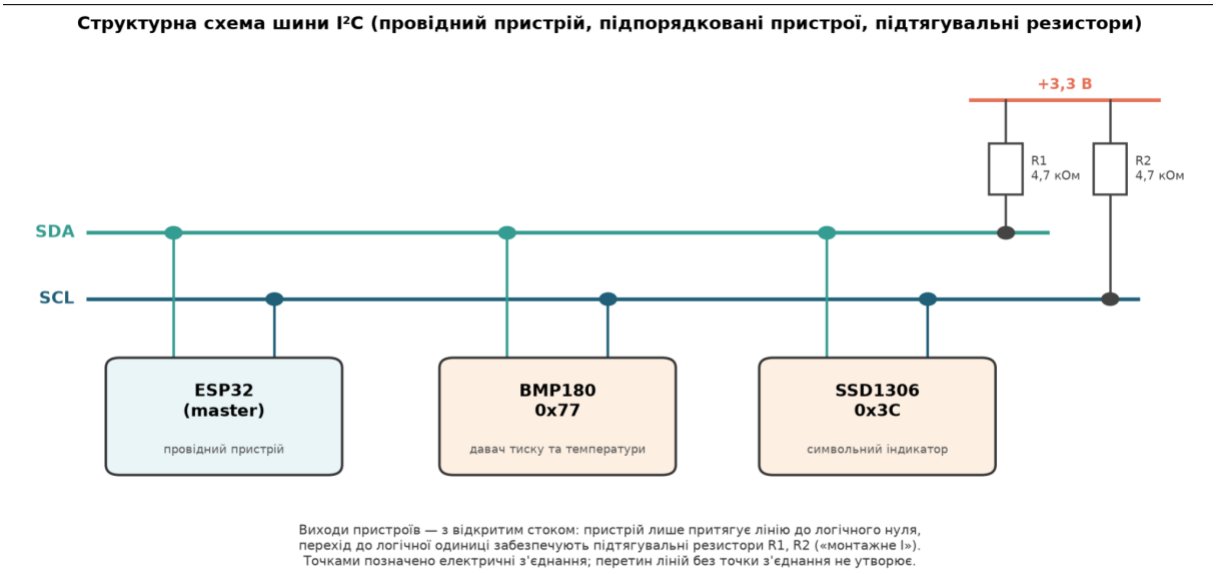


Рис. 24. Структурна схема шини I²C

Пристрій, який ініціює обмін та формує тактовий сигнал, називають провідним (master), а пристрій, який відповідає на звернення за своєю адресою, – підпорядкованим (slave). У практичних завданнях провідним пристроєм є мікроконтролер ESP32, підпорядкованим – давач атмосферного тиску та температури BMP180 або графічний OLED-індикатор з контролером SSD1306.

Виходи пристроїв на шині виконані за схемою з відкритим стоком (open drain). Пристрій може лише притягувати лінію до рівня логічного нуля, а перехід до рівня логічної одиниці забезпечують підтягувальні резистори, під'єднані до шини живлення. Опір підтягувальних резисторів обирають у межах приблизно від 2,2 кОм до 10 кОм залежно від швидкості обміну та ємності магістралі, яка не повинна перевищувати 400 пФ. Таке схемотехнічне рішення реалізує логічну функцію «монтажне І»: лінія перебуває у стані логічного нуля, якщо хоча б один пристрій притягує її до землі. Це унеможливлює пошкодження виходів при одночасній роботі кількох пристроїв та є основою механізму арбітражу магістралі. У симуляторі Wokwi підтягувальні резистори вбудовані у модель шини і на схемі окремо не зображуються.

Кожний підпорядкований пристрій має власну адресу. У базовому варіанті стандарту адреса є семибітовою, що теоретично дає 128 адрес, з яких 16 зарезервовано для службових потреб, тому на одній магістралі можна розрізнити до 112 пристроїв. Адреса задається виробником мікросхеми; частина її розрядів у багатьох мікросхемах може перемикатися зовнішніми виводами, що дозволяє під'єднати до магістралі кілька однотипних пристроїв. Так, давач BMP180 має адресу 0x77, а індикатор SSD1306 – адресу 0x3C [12, 13]. Стандарт передбачає також розширення до десятибітової адресації.

Обмін даними на шині виконується транзакціями. Транзакція розпочинається умовою СТАРТ (START) і завершується умовою СТОП (STOP). Умова СТАРТ формується переходом лінії SDA з високого рівня в низький за високого рівня лінії SCL. Умова СТОП формується зворотним переходом лінії SDA з низького рівня у високий також за високого рівня SCL. В усіх інших випадках рівень на лінії SDA повинен залишатися незмінним протягом високого рівня SCL, а зміна стану даних відбувається лише за низького рівня тактового сигналу. Часова діаграма транзакції показана на рис. 25.

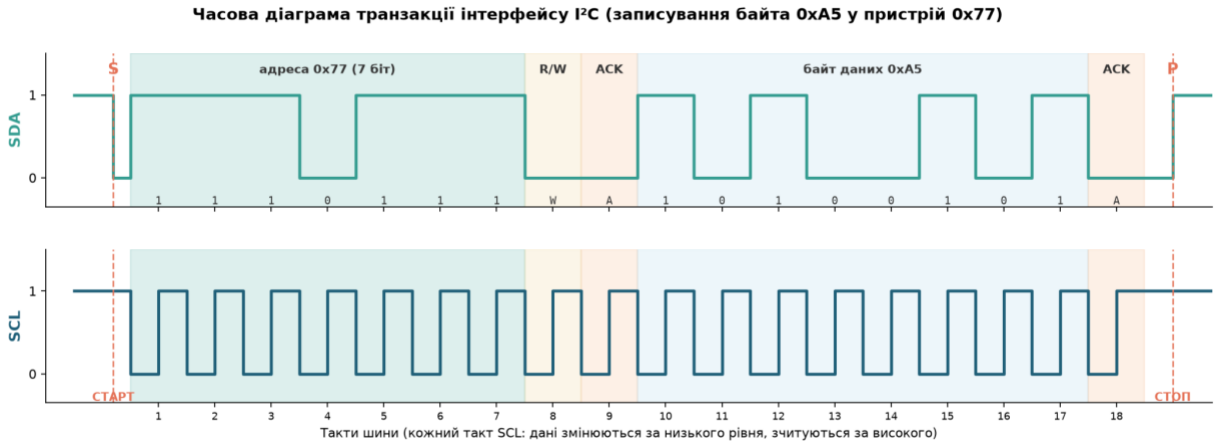


Рис. 25. Часова діаграма транзакції інтерфейсу I²C

Після умови СТАРТ провідний пристрій передає байт, у якому сім старших розрядів становлять адресу підпорядкованого пристрою, а молодший розряд визначає напрям обміну: значення 0 відповідає записуванню даних у підпорядкований пристрій, значення 1 – зчитуванню даних з нього. Кожний переданий байт супроводжується дев’ятим тактом, протягом якого приймач формує біт підтвердження. Якщо приймач притягує лінію SDA до низького рівня, формується підтвердження ACK (Acknowledge), що засвідчує успішне приймання байта. Якщо лінія залишається у високому стані, формується заперечення NACK (Not Acknowledge), яке означає відсутність пристрою із заданою адресою, його неготовність або завершення зчитування з боку провідного пристрою. Відсутність підтвердження після передавання адреси є основною ознакою того, що пристрій на магістралі відсутній.

Більшість периферійних мікросхем має регістрову програмну модель, тобто внутрішній адресний простір регістрів, доступ до яких відбувається за їх номером. Для зчитування регістра застосовують складену транзакцію: провідний пристрій передає адресу підпорядкованого пристрою з ознакою записування та номер потрібного регістра, після чого замість умови СТОП формує повторну умову СТАРТ (repeated START) і передає адресу того самого пристрою вже з ознакою зчитування. Повторна умова СТАРТ не звільняє

магістраль, чим унеможливлює втручання іншого провідного пристрою між двома частинами обміну. Структуру складеної транзакції зчитування регістра наведено на рис. 26.



Рис. 26. Структура складеної транзакції зчитування регістра з повторною умовою СТАРТ

4.2. Режими швидкості обміну шини I²C

Стандарт визначає кілька режимів швидкості обміну, наведених у табл. 14. Швидкість визначається частотою тактового сигналу SCL, який формує провідний пристрій, і не потребує узгодження з підпорядкованим пристроєм так, як це відбувається в асинхронному інтерфейсі. Підпорядкований пристрій, якому потрібен додатковий час на опрацювання даних, може примусово утримувати лінію SCL у низькому стані. Такий механізм називають розтягуванням такту (clock stretching); провідний пристрій при цьому призупиняє обмін до звільнення лінії.

Таблиця 14

Режими швидкості обміну шини I²C

Режим	Максимальна швидкість
Standard mode	100 кбіт/с
Fast mode	400 кбіт/с
Fast mode plus	1 Мбіт/с
High-speed mode	3,4 Мбіт/с
Ultra-fast mode	5 Мбіт/с (лише передавання від провідного пристрою)

4.3. Послідовний периферійний інтерфейс SPI

SPI (Serial Peripheral Interface) – синхронний послідовний інтерфейс, запропонований компанією Motorola для обміну між мікроконтролером і периферійними мікросхемами на одній платі: давачами, мікросхемами пам'яті EEPROM і Flash, аналого-цифровими перетворювачами, дисплеями, радіомодулями [1]. На відміну від I²C, для SPI не існує єдиного стандарту протоколу: формат команд, порядок бітів і допустимі режими визначає документація конкретної мікросхеми.

Інтерфейс використовує чотири лінії (рис. 27):

– SCK (Serial Clock) – тактовий сигнал, який формує провідний пристрій;
– MOSI (Master Out, Slave In) – лінія передавання даних від провідного пристрою до підпорядкованого;
– MISO (Master In, Slave Out) – лінія передавання даних від підпорядкованого пристрою до провідного;
– CS (Chip Select, також SS – Slave Select) – сигнал вибору підпорядкованого пристрою з активним низьким рівнем; для кожного підпорядкованого пристрою передбачено окрему лінію CS.

У новіших документах лінії даних позначають також як SDO/SDI або COPI/CIPO (Controller Out, Peripheral In / Controller In, Peripheral Out), проте їх призначення не змінюється.

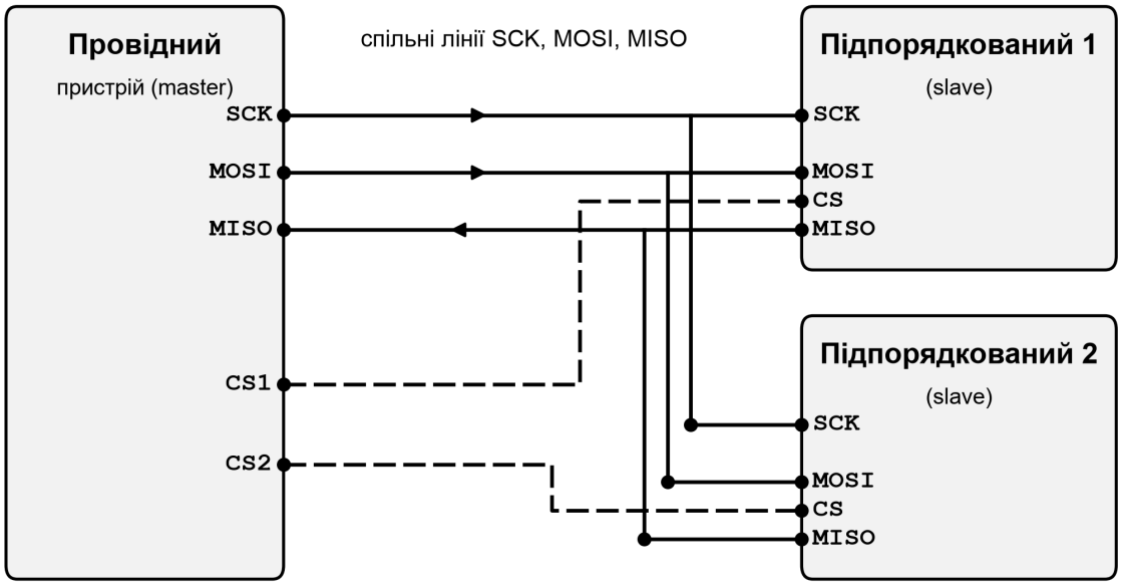


Рис. 27. Під'єднання двох підпорядкованих пристроїв до шини SPI: лінії SCK, MOSI, MISO спільні, лінія CS – окрема для кожного пристрою

Обмін у SPI організовано як взаємний обмін вмістом двох зсувних регістрів. Провідний пристрій активує сигнал CS вибраного пристрою і формує на лінії SCK серію тактових імпульсів; з кожним тактом один біт виводиться з регістра провідного пристрою на лінію MOSI, а один біт регістра підпорядкованого пристрою – на лінію MISO. За вісім тактів пристрої обмінюються байтами, тобто передавання і приймання завжди відбуваються одночасно (повний дуплекс). Щоб лише зчитати дані, провідний пристрій передає «порожні» байти (зазвичай 00h або FFh). Після завершення обміну сигнал CS деактивується, а вихід MISO невибраного пристрою переходить у високоімпедансний стан, що дозволяє об’єднувати лінії MISO кількох пристроїв.

Моменти зміни та зчитування даних визначають два параметри: полярність тактового сигналу CPOL (рівень SCK у стані спокою) та фаза CPHA (за яким фронтом тактового імпульсу – першим чи другим – зчитуються дані). Їх комбінації утворюють чотири режими роботи SPI (табл. 15); провідний і підпорядкований пристрої мають бути налаштовані на один режим. Найпоширенішими є режими 0 і 3. Часову діаграму обміну байтом у режимі 0 показано на рис. 28.

Таблиця 15

Режими роботи інтерфейсу SPI

Режим	CPOL	CPHA	Рівень SCK у стані спокою	Зчитування даних
0	0	0	низький	за першим (наростаючим) фронтом
1	0	1	низький	за другим (спадаючим) фронтом
2	1	0	високий	за першим (спадаючим) фронтом
3	1	1	високий	за другим (наростаючим) фронтом

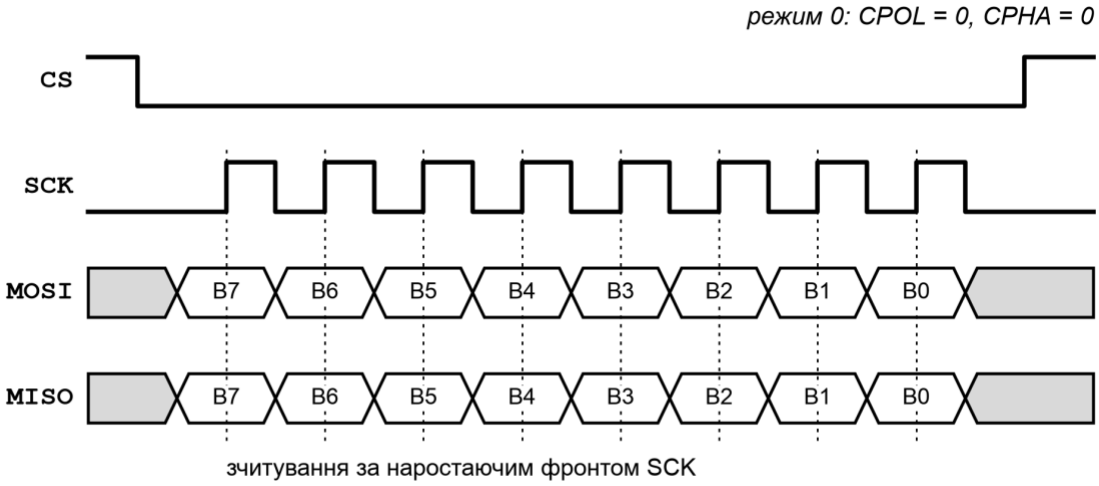


Рис. 28. Часова діаграма обміну байтом через інтерфейс SPI у режимі 0 (CPOL = 0, CPHA = 0)

Відсутність адресації та підтвердження спрощує протокол і дозволяє досягати тактових частот у десятки мегагерців – значно вищих, ніж у шини I²C. Платою за це є більша кількість ліній: кожен додатковий підпорядкований пристрій потребує окремої лінії CS, а провідний пристрій не отримує апаратного підтвердження того, що пристрій присутній і прийняв дані. Для швидкісного доступу до зовнішньої Flash-пам’яті, зокрема пам’яті програм мікроконтролерів, застосовують розширення Dual SPI і Quad SPI (QSPI), у яких дані передаються двома або чотирма лініями одночасно. У мікроконтролерах обмін SPI зазвичай поєднують з контролером прямого доступу до пам’яті (DMA), що звільняє процесорне ядро від побайтового копіювання даних; в операційній системі Linux пристрої SPI доступні прикладним програмам через файли `/dev/spidev*`.

Порівняння основних характеристик послідовних інтерфейсів, які розглядаються у посібнику, наведене в табл. 16. Завдяки повнодуплексному обміну, відсутності адресації та двотактним (push-pull) виходам, фронти сигналів яких не обмежені підтягувальними резисторами, як у шині I²C, SPI забезпечує вищу швидкість, однак потребує більшої кількості ліній зв’язку, що робить його менш зручним за значної кількості пристроїв. Тому для під’єднання давачів і повільної периферії в межах плати частіше обирають I²C, а для швидкісних дисплеїв, пам’яті та радіомодулів – SPI.

Таблиця 16

Порівняння послідовних інтерфейсів

Ознака	RS-232C (UART)	I²C	SPI
Синхронізація	асинхронна	синхронна	синхронна
Кількість ліній даних	2 (TXD, RXD)	2 (SDA, SCL)	4 і більше

Ознака	RS-232C (UART)	I²C	SPI
та такту			
Топологія	радіальна	магістральна	магістральна з окремим CS
Режим обміну	дуплексний	напівдуплексний	дуплексний
Адресація	відсутня	7 або 10 біт	лінією CS
Типова швидкість	до 115200 біт/с	100 кбіт/с – 3,4 Мбіт/с	до десятків Мбіт/с

4.4. Дослідження магістралі логічним аналізатором

Для дослідження роботи магістралі застосовують логічний аналізатор – засіб, який фіксує стан ліній зв’язку в часі та відображає його у вигляді часових діаграм. На відміну від амплітудно-часових діаграм, побудованих програмно за моделлю сигналу (практичне завдання № 2), логічний аналізатор відображає переходи рівнів на лініях SDA та SCL (у симуляторі Wokwi – на лініях моделі шини) [17]. Засоби декодування дозволяють подати зафіксовану послідовність у вигляді умов СТАРТ і СТОП, адреси, ознаки напряму обміну, байтів даних та бітів підтвердження. Приклад декодування транзакції показано на рис. 29.

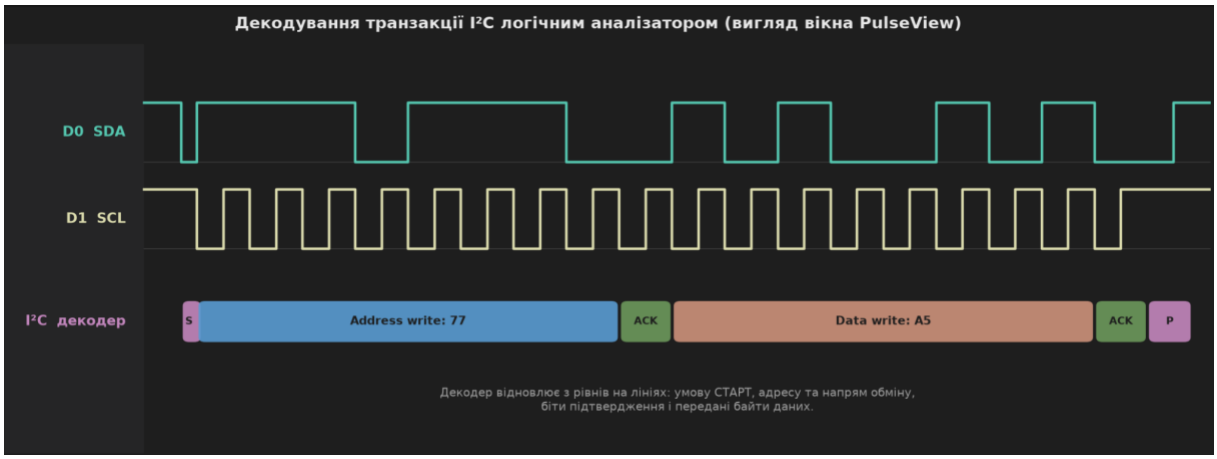


Рис. 29. Декодування транзакції I²C логічним аналізатором

4.5. Вузол моніторингу та рівні програмного забезпечення

Практичне завдання № 5 передбачає об’єднання розглянутих інтерфейсів у складі одного вузла – системи моніторингу, характерної для вбудованих комп’ютерних систем. Структурна схема такого вузла наведена на рис. 30. Периферійний давач під’єднано до мікроконтролера через шину I²C; мікроконтролер із заданою періодичністю опитує давач і передає результати вимірювання через послідовний асинхронний інтерфейс UART до комп’ютера; програма на комп’ютері виконує розбирання прийнятих рядків, збереження результатів у файл та побудову графіка.



Периферійний давач під'єднано до мікроконтролера шиною I²C; результати вимірювання передаються до комп'ютера інтерфейсом UART і опрацьовуються прикладною програмою.

Рис. 30. Структурна схема вузла моніторингу

Розроблення такого вузла ілюструє поняття рівнів програмного забезпечення периферійної підсистеми [3]. Найнижчий рівень утворює доступ до регістрів мікросхеми давача через апаратний контролер шини; наступний рівень – програмний драйвер пристрою, який приховує особливості регістрової моделі та надає прикладній програмі результат вимірювання у фізичних величинах; вище розташований рівень передавання даних каналом зв’язку, який визначає формат телеметричного повідомлення; найвищий рівень – прикладна програма, що опрацьовує та зберігає отримані дані. Розподіл функцій за рівнями наведено на рис. 31.



Рис. 31. Рівні програмного забезпечення вузла моніторингу

В операційних системах сімейства Linux доступ до розглянутих інтерфейсів уніфіковано у вигляді файлів пристроїв: магістраль I²C подається файлами `/dev/i2c-*`, послідовні порти – файлами `/dev/tty*`. Прикладна програма звертається до них через стандартні системні виклики, а безпосередню взаємодію з апаратурою виконує драйвер у просторі ядра. Такий поділ на простір користувача та простір ядра є типовим для сучасних комп'ютерних систем і пояснює, чому операції записування даних у файл на накопичувачі не є розробленням драйвера периферійного пристрою.

Обов'язковою умовою працездатності вузла є узгодження формату телеметричного повідомлення між програмою мікроконтролера та програмою комп'ютера. Формат визначає ознаку початку рядка, роздільник, кількість значущих цифр та символи завершення рядка. Невідповідність формату призводить до того, що фізично правильно передані дані не можуть бути розібрані приймальною програмою, що є характерною помилкою під час відлагодження подібних систем.

4.6. Практичне завдання № 4. Дослідження шини I²C та розроблення програмного драйвера периферійного давача

Мета завдання: опанування студентом принципів роботи синхронної двопровідної магістральної шини I²C, адресації пристроїв master–slave, технології створення програмного драйвера доступу до регістрів периферійної мікросхеми та дослідження реальних сигналів магістралі логічним аналізатором на платформі ESP32 у симуляторі Wokwi.

Постановка завдання. Тип периферійної мікросхеми студенту визначає викладач згідно з таблицею варіантів (табл. А.1). Для варіантів з індикатором (4, 6, 9) замість давача використовується графічний OLED-індикатор з контролером SSD1306 (адреса 0x3C), на який потрібно вивести повідомлення – прізвище латиницею (літери A–Z, наприклад IVANOV). Частота тактового сигналу магістралі – 100 кГц, що відповідає режиму Standard mode.

Технологія виконання завдання. Програмний драйвер має бути налаштований на характеристики, індивідуально задані студенту. Насамперед це тип периферійної мікросхеми та її адреса на магістралі, а для варіантів з індикатором – також інформаційне повідомлення (прізвище). Спочатку потрібно визначитися зі структурою транзакцій, які виконує провідний пристрій. У цьому завданні їх дві. Перша – сканування магістралі: провідний пристрій для кожної можливої адреси формує умову СТАРТ і байт адреси з ознакою записування, після чого перевіряє, чи надійшло підтвердження ACK. Наявність підтвердження означає, що пристрій із такою адресою присутній на шині, відсутність – що адреса вільна. Жодного байта даних при цьому не передається. Друга – складена транзакція зчитування регістра: провідний пристрій передає адресу підпорядкованого пристрою з ознакою записування та номер потрібного регістра, після чого замість умови СТОП формує повторну умову СТАРТ і передає ту саму адресу вже з ознакою зчитування, після чого приймає байти даних. Повторна умова СТАРТ не звільняє магістраль і унеможливлює втручання іншого провідного пристрою між двома частинами обміну.

Далі визначають параметри магістралі: виводи мікроконтролера, до яких під’єднано лінії SDA і SCL, та частоту тактового сигналу. У навчальному проєкті лінію даних SDA під’єднано до виводу GPIO21, лінію тактового сигналу SCL – до виводу GPIO22, частоту задано 100 кГц. Виходи пристроїв виконано за схемою з відкритим стоком, тому перехід ліній до рівня логічної одиниці забезпечують підтягувальні резистори; у моделі симулятора вони вбудовані в шину і на схемі окремо не зображуються.

Для створення драйвера необхідно розробити відповідну схему алгоритму (ініціалізація контролера шини → сканування магістралі та перевірка наявності пристрою за заданою адресою → зчитування калібрувальних коефіцієнтів мікросхеми → циклічне запускання вимірювання і приймання результату → перетворення кодів у фізичні величини → виведення). Розроблення схеми алгоритму є одним із показників поглибленого розуміння логіки взаємодії пристроїв у периферійній підсистемі на основі магістрального послідовного інтерфейсу.

У курсі програмний драйвер реалізується трьома пов'язаними файлами.

Приклади реалізації (для варіантів з давачем тиску):

Опис апаратної частини: `diagram.json` (лістинг Б.13). Плата ESP32, мікросхема давача BMP180 (адреса 0x77) та під'єднаний до ліній SDA/SCL логічний аналізатор.

Драйвер пристрою: `bmp180.py` (лістинг Б.11). Зчитування з регістрів, починаючи з адреси 0xAA, одинадцяти калібрувальних коефіцієнтів мікросхеми; записування коду команди в регістр керування 0xF4; витримування часу перетворення; зчитування результату з регістра 0xF6; повернення температури у градусах Цельсія та тиску в гектопаскалях.

Прикладна програма: `main.py` (лістинг Б.12). Створення об'єкта контролера шини, сканування магістралі, періодичне звертання до драйвера і виведення результату у Serial Monitor.

Serial Monitor – це текстовий термінал, у якому відображається все, що мікроконтролер передає інтерфейсом UART; у симуляторі це нижня панель вікна, а на реальній платі його роль виконує термінальна програма, під'єднана до віртуального COM-порту через перетворювач USB-UART. У файлі `diagram.json` термінал описано як компонент схеми, під'єднаний до виводів TX і RX мікроконтролера перехресно, тому оператор print не є виведенням у консоль: рядок фізично передається лінією як послідовність старт-стопних кадрів, по одному на кожний символ. Serial Monitor показує готовий текст, переданий лінією TXD, тоді як логічний аналізатор фіксує рівні напруги на лініях SDA і SCL магістралі I²C.

Логічний аналізатор у симуляторі записує рівні на дротах, але не відображає осцилограму у вікні браузера. Після зупинення симуляції формується файл `wokwi-logic.vcd` у форматі **Value Change Dump** (VCD), який відкривають у зовнішній програмі. Реалізація зчитування сигналів реальним апаратним логічним аналізатором – це додаткова задача у цьому практичному завданні для отримання максимальної оцінки.

Кроки виконання

У прикладах використано прізвище PETRENKO і давач BMP180 (як типовий давач температури та тиску з таблиці варіантів). Під час виконання практичного завдання потрібно підставити власне прізвище (латиницею A–Z) та параметри свого варіанту (номер варіанту, тип периферійної мікросхеми).

Крок 1. Визначення вихідних даних. За таблицею варіантів (табл. А.1) студент фіксує: номер варіанту, тип периферійної мікросхеми та її адресу на магістралі, частоту тактового сигналу і відповідний режим швидкості обміну, а для варіантів з індикатором – також повідомлення, тобто власне прізвище латиницею. Це саме повідомлення має бути у змінній SURNAME файла `main.py` і на екрані індикатора після запуску.

Спочатку потрібно визначитися зі структурою транзакцій. У звіті навести обидві: сканування магістралі (умова СТАРТ, байт адреси з ознакою записування, аналіз підтвердження ACK, умова СТОП) та складене зчитування регістра (адреса з ознакою записування, номер регістра, повторна умова СТАРТ, адреса з ознакою зчитування, байти даних, заперечення NACK від провідного пристрою, умова СТОП). Указати частоту тактового сигналу і відповідний режим за стандартом. У звіті також порівняти інтерфейси I²C та SPI за синхронізацією, кількістю ліній, топологією, способом адресації та швидкістю.

Крок 2. Складання проєкту в симуляторі. Відкрити середовище Wokwi MicroPython ESP32 [15, 16] (https://wokwi.com/projects/new/micropython-esp32) і створити три файли, вставивши їх вміст із додатка Б:

- `main.py` – прикладна програма (лістинг Б.12);
- `diagram.json` – опис схеми: плата, давач, логічний аналізатор (лістинг Б.13);
- `bmp180.py` – драйвер давача (лістинг Б.11).

Для варіантів з індикатором використовують файли `main.py` і `diagram.json` проєкту з індикатором (лістинги Б.15, Б.16) та драйвер `ssd1306.py` (лістинг Б.14), а у файлі `main.py` замінюють значення змінної SURNAME на власне прізвище.

Файл драйвера створюють у проєкті Wokwi як окремий новий файл (File → New file) з тим самим іменем, що й у лістингу, поруч із `main.py` та `diagram.json`.

За наявності фізичної плати ESP32 і відповідного давача роботу програми можна продемонструвати на реальному обладнанні (для давача BME280 потрібен окремий драйвер – див. примітку до табл. А.1).

Крок 3. Сканування магістралі та зчитування давача. Запустити симуляцію (Start Simulation) і відкрити Serial Monitor (рис. 32).

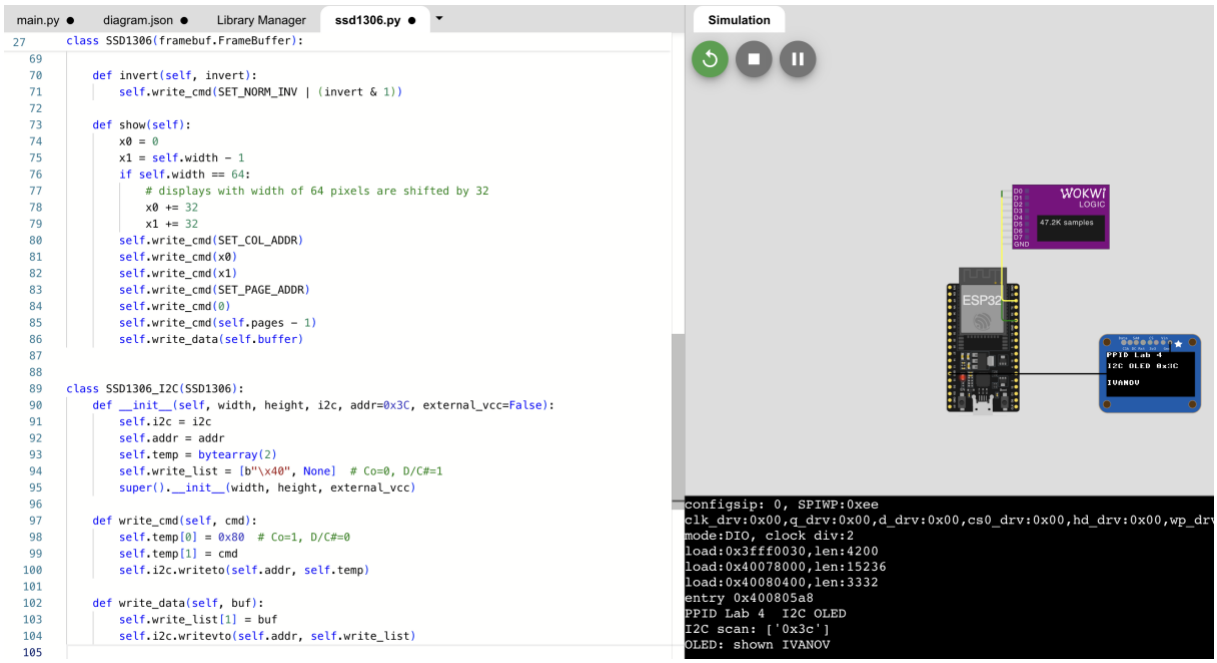


Рис. 32. Приклад симуляції OLED-дисплея у Wokwi

Очікуваний вивід консолі для варіантів з індикатором:

```
PPID Lab 4 — I2C OLED
I2C scan: ['0x3c']
OLED: shown IVANOV
```

Для варіантів з давачем очікуваний вивід:

```
PPID Lab 4 — I2C sensor
I2C scan: ['0x77']
TEMP=24.0 PRESS=1013.2
TEMP=24.0 PRESS=1013.2
```

Де рядок I²C scan – результат сканування магістралі: список адрес, від яких надійшло підтвердження ACK. Наявність у списку значення 0x77 підтверджує, що давач присутній на шині та відповідає за своєю адресою. Рядки TEMP і PRESS – результат роботи драйвера: температура у градусах Цельсія і тиск у гектопаскалях (задані в моделі давача 101325 Па відповідають 1013,25 гПа), обчислені з кодів регістрів за калібрувальними коефіцієнтами мікросхеми.

У звіті навести рядок сканування, кілька рядків результатів вимірювання та знімок екрана вікна Serial Monitor, а для варіантів з індикатором – також знімок екрана екрана з виведеним прізвищем.

Крок 4. Перевірка незалежності прикладної програми від апаратури. Для варіантів з давачем під час роботи симуляції клацнути на зображенні давача на схемі – з’являться повзунки temperature і pressure. Змінити їх

положення і переконатися, що значення у Serial Monitor оновилися без будь-яких змін у програмі. Це демонструє, що прикладна програма оперує результатом вимірювання у фізичних величинах, а не станом ліній магістралі: перетворення виконує драйвер. У звіті навести два знімки екрана – до і після зміни значень.

Крок 5. Дослідження магістралі логічним аналізатором. Переконатися, що під час симуляції на логічному аналізаторі зростає лічильник відліків, після чого натиснути Stop – браузер завантажить файл `wokwi-logic.vcd`. Відкрити файл у програмі PulseView [17]: Open → Import Value Change Dump data…, у діалозі імпорту встановити Downsampling factor = 50. Додати декодер I²C (кнопка Add protocol decoder), указавши канали SDA = D0 (GPIO21) і SCL = D1 (GPIO22).

У звіті навести власний знімок екрана із позначеними умовою СТАРТ, байтом адреси (на лінії передається байт EEh – адреса 0x77, зсунута на один розряд, з бітом R/W = 0; декодер PulseView показує її як 0x77), бітом підтвердження ACK та умовою СТОП. Пояснити, чому у стані спокою обидві лінії перебувають на високому рівні та чому активність на магістралі має вигляд окремих пачок переходів, розділених паузами. Пояснити відмінність цього запису від амплітудно-часової діаграми практичного завдання № 2: логічний аналізатор фіксує переходи рівнів на лініях шини (у симуляторі – моделі шини), а не програмно побудовану модель сигналу.

За неможливості встановити PulseView потрібно використати веб-переглядач Surfer (https://app.surfer-project.org/), перетягнувши файл у вікно браузера і додавши сигнали D0 і D1. Декодера інтерфейсу I²C ця програма не має, тому адресу пристрою у звіті підтверджують результатом сканування магістралі з Serial Monitor.

Крок 6. Порівняння і аналіз результатів. Проаналізувати отримані результати і оформити звіт.

4.7. Практичне завдання № 5. Розроблення та дослідження інтегрованого вузла моніторингу комп'ютерної системи

Мета завдання: опанування студентом технології інтеграції магістральної шини I²C, послідовного асинхронного інтерфейсу UART та рівня зберігання даних у складі єдиного вузла моніторингу; набуття навичок розроблення багаторівневого програмного забезпечення периферійної підсистеми – від драйвера мікросхеми давача до прикладної програми опрацювання телеметрії.

Постановка завдання. Інтервал опитування давача (200, 500 або 1000 мс), швидкість обміну і формат кадру інтерфейсу UART визначає варіант (табл. А.1). Формат телеметричного рядка – TEMP=<значення> із завершенням символами повернення каретки та переведення рядка.

Технологія виконання завдання. Вузол моніторингу складається з трьох частин: периферійної мікросхеми давача, під'єднаної до мікроконтролера магістраллю I²C; програми мікроконтролера, яка з заданою періодичністю опитує давач і передає результат вимірювання інтерфейсом UART; прикладної програми комп'ютера, яка приймає рядки телеметрії, зберігає їх у файл і будує графік. Спочатку потрібно визначитися з розподілом функцій між цими частинами та з форматом телеметричного повідомлення, який є угодою між прошивкою мікроконтролера і програмою комп'ютера.

Формат телеметричного повідомлення визначає ознаку початку рядка, роздільник, кількість значущих цифр і символи завершення рядка. У навчальному проєкті прошивка формує рядок виду TEMP=22.1 з одним знаком після коми. Програма комп'ютера відшукує в кожному прийнятому рядку цю ознаку за регулярним виразом і вилучає числове значення; рядки, які не відповідають формату, пропускаються без аварійного завершення програми. Невідповідність формату призводить до того, що фізично правильно передані дані не можуть бути розібрані приймальною програмою – це характерна помилка під час відлагодження подібних систем, оскільки зовні вона виглядає як відсутність зв'язку, хоча канал передавання справний.

Слід звернути увагу на те, що програма мікроконтролера використовує два різні порти UART. Порт UART0 задіяно інтерпретатором MicroPython для стандартного потоку виведення: усе, що виводить оператор print, потрапляє у Serial Monitor – текстовий термінал, під'єднаний у схемі до виводів TX і RX мікроконтролера перехресно. Порт UART1, налаштований окремо на швидкість 9600 біт/с з виводом передавача на GPIO17, є власне каналом телеметрії: саме в нього метод write передає сформований рядок. Прошивка навмисно дублює рядок в обидва порти. У каналі UART1 на протилежному кінці лінії в умовах

симулятора приймача немає, тому джерелом даних для прикладної програми слугує копія, збережена з Serial Monitor. На реальному обладнанні дублювання не потрібне: комп'ютер під'єднують безпосередньо до ліній порту UART1 через перетворювач USB-UART, і прикладна програма читає дані з віртуального COM-порту.

Логічний аналізатор у схемі під'єднано до трьох ліній: до ліній SDA і SCL магістралі I²C (канали D0 і D1) та до лінії передавача порту UART1 (канал D2). Це дозволяє в одному записі спостерігати обидва інтерфейси одночасно: короткі пачки переходів на лініях магістралі, що відповідають опитуванню давача, і старт-стопні кадри на лінії UART, що відповідають передаванню телеметричного рядка. Період повторення цих груп дещо більший за заданий інтервал опитування: затримка sleep_ms додається до часу перетворення давача і виведення рядка, тому для точної періодичності використовують таймери або функцію ticks_ms. Саме цей запис є основним експериментальним підтвердженням того, що обидва інтерфейси працюють у складі одного вузла узгоджено.

Для створення вузла необхідно розробити схеми алгоритмів обох програм. Для прошивки мікроконтролера: ініціалізація контролера магістралі та порту UART → зчитування калібрувальних коефіцієнтів давача → у циклі: запускання вимірювання, приймання результату, перетворення кодів у фізичну величину, формування телеметричного рядка, передавання його в порт → витримування заданого інтервалу. Для прикладної програми комп'ютера: відкриття джерела даних → построкове читання → розпізнавання ознаки та вилучення числового значення → накопичення масиву результатів → записування у файл формату CSV → побудова графіка. Розроблення схем алгоритмів є одним із показників поглибленого розуміння розподілу функцій між рівнями програмного забезпечення периферійної підсистеми.

У курсі вузол моніторингу реалізується чотирма пов'язаними файлами. Приклади реалізації:

Опис апаратної частини: `diagram.json` (лістинг Б.18). Плата ESP32, мікросхема давача BMP180 (адреса 0x77) та логічний аналізатор, під'єднаний до ліній магістралі і до лінії передавача телеметрії.

Драйвер пристрою: `bmp180.py` (лістинг Б.11). Той самий драйвер, що в практичному завданні № 4: доступ до регістрів мікросхеми, калібрування, повернення температури у градусах Цельсія.

Прошивка мікроконтролера: `main.py` (лістинг Б.17). Ініціалізація магістралі та порту UART, циклічне опитування давача із заданим інтервалом INTERVAL_MS, формування та передавання телеметричного рядка.

Прикладна програма комп'ютера: `host/capstone_host.py` (лістинг Б.19). Розбирання рядків телеметрії, записування результатів у файл CSV, побудова графіка та експорт на імітаційний накопичувач.

Структурну схему вузла моніторингу наведено на рис. 30, а рівні його програмного забезпечення – на рис. 31 (підрозділ 4.5). Реалізація приймання телеметрії безпосередньо з послідовного порту в реальному часі – це додаткова задача у цьому практичному завданні для отримання максимальної оцінки.

Кроки виконання

У прикладах використано інтервал опитування 500 мс і давач BMP180. Під час виконання практичного завдання потрібно підставити параметри свого варіанту (номер варіанту, інтервал опитування, швидкість обміну і формат кадру інтерфейсу UART).

Крок 1. Визначення вихідних даних. За таблицею варіантів (табл. А.1) студент фіксує: номер варіанту, інтервал опитування давача, швидкість обміну та формат кадру інтерфейсу UART, а також формат телеметричного рядка. У звіті навести структурну схему вузла (рис. 30) з позначенням, який інтерфейс з'єднує кожну пару елементів, і зазначити, які саме параметри є предметом угоди між прошивкою та програмою комп'ютера.

Крок 2. Складання проєкту в симуляторі. Відкрити середовище Wokwi MicroPython ESP32 [15, 16] (https://wokwi.com/projects/new/micropython-esp32) і створити три файли, вставивши їх вміст із додатка Б:

- `main.py` – прошивка вузла (лістинг Б.17);
- `diagram.json` – опис схеми (лістинг Б.18);
- `bmp180.py` – драйвер давача (лістинг Б.11).

У файлі `main.py` виставити значення змінної INTERVAL_MS, що дорівнює інтервалу опитування свого варіанту, а параметри порту UART1 – відповідно до заданої швидкості обміну. Для варіантів з іншим форматом кадру задають також кількість бітів даних, тип контролю парності та кількість стопових бітів, наприклад для варіанту 5 (9600 біт/с, 7E1): `UART(1, baudrate=9600, bits=7, parity=0, stop=1, tx=17, rx=16)`, де parity=0 означає контроль на парність, parity=1 – на непарність, parity=None – без контролю.

За наявності фізичної плати ESP32 і відповідного давача роботу програми можна продемонструвати на реальному обладнанні (для давача BME280 потрібен окремий драйвер – див. примітку до табл. А.1).

Крок 3. Накопичення телеметрії. Запустити симуляцію (Start Simulation) і відкрити Serial Monitor.

Очікуваний вивід консолі:

```
PPID Lab 5 — Capstone node
I2C: ['0x77']
TEMP=22.1
TEMP=22.3
TEMP=22.5
```

Де рядок I²C – результат сканування магістралі, який підтверджує наявність давача за адресою 0x77; наступні рядки – телеметрія, що надходить із заданим інтервалом. Під час роботи симуляції клацнути на зображенні давача і кілька разів змінити положення повзунка temperature, щоб у записі були різні значення – графік буде змістовнішим. Накопичений вивід зберегти у текстовий файл `my_log.txt`. У файлі мають бути рядки з ознакою TEMP=; рядки іншого вигляду програма пропускає. Значення температури залежать від положення повзунка давача (на початку симуляції – 24 °C).

Крок 4. Дослідження обох інтерфейсів логічним аналізатором. Натиснути Stop – браузер завантажить файл `wokwi-logic.vcd`. Відкрити його у програмі PulseView (Open → Import Value Change Dump data…, Downsampling factor = 50) і додати канали D0 (SDA), D1 (SCL) та D2 (лінія передавача UART1). До каналів D0 і D1 під'єднати декодер I²C, до каналу D2 – декодер UART, указавши задану швидкість обміну і формат кадру.

У звіті навести знімок екрана, на якому в межах одного інтервалу опитування видно пачку транзакцій магістралі I²C і наступну за нею послідовність кадрів UART. Пояснити, чому ці групи чергуються з періодичністю, близькою до заданого інтервалу, чому реальний період дещо більший, та зіставити кількість кадрів UART з довжиною телеметричного рядка у символах.

Крок 5. Опрацювання телеметрії прикладною програмою. З кореневого каталогу репозиторію запустити програму комп'ютера (лістинг Б.19), указавши збережений файл:

```
$ python -m host.capstone_host --input my_log.txt --plot capstone_plot.png \
      --csv readings.csv
```

Очікуваний вивід:

```
Знайдено зчитувань: 10
CSV: readings.csv
Графік: capstone_plot.png
```

Файл `readings.csv` містить заголовок і по одному рядку на кожне розпізнане зчитування, де перше поле – індекс рядка у вихідному записі (відлік від нуля), друге – значення температури:

```
sample,temp_c
2,22.1
3,22.3
4,22.5
```

У звіті навести фрагмент файла `my_log.txt`, вміст або таблицю файла `readings.csv` та графік `capstone_plot.png`. Пояснити, чому перше зчитування має індекс 2.

Крок 6. Експорт результатів на накопичувач. Повторити запуск програми з додатковим параметром `--export-usb`. Копіювання файла CSV у тимчасовий каталог імітаційного накопичувача виконується засобами файлової системи, як і запис у практичному завданні № 3; програма виводить шлях до каталогу, проте цей каталог автоматично видаляється після завершення програми, тому у звіті наводять виведений шлях. У звіті пояснити, що ця операція належить до рівня прикладної програми і не є розробленням драйвера накопичувача.

Крок 7. (Для максимальної оцінки.) Приймання телеметрії в реальному часі.

а) реалізувати приймання рядків безпосередньо з послідовного порту засобами бібліотеки pySerial [14], без проміжного збереження у файл;
б) використати віртуальну пару портів з практичного завдання № 1 або реальний перетворювач USB-UART;
в) забезпечити побудову графіка в міру надходження даних;
г) порівняти отриману реалізацію з опрацюванням збереженого запису і пояснити, які додаткові задачі виникають у режимі реального часу.

Крок 8. Порівняння і аналіз результатів. Проаналізувати отримані результати і оформити звіт.

Питання для самоперевірки

1. Навести класифікацію інтерфейсу I²C та пояснити зміст термінів «послідовний», «магістральний», «напівдуплексний», «синхронний».
2. Навести призначення ліній SDA і SCL та пояснити роль провідного і підпорядкованого пристроїв.
3. Пояснити, навіщо виходи пристроїв виконано за схемою з відкритим стоком, та яку функцію виконують підтягувальні резистори.
4. Навести особливості структур транзакцій інтерфейсу I²C: умови СТАРТ і СТОП, байт адреси, ознака напряму обміну, біти підтвердження ACK і NACK.
5. Пояснити, як виконується зчитування регістра давача та навіщо застосовують повторну умову СТАРТ.
6. Пояснити, як визначити адресу пристрою на магістралі та за якою ознакою виявляють його відсутність.
7. Навести основні етапи відлагодження драйвера та прикладних програм практичного завдання.
8. Пояснити роботу написаного програмного драйвера.
9. Пояснити основні принципи розроблення схеми алгоритму сканування магістралі.
10. Пояснити основні принципи розроблення схеми алгоритму зчитування регістрів периферійної мікросхеми.
11. Пояснити механізм розтягування такту (clock stretching) і навести режими швидкості обміну інтерфейсу I²C.
12. Чим інтерфейс I²C відрізняється від SPI за кількістю ліній, способом адресації та швидкістю? Чому для під’єднання давачів у межах плати частіше обирають I²C?
13. Чим інтерфейс I²C відрізняється від RS-232C за синхронізацією та топологією?
14. Що показує логічний аналізатор на лініях SDA і SCL під час адресації та чим цей запис відрізняється від діаграми, побудованої програмно у практичному завданні № 2?
15. Які рівні програмного забезпечення реалізують файли `main.py` і `bmp180.py` навчального проєкту і що описує файл `diagram.json`?
16. Поясніть призначення ліній SCK, MOSI, MISO, CS інтерфейсу SPI та режими його роботи, які задають параметри CPOL і CPHA.
17. Навести структурну схему вузла моніторингу та пояснити призначення кожного її елемента.

18. Які інтерфейси використано у складі вузла та яку роль виконує кожний із них?
19. Перелічити рівні програмного забезпечення від мікросхеми давача до файла CSV на комп’ютері та пояснити функції кожного рівня.
20. Пояснити, який складник системи є драйвером пристрою, а який – прикладною програмою.
21. Пояснити, навіщо потрібне узгодження формату телеметричного повідомлення між прошивкою та програмою комп’ютера і до чого призводить його порушення.
22. Пояснити, чому програма мікроконтролера використовує два різні порти UART та яке призначення кожного з них.
23. Навести основні етапи відлагодження вузла у разі, коли давач опитується успішно, а прикладна програма не знаходить жодного зчитування.
24. Пояснити роботу написаних програм – прошивки мікроконтролера і прикладної програми комп’ютера.
25. Пояснити основні принципи розроблення схеми алгоритму циклічного опитування давача із заданим інтервалом.
26. Що видно на записі логічного аналізатора в межах одного інтервалу опитування і чому групи сигналів на лініях I²C та UART чергуються?
27. Як у операційних системах сімейства Linux уніфіковано доступ до інтерфейсів через файли пристроїв `/dev/i2c-*` та `/dev/tty*`?
28. Пояснити поділ на простір користувача та простір ядра і чому записування даних у файл не є розробленням драйвера периферійного пристрою.
29. Як зміниться архітектура вузла, якщо замість інтерфейсу UART використати USB-CDC?
30. Чому реальний період надходження телеметрії перевищує значення INTERVAL_MS?
31. Яку роль виконує імітаційний накопичувач у ланцюжку зберігання даних і чим запис на нього відрізняється від запису на реальний носій?

Тестові завдання

Оберіть одну правильну відповідь. Відповіді до тестових завдань наведено в додатку В.

1. Лінії шини I²C:
   а) TXD, RXD
   б) MOSI, MISO, SCK, CS

в) SDA, SCL
г) D+, D−

2. Умова СТАРТ шини I²C – це:
   а) імпульс на лінії CS
   б) перехід SCL у низький рівень
   в) перехід SDA з низького рівня у високий за високого рівня SCL
   г) перехід SDA з високого рівня в низький за високого рівня SCL
3. Відсутність пристрою із заданою адресою на шині I²C виявляють за:
   а) бітом підтвердження ACK
   б) розтягуванням такту
   в) умовою СТОП
   г) відсутністю підтвердження (NACK) після байта адреси
4. Адреса давача BMP180 на шині I²C:
   а) 0x3C
   б) 0x77
   в) 0x50
   г) 0x68
5. Максимальна швидкість режиму Fast mode шини I²C:
   а) 400 кбіт/с
   б) 100 кбіт/с
   в) 1 Мбіт/с
   г) 3,4 Мбіт/с
6. Повторну умову СТАРТ використовують для:
   а) зміни напряму обміну без звільнення магістралі
   б) завершення транзакції
   в) скидання підпорядкованого пристрою
   г) розтягування такту
7. У режимі 0 інтерфейсу SPI дані зчитуються:
   а) за фронтом сигналу MISO
   б) за спадаючим фронтом SCK, рівень SCK у спокої високий
   в) за рівнем сигналу CS
   г) за наростаючим фронтом SCK, рівень SCK у спокої низький
8. Окрему лінію для кожного підпорядкованого пристрою інтерфейсу SPI має сигнал:
   а) CS
   б) MOSI
   в) MISO
   г) SCK

ТЕРМІНОЛОГІЧНИЙ СЛОВНИК

Адреса пристрою – унікальний номер підпорядкованого пристрою на магістралі (у шині I²C – семи- або десятибітовий, у Modbus RTU – від 1 до 247), який передається на початку кожної транзакції і за яким пристрій розпізнає звернення до себе.

Амплітудно-часова діаграма – графічне зображення послідовності рівнів сигналу на лінії зв'язку в часі; для UART показує стартовий біт, біти даних, біт парності та стопові біти кожного символу.

Арбітраж шини – процедура вирішення конфлікту, коли кілька вузлів одночасно починають передавання; у шинах CAN та I²C арбітраж побітовий і неруйнівний – передавання продовжує вузол, рівень якого переважає на лінії.

Асинхронний інтерфейс – інтерфейс, у якому тактовий сигнал не передається, а приймач синхронізується за стартовим бітом кожного символу з використанням власного генератора, налаштованого на ту саму швидкість обміну.

Біт парності – службовий біт кадру UART, значення якого доповнює кількість одиниць у полі даних до парної (even) або непарної (odd) і дозволяє виявити одиничну помилку передавання.

Бодова швидкість (baud rate) – кількість змін стану лінії зв'язку за одну секунду; в UART один стан лінії кодує один біт, тому бодова швидкість дорівнює бітовій.

Віртуальний COM-порт – програмне подання послідовного порту операційною системою (COM3, /dev/ttyUSB0), яке створює драйвер USB-UART адаптера або програма віртуальної пари портів.

Вставлення біта (bit stuffing) – процедура кодування USB 2.0, за якої після шести послідовних одиниць у потік даних вставляється біт 0, щоб забезпечити зміну рівня сигналу NRZI і підтримати синхронізацію приймача.

Дескриптор USB – структура даних, яку пристрій USB передає хосту під час опитування шини; описує клас пристрою, ідентифікатори VID/PID, конфігурації, інтерфейси та кінцеві точки.

Диференційна пара – дві лінії зв'язку (D+ і D− у USB, A і B у RS-485), логічний стан на яких визначається різницею потенціалів між лініями, що забезпечує придушення синфазних завад.

Драйвер пристрою – програмний компонент, який приховує особливості апаратури (регістрову модель, протокол обміну) і надає прикладній програмі зручний інтерфейс, наприклад результат вимірювання у фізичних величинах.

Дуплексний режим (full-duplex) – режим обміну, за якого дані одночасно передаються в обох напрямках окремими лініями зв’язку (TXD і RXD у RS-232C).

Імітаційний пристрій (mock) – програмна модель пристрою, яка відтворює його поведінку без реальної апаратури; у практичному завданні № 3 – перелік USB-пристроїв із файлу JSON і тимчасовий каталог замість накопичувача.

Інтерфейс 1-Wire – послідовний інтерфейс з однією сигнальною лінією та спільним проводом, у якому кожен пристрій має унікальний 64-бітний серійний номер.

Інтерфейс SPI (Serial Peripheral Interface) – синхронний повнодуплексний послідовний інтерфейс з лініями SCK, MOSI, MISO та окремою лінією вибору CS для кожного підпорядкованого пристрою.

Кадр UART – старт-стопний символ: стартовий біт, поле даних (5–8 біт), біт парності (за наявності) та один або два стопові біти.

Квитування (handshake) – механізм асинхронного обміну, за якого передавач формує строб, а приймач підтверджує готовність або приймання даних сигналом відповіді (BUSY, ACK).

Кінцева точка (endpoint) – буфер у контролері пристрою USB, який є джерелом або приймачем даних; кінцева точка 0 використовується для конфігурації, точки 1–15 – для прикладних даних.

Класовий драйвер – стандартний драйвер операційної системи для всіх пристроїв певного класу USB (HID, CDC ACM, Mass Storage), завдяки якому пристрій працює без встановлення окремого драйвера.

Кодування NRZI – спосіб лінійного кодування USB 2.0, за якого логічний 0 передається зміною рівня сигналу, а логічна 1 – збереженням рівня.

Контрольна сума CRC – циклічний надлишковий код, який обчислюється над даними кадру і передається разом з ними для виявлення помилок (CRC16 у Modbus RTU і пакетах даних USB).

Логічний аналізатор – вимірювальний засіб, який фіксує стан цифрових ліній зв’язку в часі та за допомогою декодерів протоколів подає зафіксовану послідовність у вигляді кадрів, адрес і байтів даних.

Магістральний інтерфейс (шина) – інтерфейс, у якому багато пристроїв під’єднано до спільних ліній зв’язку, а розрізнення пристроїв виконується за адресою.

Модель OSI – семирівнева еталонна модель взаємодії відкритих систем; для послідовного каналу «точка-точка» фактично задіяні фізичний, канальний і прикладний рівні.

Напівдуплексний режим (half-duplex) – режим обміну, за якого дані передаються в обох напрямках по черзі однією лінією або парою ліній (I²C, RS-485).

Опитування шини (enumeration) – процедура, під час якої хост USB виявляє під'єднаний пристрій, зчитує його дескриптори, призначає адресу та обирає драйвер.

Паралельний інтерфейс – інтерфейс, у якому всі біти слова передаються одночасно окремими лініями шини даних (Centronics, ISA, IDE, паралельна шина дисплея типу Intel 8080).

Переривання – сигнал, яким пристрій повідомляє процесор про подію (готовність даних, завершення передавання), після чого процесор виконує обробник переривання.

Перетворювач рівнів – мікросхема (MAX232, MAX3232), яка узгоджує рівні ТТЛ мікроконтролера з рівнями лінії RS-232C і інвертує сигнал.

Перехресні завади (crosstalk) – наведення хибних імпульсів на лінію зв'язку від крутих фронтів сигналу на сусідніх лініях.

Периферійний інтерфейс – сукупність ліній зв'язку, електричних, логічних і конструктивних характеристик та протоколу обміну, що забезпечують взаємодію центральної частини комп'ютерної системи з периферійними пристроями.

Підпорядкований пристрій (slave) – пристрій, який не ініціює обмін, а відповідає на звернення провідного пристрою за своєю адресою.

Підтвердження ACK і заперечення NACK – біт або пакет квитування, яким приймач повідомляє про успішне (ACK) або неуспішне (NACK, у USB – NAK) приймання даних.

Підтягувальний резистор – резистор між лінією зв'язку і шиною живлення, який забезпечує рівень логічної одиниці на лінії з виходами за схемою з відкритим стоком.

Повторна умова СТАРТ (repeated START) – умова СТАРТ, яку провідний пристрій I²C формує замість умови СТОП, щоб змінити напрям обміну без звільнення магістралі.

Послідовний інтерфейс – інтерфейс, у якому біти слова передаються по черзі однією лінією зв'язку або однією диференційною парою.

Приймач-передавач (transceiver) – мікросхема фізичного рівня (MAX485, MAX3485), яка перетворює рівні ТТЛ у диференційний сигнал лінії і навпаки, не аналізуючи змісту даних.

Провідний пристрій (master) – пристрій, який ініціює обмін, формує адресу та (у синхронних інтерфейсах) тактовий сигнал.

Простір ядра і простір користувача – привілейований режим виконання операційної системи з прямим доступом до апаратури та непривілейований режим прикладних програм, які звертаються до апаратури лише через системні виклики.

Протокол керування потоком даних – механізм призупинення передавання, коли приймач не готовий приймати дані: апаратний (сигнали RTS/CTS) або програмний (символи XON/XOFF).

Протокол Modbus RTU – прикладний протокол обміну «запит – відповідь» між провідним і підпорядкованими пристроями, кадр якого містить адресу, код функції, дані та контрольну суму CRC16.

Прямий доступ до пам'яті (DMA) – спосіб обміну, за якого контролер DMA пересилає блок даних між пристроєм і пам'яттю без участі процесора.

Радіальний інтерфейс (точка-точка) – інтерфейс, який забезпечує взаємодію лише двох пристроїв виділеною лінією зв'язку.

Регістрова програмна модель – подання периферійної мікросхеми як набору регістрів, доступ до яких виконується за їх номерами (адресами).

Режим SPI – комбінація полярності CPOL і фази CPHA тактового сигналу інтерфейсу SPI, яка визначає рівень SCK у стані спокою і фронт, за яким зчитуються дані (режими 0–3).

Розтягування такту (clock stretching) – механізм шини I²C, за якого підпорядкований пристрій утримує лінію SCL у низькому стані, щоб отримати додатковий час на опрацювання даних.

Сигнал DE/RE – сигнал керування приймачем-передавачем RS-485 (Driver Enable / Receiver Enable), який вмикає передавач вузла лише на час передавання кадру.

Синхронний інтерфейс – інтерфейс, у якому разом з даними окремою лінією передається тактовий сигнал, за яким приймач визначає моменти зчитування бітів.

Стартовий біт – перший біт кадру UART (логічний 0), перехід до якого зі стану спокою сигналізує приймачу про початок символу.

Стоповий біт – завершальний біт кадру UART (логічна 1), який повертає лінію у стан спокою перед наступним символом.

Стробування – спосіб синхронного обміну паралельною шиною, за якого дані фіксуються фронтом стробувального імпульсу (/WR, /RD) заданої тривалості.

Схема з відкритим стоком (open drain) – схема виходу, яка може лише притягувати лінію до рівня логічного нуля; рівень логічної одиниці забезпечує підтягувальний резистор.

Телеметричне повідомлення – рядок заданого формату з результатом вимірювання (наприклад, TEMP=22.1), який передається від вузла до комп'ютера; формат є угодою між прошивкою і прикладною програмою.

Термінал послідовного порту (Serial Monitor) – програма, яка відображає текст, що надходить послідовним портом, і передає в порт введені символи.

Топологія – спосіб з'єднання пристроїв між собою: точка-точка, шина, зірка, послідовний ланцюжок, дерево, кільце, коміркова мережа.

Транзакція USB – елементарний обмін на шині USB, що складається з пакета маркера (Token), пакета даних (Data) та пакета квитування (Handshake).

Тривалість такту – час передавання одного біта, обернений до швидкості обміну: $T_{такт} = 1 / baudrate$.

Умови СТАРТ і СТОП – переходи лінії SDA шини I²C за високого рівня SCL (з високого рівня в низький і навпаки), якими розпочинається і завершується транзакція.

Універсальний асинхронний приймач-передавач (UART) – апаратний вузол, який перетворює паралельний код у послідовність кадрів для передавання і виконує зворотне перетворення під час приймання.

Файл пристрою – спеціальний файл (наприклад, /dev/ttyUSB0), через який прикладна програма в Linux звертається до драйвера пристрою стандартними файловими операціями.

Формат кадру – позначення параметрів кадру UART: кількість бітів даних, тип контролю парності та кількість стопових бітів (8N1, 7E1, 8N2).

Хост (host) – провідний вузол шини USB (контролер комп'ютера з кореневим концентратором), який керує всім обміном; пристрої обмінюються даними лише через хост.

Часовий перекіс (skew) – неодночасне надходження до приймача бітів одного слова, переданих різними лініями паралельної шини, через різницю довжин ліній і паразитні ємності.

Шина CAN (Controller Area Network) – мультимайстерна послідовна магістральна шина з диференційною парою CAN_H/CAN_L, у якій пріоритет повідомлення визначає його ідентифікатор.

TTL-UART – варіант послідовного асинхронного інтерфейсу, у якому сигнали передаються рівнями ТТЛ (0 В і +3,3 В або +5 В) без інвертування.

СПИСОК ЛІТЕРАТУРИ

Базова

1. Парамуд Я. С. Периферійні пристрої, інтерфейси та драйвери : навч. посіб. / Я. С. Парамуд. – Львів : Магнолія 2006, 2022. – 210 с. – ISBN 978-617-574-248-8.
2. Парамуд Я. С. Лабораторний практикум з дисципліни «Периферійні пристрої, інтерфейси та драйвери» : навч. посіб. / Я. С. Парамуд, А. М. Миц. – Львів : Видавництво Львівської політехніки, 2021. – 100 с. – ISBN 978-966-941-694-0. – URL: http://eom.lp.edu.ua/textbooks/np-ppid.pdf (дата звернення: 11.11.2026).
3. Кіберфізичні системи: технології збору даних : монографія / О. Ю. Бочкарьов, В. А. Голембо, Я. С. Парамуд, В. О. Яцук ; за ред. А. О. Мельника. – Львів : Магнолія 2006, 2019. – 176 с.
4. TIA-232-F. Interface Between Data Terminal Equipment and Data Circuit-Terminating Equipment Employing Serial Binary Data Interchange. – Arlington : Telecommunications Industry Association, 1997.
5. TIA-485-A. Electrical Characteristics of Generators and Receivers for Use in Balanced Digital Multipoint Systems. – Arlington : Telecommunications Industry Association, 1998.
6. MODBUS Application Protocol Specification V1.1b3 [Електронний ресурс]. – Modbus Organization, 2012. – URL: https://modbus.org/specs.php (дата звернення: 11.11.2026).
7. MODBUS over Serial Line Specification and Implementation Guide V1.02 [Електронний ресурс]. – Modbus-IDA, 2006. – URL: https://modbus.org/specs.php (дата звернення: 11.11.2026).
8. Universal Serial Bus Specification. Revision 2.0 [Електронний ресурс]. – USB Implementers Forum, 2000. – URL: https://www.usb.org/documents (дата звернення: 11.11.2026).
9. Universal Serial Bus 3.2 Specification [Електронний ресурс]. – USB Implementers Forum, 2017. – URL: https://www.usb.org/documents (дата звернення: 11.11.2026).
10. Universal Serial Bus Type-C Cable and Connector Specification. Release 2.1 [Електронний ресурс]. – USB Implementers Forum, 2021. – URL: https://www.usb.org/documents (дата звернення: 11.11.2026).
11. NXP Semiconductors. UM10204. I²C-bus specification and user manual. Rev. 7.0 [Електронний ресурс]. – 2021. – URL: https://www.nxp.com/docs/en/user-guide/UM10204.pdf (дата звернення: 11.11.2026).

12. BMP180. Digital pressure sensor : data sheet [Електронний ресурс]. – Bosch Sensortec, 2013. – URL: https://www.bosch-sensortec.com/ (дата звернення: 11.11.2026).
13. SSD1306. 128 × 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller : advance information. – Solomon Systech, 2008.
14. pySerial documentation [Електронний ресурс]. – URL: https://pyserial.readthedocs.io/ (дата звернення: 11.11.2026).
15. MicroPython documentation [Електронний ресурс]. – URL: https://docs.micropython.org/ (дата звернення: 11.11.2026).
16. Wokwi Documentation [Електронний ресурс]. – URL: https://docs.wokwi.com/ (дата звернення: 11.11.2026).
17. PulseView – sigrok [Електронний ресурс]. – URL: https://sigrok.org/wiki/PulseView (дата звернення: 11.11.2026).
18. Веб-сайт курсу «Периферійні пристрої, інтерфейси та драйвери» [Електронний ресурс]. – URL: https://dmytro-kushnir.github.io/#/peripheral-devices/ (дата звернення: 11.11.2026).
19. ISO 11898-1:2015. Road vehicles – Controller area network (CAN) – Part 1: Data link layer and physical signalling. – Geneva : International Organization for Standardization, 2015.
20. The Linux Kernel documentation. Driver implementer’s API guide [Електронний ресурс]. – URL: https://docs.kernel.org/driver-api/ (дата звернення: 11.11.2026).
21. Windows Driver Kit (WDK) documentation [Електронний ресурс] / Microsoft. – URL: https://learn.microsoft.com/windows-hardware/drivers/ (дата звернення: 11.11.2026).
22. Corbet J., Rubini A., Kroah-Hartman G. Linux Device Drivers. – 3rd ed. – Sebastopol : O’Reilly Media, 2005. – 640 p.
23. Universal Serial Bus Power Delivery Specification. Revision 3.1 [Електронний ресурс]. – USB Implementers Forum, 2021. – URL: https://www.usb.org/documents (дата звернення: 11.11.2026).

Допоміжна

24. Парамуд Я. С. Інтерфейси периферійних пристроїв ЕОМ : навч. посіб. / Я. С. Парамуд. – Київ : ІСДО, 1995. – 76 с.
25. Міюшкович Є. Г., Гребеняк А. В., Парамуд Я. С. Телекомунікаційні підсистеми кіберфізичних систем // Вісник Національного університету «Львівська політехніка». Серія: Комп’ютерні системи та мережі. – 2016. – № 857. – С. 65–73.

26. Міюшкович Є. Г., Парамуд Я. С. Особливості класифікації та застосування телекомунікаційних інтерфейсів у кіберфізичних системах // Вісник Національного університету «Львівська політехніка». Серія: Комп’ютерні системи та мережі. – 2015. – № 830. – С. 106–115.
27. Міюшкович Є. Г., Парамуд Я. С. Телекомунікаційні інтерфейси кіберфізичних систем: концепція застосування // Кіберфізичні системи: досягнення та виклики : матеріали І наук. семінару, 25–26 червня 2015 р., Львів. – Львів : Українські технології, 2015. – С. 28–37.

ДОДАТОК А. ВАРІАНТИ ЗАВДАНЬ ТА ЕЛЕКТРОННІ РЕСУРСИ КУРСУ

А.1. Варіанти завдань

Номер варіанту студенту призначає викладач. Параметри варіантів наведено в табл. А.1. Швидкість обміну використовують у практичних завданнях № 1, 2 і 5, формат кадру – у практичних завданнях № 2 і 5 (у практичному завданні № 1 формат кадру сталий – 8N1).

Таблиця А.1

Варіанти практичних завдань

№	Швидкість, біт/с	Формат	Мікросхема (ПЗ 4)	Інтервал, мс (ПЗ 5)	Імітаційний пристрій USB (ПЗ 3)
1	9600	8N1	давач BME280*	500	SanDisk Cruzer (Mass Storage)
2	9600	8N1	давач BME280*	500	Espressif USB JTAG/serial (CDC)
3	19200	8N1	давач BME280*	1000	Logitech USB Receiver
4	19200	8N1	OLED SSD1306	500	Intel Bluetooth (USB)
5	9600	7E1	давач BME280*	200	Linux Foundation 2.0 root hub
6	9600	8N1	OLED SSD1306	1000	SanDisk Cruzer (Mass Storage)
7	38400	8N1	давач BME280*	500	Espressif USB JTAG/serial (CDC)
8	38400	8N1	давач BME280*	200	Logitech USB Receiver
9	9600	8N2	OLED SSD1306	500	Intel Bluetooth (USB)
10	19200	8N1	давач BME280*	1000	Linux Foundation 2.0 root hub

* У симуляторі Wokwi для варіантів з давачем використовують модель давача BMP180 (адреса 0x77), на реальному обладнанні – BME280. Драйвер `bmp180.py` з мікросхемою BME280 не сумісний: вона має іншу карту регістрів і типову адресу 0x76, тому для неї потрібен окремий драйвер.

Позначення формату кадру: 8N1 – 8 бітів даних, без контролю парності, 1 стоповий біт; 7E1 – 7 бітів даних, контроль на парність (even), 1 стоповий біт; 8N2 – 8 бітів даних, без контролю парності, 2 стопові біти.

Повідомлення для практичних завдань № 1–4 – прізвище студента великими латинськими літерами A–Z без пробілів (наприклад, IVANOV); те

саме повідомлення використовують на стороні передавача і приймача. Пакет завершується символом повернення каретки `\r`. Якщо прізвище коротше за чотири символи, до нього без пробілу додають ім’я (за погодженням з викладачем).

А.2. Електронні ресурси курсу

Актуальну версію таблиці варіантів розміщено на веб-сайті курсу [18]:

`https://dmytro-kushnir.github.io/#/peripheral-devices/variants`

Тексти програмних драйверів та шаблони проєктів для виконання практичних завдань розміщено в електронному репозиторії курсу:

`https://github.com/dmytro-kushnir/ppid-labs`

Структура основних каталогів репозиторію:

- `host/` – програми хоста (практичні завдання № 1, 3, 5): модулі `uart_host.py`, `uart_device_emu.py`, `uart_pty_pair.py`, `usb_transaction.py`, `usb_scan.py`, `usb_gui.py`, `capstone_host.py`, `signal_gui.py`;
- `encoding/` – програми побудови діаграм сигналів і кодування (практичне завдання № 2): модулі `uart_plot.py`, `usb_nrzi.py`;
- `fixtures/` – навчальні дані (імітаційний перелік USB-пристроїв `usb_devices.json`);
- `wokwi/` – шаблони проєктів симулятора Wokwi (практичні завдання № 4, 5): проєкти lab04-i2c-sensor, lab04-i2c-oled, lab05-capstone та спільна бібліотека драйверів у каталозі lib.

Повні тексти програм наведено в додатку Б. Назви каталогів і програм репозиторію, а також заголовки вікон і повідомлень програм (наприклад, lab04-i2c-sensor, «PPID Lab 3 – Mock USB transfer») є ідентифікаторами програмного забезпечення курсу і відповідають практичним завданням № 1–5. Вказівки щодо встановлення інтерпретатора Python, бібліотек і запуску програм наведено в підрозділі А.3.

У звіті з кожного практичного завдання текст використаних програм наводять у додатку до звіту студента.

А.3. Підготовка програмного середовища

Для виконання практичних завдань № 1–3 і 5 потрібні інтерпретатор Python версії 3.11 або новішої та бібліотеки pySerial і matplotlib. Програми рекомендовано запускати у віртуальному середовищі Python (venv), створеному в кореневому каталозі репозиторію курсу: тоді бібліотеки встановлюються лише для проєкту, а інтерпретатор в усіх операційних системах викликається однаковою командою `python`.

Підготовка середовища в Linux і macOS:

```
$ git clone https://github.com/dmytro-kushnir/ppid-labs.git
$ cd ppid-labs
$ python3 -m venv .venv
$ source .venv/bin/activate
$ python -m pip install -r requirements.txt
```

Підготовка середовища у Windows (PowerShell):

```
> git clone https://github.com/dmytro-kushnir/ppid-labs.git
> cd ppid-labs
> py -m venv .venv
> .venv\Scripts\Activate.ps1
> python -m pip install -r requirements.txt
```

Якщо політика виконання сценаріїв PowerShell забороняє активацію, використовують командний рядок cmd і команду `.venv\Scripts\activate.bat`. Віртуальне середовище активують у кожному новому терміналі перед запуском програм; ознакою активованого середовища є префікс `(.venv)` у запрошенні командного рядка.

У прикладах команд символи $ і > позначають запрошення командного рядка і не вводяться. Усі команди в тексті посібника наведено для активованого віртуального середовища, тому інтерпретатор викликається командою `python`. Без віртуального середовища в Linux і macOS використовують команду `python3` (команда `python` може бути відсутньою або запускати застарілий Python 2), а у Windows – команду `py` або `python` залежно від способу встановлення інтерпретатора. Версію інтерпретатора перевіряють командою `python --version`. У коментарях до текстів програм (додаток Б) трапляється команда `python3` – її вибирають за тим самим правилом.

Імена послідовних портів залежать від операційної системи: у Windows – COM1, COM2 тощо (віртуальну пару портів створюють програмою com0com, наприклад COM5 ↔ COM6); у Linux USB-UART адаптер подається як /dev/ttyUSB0 або /dev/ttyACM0, у macOS – як `/dev/cu.usbserial-*`. Програма `host/uart_pty_pair.py` створює віртуальну пару портів лише в Linux і macOS.

А.4. Вимоги до оформлення результатів і критерії оцінювання

Результати виконання кожного практичного завдання студент оформлює у вигляді звіту, який містить:

1. Титульний аркуш із назвою та номером практичного завдання, номером варіанту, прізвищем та групою студента.
2. Мету завдання та параметри варіанту.
3. Короткі теоретичні відомості, необхідні для виконання завдання (зокрема формат кадру, структуру транзакції або схему під'єднання).
4. Схему алгоритму розробленої або дослідженої програми.
5. Хід виконання: команди, отримані результати (вивід програм, знімки екрана, діаграми, записи логічного аналізатора) та їх аналіз відповідно до кроків завдання.
6. Висновки.
7. Тексти використаних або розроблених програм (у додатку до звіту).

Під час захисту студент демонструє роботу програм для свого варіанту та відповідає на питання для самоперевірки відповідного розділу. Виконання практичного завдання оцінюють за рівнями, наведеними в табл. А.2; розподіл балів між практичними завданнями та контрольними заходами визначає робоча програма дисципліни.

Таблиця А.2

Критерії оцінювання практичних завдань

Рівень	Критерії
Достатній	виконано основні кроки завдання для свого варіанту, отримано очікувані результати, звіт містить усі обов'язкові складові
Добрий	додатково наведено аналіз результатів (розрахунки, зіставлення з теорією, пояснення розбіжностей); студент пояснює роботу програм і відповідає на більшість питань для самоперевірки
Високий	виконано завдання для отримання максимальної оцінки (там, де воно передбачене); студент вільно пояснює роботу всіх рівнів програмного забезпечення і відповідає на питання для самоперевірки

ДОДАТОК Б. ТЕКСТИ ПРОГРАМ

У додатку наведено тексти програм, які використовуються під час виконання практичних завдань. Програми хоста (лістинги Б.1–Б.10, Б.19) запускають з кореневого каталогу репозиторію курсу командою python -m <пакет>.<модуль> (див. підрозділ А.3). Файли проєктів симулятора Wokwi (лістинги Б.11–Б.18) створюють у середовищі Wokwi MicroPython ESP32 з тими самими іменами, що зазначені в заголовках лістингів. Під заголовком кожного лістингу наведено адресу того самого файлу в репозиторії курсу (https://github.com/dmytro-kushnir/ppid-labs), звідки текст програми зручно копіювати. Довгі рядки програм у лістингах перенесено; перенесені частини рядків позначено символом «↪».

Б.1. Програми до практичного завдання № 1

Лістинг Б.1. Програма передавача host/uart_host.py

Файл у репозиторії: `https://github.com/dmytro-kushnir/ppid-labs/blob/praktikum-2026/host/uart_host.py`

```
"""UART host utilities for PPID lab 1."""

from __future__ import annotations

import argparse
import sys
from typing import Optional

import serial

DEFAULT_BAUD = 9600
DEFAULT_ENCODING = "cp1251"


def open_port(
    name: str,
    baud: int = DEFAULT_BAUD,
    timeout: float = 1.0,
) -> serial.Serial:
    # Lab 1 keeps line format fixed at 8N1; parity/stop variants are explored in ↪
        lab 2.
    kwargs = {
        "baudrate": baud,
        "bytesize": serial.EIGHTBITS,
        "parity": serial.PARITY_NONE,
        "stopbits": serial.STOPBITS_ONE,
        "timeout": timeout,
    }
    # loop:// and other URL schemes need serial_for_url (Serial() treats them as ↪
        paths)
    try:
        if "://" in name:
            return serial.serial_for_url(name, **kwargs)
        return serial.Serial(port=name, **kwargs)
    except serial.SerialException as exc:
        hint = ""
        if name.startswith("/tmp/com") or name.startswith("/dev/tty"):
            hint = (
                "\nHint (Linux/macOS): start a virtual pair first:\n"
                "  python3 -m host.uart_pty_pair\n"
                "Or install socat and create /tmp/comA ↔ /tmp/comB (see ↪
                    docs/SETUP.md)."
            )
        raise serial.SerialException(f"{exc}{hint}") from None


def configure_port(ser: serial.Serial, baud: int = DEFAULT_BAUD) -> None:
    ser.baudrate = baud
    ser.bytesize = serial.EIGHTBITS
    ser.parity = serial.PARITY_NONE
    ser.stopbits = serial.STOPBITS_ONE


def encode_message(text: str, encoding: str = DEFAULT_ENCODING) -> bytes:
    return (text + "\r").encode(encoding)


def decode_message(data: bytes, encoding: str = DEFAULT_ENCODING) -> str:
    return data.decode(encoding, errors="replace").strip("\r\n")
```

```
def send_message(
    ser: serial.Serial,
    text: str,
    encoding: str = DEFAULT_ENCODING,
) -> int:
    payload = encode_message(text, encoding)
    return ser.write(payload)


def receive_message(
    ser: serial.Serial,
    encoding: str = DEFAULT_ENCODING,
    until: bytes = b"\r",
) -> str:
    data = ser.read_until(expected=until)
    return decode_message(data, encoding)


def exchange_loopback(
    ser: serial.Serial,
    text: str,
    encoding: str = DEFAULT_ENCODING,
) -> str:
    """Send and read echo on loop:// port."""
    send_message(ser, text, encoding)
    return receive_message(ser, encoding)


def format_hex(data: bytes) -> str:
    return " ".join(f"{b:02x}" for b in data)


def verify_loopback_reply(sent: str, reply: str) -> bool:
    return reply == sent


def verify_ack_reply(sent: str, reply: str) -> bool:
    return reply == f"ACK:{sent}"


def main(argv: Optional[list[str]] = None) -> int:
    parser = argparse.ArgumentParser(description="UART host send/receive")
    parser.add_argument(
        "--port",
        default="loop://",
        help="loop:// (all OS), COM5 (Windows), /dev/ttyUSB0 (Linux), /tmp/comA ↪
            (socat)",
    )
    parser.add_argument("--baud", type=int, default=DEFAULT_BAUD)
    parser.add_argument("--message", default="SAMPLE")
    parser.add_argument("--encoding", default=DEFAULT_ENCODING)
    parser.add_argument(
        "--wait-ack",
        action="store_true",
        help="Wait for ACK:… from peer (use with uart_device_emu on virtual COM)",
    )
    args = parser.parse_args(argv)

    with open_port(args.port, args.baud, timeout=2) as ser:
        configure_port(ser, args.baud)
        payload = encode_message(args.message, args.encoding)
        nbytes = send_message(ser, args.message, args.encoding)
        print(f"TX text: {args.message!r}")
        print(f"Надіслано байт: {nbytes}")
        print(f"TX hex: {format_hex(payload)}")
        if args.port.startswith("loop"):
            # Echo self-check of TX write — not the device RX role (use --wait-ack ↪
                + emu).
            reply = receive_message(ser, args.encoding)
            print(f"RX text (loopback): {reply!r}")
            if verify_loopback_reply(args.message, reply):
```

```
                print("Verify: OK")
            else:
                print(f"Verify: FAIL (очікувалось {args.message!r})")
        elif args.wait_ack:
            reply = receive_message(ser, args.encoding, until=b"\n")
            print(f"RX text: {reply!r}")
            expected = f"ACK:{args.message}"
            if verify_ack_reply(args.message, reply):
                print("Verify: OK")
            else:
                print(f"Verify: FAIL (очікувалось {expected!r})")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

Лістинг Б.2. Програма приймача (емулятор пристрою) host/uart_device_emu.py

Файл у репозиторії: https://github.com/dmytro-kushnir/ppid-labs/blob/praktikum-2026/host/uart_device_emu.py

```
"""Lab 1 UART device emulator (virtual COM pair) — compulsory RX role.

Listens on one end of a linked pair, reads a line ending with ``\\r``,
prints an exchange block (hex / ACK), and replies ``ACK:<message>\\n``.

Pair with ``host.uart_pty_pair`` (or socat / com0com) and
``host.uart_host --wait-ack``.

Example (Linux/macOS, no socat)::

    # terminal 0
    python3 -m host.uart_pty_pair

    # terminal 1 (device / RX)
    python3 -m host.uart_device_emu --port /tmp/comB

    # terminal 2 (host / TX)
    python3 -m host.uart_host --message "IVANOV" --port /tmp/comA --wait-ack
"""

from __future__ import annotations

import argparse
import sys
from typing import Optional

from host.uart_host import (
    DEFAULT_BAUD,
    DEFAULT_ENCODING,
    configure_port,
    decode_message,
    format_hex,
    open_port,
)


def build_ack(line: str, encoding: str = DEFAULT_ENCODING) -> bytes:
    return f"ACK:{line}\n".encode(encoding)


def handle_packet(packet: bytes, encoding: str = DEFAULT_ENCODING) -> tuple[str, ↪
    bytes]:
    """Decode RX packet (…\\r) and build ACK reply. Returns (line, ack_bytes)."""
    line = decode_message(packet, encoding)
    return line, build_ack(line, encoding)
```

```
def serve_once(ser, encoding: str = DEFAULT_ENCODING) -> bool:
    """Read one frame, print exchange, send ACK. Returns False on empty/timeout."""
    packet = ser.read_until(expected=b"\r")
    if not packet:
        return False

    line, ack = handle_packet(packet, encoding)
    written = ser.write(ack)

    print("--- exchange ---")
    print(f"RX text: {line}")
    print(f"RX bytes ({len(packet)}): {format_hex(packet)}")
    print(f"TX ACK: ACK:{line}")
    print(f"TX bytes ({written}): {format_hex(ack)}")
    if written == len(ack) and line:
        print("Verify: OK")
    else:
        print("Verify: FAIL")
    print("--------------")
    return True


def main(argv: Optional[list[str]] = None) -> int:
    parser = argparse.ArgumentParser(
        description="Lab 1 UART device emulator (compulsory RX on virtual COM)."
    )
    parser.add_argument(
        "--port",
        default="/tmp/comB",
        help="Peer port: /tmp/comB (uart_pty_pair), COM6 (com0com), …",
    )
    parser.add_argument("--baud", type=int, default=DEFAULT_BAUD)
    parser.add_argument("--encoding", default=DEFAULT_ENCODING)
    parser.add_argument(
        "--once",
        action="store_true",
        help="Exit after one successful exchange (default: keep listening)",
    )
    args = parser.parse_args(argv)

    print("PPID Lab 1 — UART device emulator (RX)")
    print(f"Port: {args.port} | Baud: {args.baud}")
    print("Waiting for Host TX…")

    with open_port(args.port, args.baud, timeout=1.0) as ser:
        configure_port(ser, args.baud)
        while True:
            if serve_once(ser, args.encoding):
                if args.once:
                    return 0
            # empty read = timeout; keep looping unless --once already returned

    return 0


if __name__ == "__main__":
    sys.exit(main())
```

Лістинг Б.3. Створення віртуальної пари портів host/uart_pty_pair.py

Файл у репозиторії: https://github.com/dmytro-kushnir/ppid-labs/blob/praktikum-2026/host/uart_pty_pair.py

```
"""Create a linked virtual COM pair without socat (Linux/macOS).

Bridges two PTYs and optionally symlinks them to ``/tmp/comA`` and ``/tmp/comB``.
Leave this process running while you use ``uart_host`` and ``uart_device_emu``.

Example::
```

```
    # terminal 0
    python3 -m host.uart_pty_pair

    # terminal 1
    python3 -m host.uart_device_emu --port /tmp/comB

    # terminal 2
    python3 -m host.uart_host --message "IVANOV" --port /tmp/comA --wait-ack

Windows: use com0com instead (this module needs ``pty``).
"""

from __future__ import annotations

import argparse
import os
import pty
import select
import sys
from pathlib import Path


def _symlink(link: Path, target: str) -> None:
    if link.exists() or link.is_symlink():
        link.unlink()
    link.symlink_to(target)


def bridge_pair(link_a: Path, link_b: Path) -> int:
    master_a, slave_a = pty.openpty()
    master_b, slave_b = pty.openpty()
    name_a = os.ttyname(slave_a)
    name_b = os.ttyname(slave_b)
    # Keep slave FDs open so the PTYs stay alive; clients open the same tty by name.

    try:
        _symlink(link_a, name_a)
        _symlink(link_b, name_b)
    except OSError as exc:
        print(f"Could not create symlinks {link_a} / {link_b}: {exc}", ↪
            file=sys.stderr)
        print(f"Use ports directly:\n  A: {name_a}\n  B: {name_b}", file=sys.stderr)
        link_a_s, link_b_s = name_a, name_b
    else:
        link_a_s, link_b_s = str(link_a), str(link_b)

    print("PPID Lab 1 — virtual COM pair (no socat)")
    print(f"  {link_a_s}  ↔  {link_b_s}")
    print(f"  ({name_a} ↔ {name_b})")
    print("Leave this running. Ctrl+C to stop.")
    print()
    print("Then:")
    print(f"  python3 -m host.uart_device_emu --port {link_b_s}")
    print(f'  python3 -m host.uart_host --message "IVANOV" --port {link_a_s} ↪
        --wait-ack')
    sys.stdout.flush()

    try:
        while True:
            readable, _, _ = select.select([master_a, master_b], [], [])
            for src, dst in ((master_a, master_b), (master_b, master_a)):
                if src not in readable:
                    continue
                try:
                    data = os.read(src, 4096)
                except OSError:
                    return 1
                if not data:
                    continue
                os.write(dst, data)
```

```
    except KeyboardInterrupt:
        print("\nPair stopped.")
    finally:
        for fd in (master_a, master_b, slave_a, slave_b):
            try:
                os.close(fd)
            except OSError:
                pass
        for link in (link_a, link_b):
            try:
                if link.is_symlink():
                    link.unlink()
            except OSError:
                pass
    return 0


def main(argv: list[str] | None = None) -> int:
    if os.name == "nt":
        print(
            "uart_pty_pair needs Linux/macOS. On Windows use com0com "
            "(see docs/SETUP.md).",
            file=sys.stderr,
        )
        return 1

    parser = argparse.ArgumentParser(
        description="Linked PTY pair for optional lab 1 PC exchange (no socat)"
    )
    parser.add_argument("--link-a", default="/tmp/comA", help="symlink for host TX")
    parser.add_argument("--link-b", default="/tmp/comB", help="symlink for device ↪
        emu")
    args = parser.parse_args(argv)
    return bridge_pair(Path(args.link_a), Path(args.link_b))


if __name__ == "__main__":
    sys.exit(main())
```

Б.2. Програми до практичного завдання № 2

Лістинг Б.4. Побудова амплітудно-часової діаграми UART encoding/uart_plot.py

Файл у репозиторії: `https://github.com/dmytro-kushnir/ppid-labs/blob/praktikum-2026/encoding/uart_plot.py`

```
"""UART frame timing diagrams for PPID lab 2."""

from __future__ import annotations

import argparse
from dataclasses import dataclass
from typing import List, Literal, Optional, Tuple

import matplotlib.pyplot as plt
from matplotlib.figure import Figure

ParityMode = Literal["N", "E", "O"]

# TTL logic levels: idle/mark (1) -> high, start/space (0) -> low.
# (RS-232 uses inverted voltage on the wire; see §Теорія. Data bits are drawn
# MSB-first for readability — the physical line order is LSB-first.)


@dataclass(frozen=True)
class LineFormat:
    data_bits: int
    parity: ParityMode
    stop_bits: int

    @property
    def label(self) -> str:
        return f"{self.data_bits}{self.parity}{self.stop_bits}"


def parse_line_format(fmt: str) -> LineFormat:
    """Parse UART line format strings such as 8N1, 7E1, 8N2."""
    fmt = fmt.strip().upper()
    if len(fmt) != 3:
        raise ValueError(f"Invalid line format: {fmt!r}")
    data_bits = int(fmt[0])
    parity = fmt[1]
    stop_bits = int(fmt[2])
    if data_bits not in (5, 6, 7, 8):
        raise ValueError(f"Unsupported data bits: {data_bits}")
    if parity not in ("N", "E", "O"):
        raise ValueError(f"Unsupported parity: {parity}")
    if stop_bits not in (1, 2):
        raise ValueError(f"Unsupported stop bits: {stop_bits}")
    return LineFormat(data_bits=data_bits, parity=parity, stop_bits=stop_bits)


def _parity_bit(data_bits: str, parity_mode: ParityMode) -> str:
    if parity_mode == "N":
        return ""
    ones = data_bits.count("1")
    if parity_mode == "E":
        return "0" if ones % 2 == 0 else "1"
    return "1" if ones % 2 == 0 else "0"


def char_to_frame_bits(
    ch: str,
    data_bits: int = 8,
    parity_mode: ParityMode = "N",
    stop_bits: int = 1,
```

```
) -> str:
    value = ord(ch)
    mask = (1 << data_bits) - 1
    data = format(value & mask, f"0{data_bits}b")
    parity = _parity_bit(data, parity_mode)
    stop = "1" * stop_bits
    return "0" + data + parity + stop


def char_to_frame_bits_from_format(ch: str, fmt: str | LineFormat = "8N1") -> str:
    line = parse_line_format(fmt) if isinstance(fmt, str) else fmt
    return char_to_frame_bits(ch, line.data_bits, line.parity, line.stop_bits)


def message_to_frames(message: str, fmt: str | LineFormat = "8N1") -> List[str]:
    line = parse_line_format(fmt) if isinstance(fmt, str) else fmt
    return [char_to_frame_bits(c, line.data_bits, line.parity, line.stop_bits) for ↪
        c in message]


def frames_to_signal(
    frames: List[str],
    bit_time: float = 1.0,
    gap_bits: int = 1,
) -> Tuple[List[float], List[int]]:
    times: List[float] = []
    levels: List[int] = []
    t = 0.0

    for frame in frames:
        for bit in frame:
            level = 1 if bit == "1" else 0
            times.extend([t, t + bit_time])
            levels.extend([level, level])
            t += bit_time
        times.extend([t, t + gap_bits * bit_time])
        levels.extend([levels[-1], levels[-1]])
        t += gap_bits * bit_time

    return times, levels


def bits_per_symbol(fmt: str | LineFormat = "8N1") -> int:
    line = parse_line_format(fmt) if isinstance(fmt, str) else fmt
    parity_count = 0 if line.parity == "N" else 1
    return 1 + line.data_bits + parity_count + line.stop_bits


def transmission_time(
    message: str,
    baud: int,
    fmt: str | LineFormat = "8N1",
    gap_bits: int = 1,
) -> float:
    bit_time = 1.0 / baud
    per_char = (bits_per_symbol(fmt) + gap_bits) * bit_time
    return len(message) * per_char


def _plot_signal(
    times: List[float],
    levels: List[int],
    title: str,
) -> Figure:
    fig, ax = plt.subplots(figsize=(12, 3))
    ax.step(times, levels, where="post")
    ax.set_ylim(-0.2, 1.2)
    ax.set_xlabel("Час, с")
    ax.set_ylabel("Рівень (лог.)")
    ax.set_title(title)
    ax.grid(True)
```

```
    fig.tight_layout()
    return fig


def plot_to_figure(
    message: str,
    baud: int = 9600,
    fmt: str = "8N1",
    char_index: Optional[int] = None,
    gap_bits: int = 1,
) -> Tuple[Figure, float]:
    line = parse_line_format(fmt)
    if char_index is not None:
        if not message:
            raise ValueError("Message is empty")
        if char_index < 0 or char_index >= len(message):
            raise IndexError(f"char_index out of range: {char_index}")
        plot_message = message[char_index]
        frames = message_to_frames(plot_message, line)
        title = (
            f"UART {line.label}: {plot_message!r} "
            f"(символ {char_index} з {message!r}) @ {baud} бод"
        )
        duration = transmission_time(plot_message, baud, line, gap_bits)
    else:
        frames = message_to_frames(message, line)
        duration = transmission_time(message, baud, line, gap_bits)
        title = f"UART {line.label}: {message!r} @ {baud} бод, T≈{duration:.4f} с"

    bit_time = 1.0 / baud
    times, levels = frames_to_signal(frames, bit_time=bit_time, gap_bits=gap_bits)
    fig = _plot_signal(times, levels, title)
    return fig, duration


def plot_message(
    message: str,
    baud: int = 9600,
    show: bool = True,
    fmt: str = "8N1",
    char_index: Optional[int] = None,
) -> float:
    fig, duration = plot_to_figure(message, baud, fmt, char_index)
    if show:
        plt.show()
    else:
        plt.close(fig)
    return duration


def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(description="Plot UART timing diagram")
    parser.add_argument("--message", default="IVANOV")
    parser.add_argument("--baud", type=int, default=9600)
    parser.add_argument("--format", default="8N1", dest="line_format")
    parser.add_argument("--char-index", type=int, default=None)
    parser.add_argument("--no-show", action="store_true")
    args = parser.parse_args(argv)

    duration = plot_message(
        args.message,
        args.baud,
        show=not args.no_show,
        fmt=args.line_format,
        char_index=args.char_index,
    )
    print(f"Розрахований час передачі: {duration:.6f} с")
    return 0


if __name__ == "__main__":
```

```
    raise SystemExit(main())
```

Лістинг Б.5. Кодування NRZI та bit stuffing encoding/usb_nrzi.py

Файл у репозиторії: `https://github.com/dmytro-kushnir/ppid-labs/blob/praktikum-2026/encoding/usb_nrzi.py`

```
"""USB NRZI encoding and bit stuffing for PPID lab 2."""

from __future__ import annotations

import argparse
from typing import List, Tuple

import matplotlib.pyplot as plt
from matplotlib.figure import Figure

SYNC_RAW = "01010100"  # KJKJKJKK before NRZI (educational simplification)


def char_to_bits(ch: str) -> str:
    return format(ord(ch) & 0xFF, "08b")


def message_to_bits(message: str) -> str:
    return "".join(char_to_bits(c) for c in message)


def bit_stuff(bits: str) -> str:
    """Insert 0 after six consecutive 1 bits."""
    out: List[str] = []
    ones = 0
    for b in bits:
        out.append(b)
        if b == "1":
            ones += 1
            if ones == 6:
                out.append("0")
                ones = 0
        else:
            ones = 0
    return "".join(out)


def nrzi_encode(bits: str, initial_level: int = 1) -> List[int]:
    levels = [initial_level]
    for b in bits:
        if b == "0":
            levels.append(1 - levels[-1])
        else:
            levels.append(levels[-1])
    return levels


def nrzi_bits_string(bits: str, initial_level: int = 1) -> str:
    levels = nrzi_encode(bits, initial_level)
    return "".join(str(level) for level in levels[1:])


def nrzi_encode_with_stuffing(message: str) -> tuple[str, str, List[int]]:
    raw = message_to_bits(message)
    stuffed = bit_stuff(raw)
    levels = nrzi_encode(stuffed)
    return raw, stuffed, levels


def char_nrzi_bits(ch: str) -> tuple[str, str, List[int]]:
    """Encode a single character: raw 8 bits -> bit stuffing -> NRZI levels."""
    raw = char_to_bits(ch)
    stuffed = bit_stuff(raw)
```

```
    levels = nrzi_encode(stuffed)
    return raw, stuffed, levels


def _plot_levels(levels: List[int], title: str) -> Figure:
    times = list(range(len(levels)))
    fig, ax = plt.subplots(figsize=(12, 3))
    ax.step(times, levels, where="post")
    ax.set_ylim(-0.2, 1.2)
    ax.set_xlabel("Біт (індекс)")
    ax.set_ylabel("NRZI рівень")
    ax.set_title(title)
    ax.grid(True)
    fig.tight_layout()
    return fig


def plot_nrzi_to_figure(message: str) -> Tuple[Figure, str, str, List[int]]:
    raw, stuffed, levels = nrzi_encode_with_stuffing(message)
    title = f"USB NRZI: {message!r} (raw {len(raw)} біт, stuffed {len(stuffed)})"
    fig = _plot_levels(levels, title)
    return fig, raw, stuffed, levels


def plot_nrzi_char_to_figure(message: str, index: int) -> Tuple[Figure, str, str, ↪
    List[int]]:
    if not message:
        raise ValueError("Message is empty")
    if index < 0 or index >= len(message):
        raise IndexError(f"index out of range: {index}")
    ch = message[index]
    raw, stuffed, levels = char_nrzi_bits(ch)
    title = f"USB NRZI: {ch!r} (символ {index} з {message!r})"
    fig = _plot_levels(levels, title)
    return fig, raw, stuffed, levels


def plot_nrzi(message: str, show: bool = True) -> None:
    fig, _, _, _ = plot_nrzi_to_figure(message)
    if show:
        plt.show()
    else:
        plt.close(fig)


def plot_nrzi_char(message: str, index: int, show: bool = True) -> None:
    fig, _, _, _ = plot_nrzi_char_to_figure(message, index)
    if show:
        plt.show()
    else:
        plt.close(fig)


def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(description="USB NRZI plot")
    parser.add_argument("--message", default="AB")
    parser.add_argument("--char-index", type=int, default=None)
    parser.add_argument("--no-show", action="store_true")
    args = parser.parse_args(argv)

    if args.char_index is not None:
        raw, stuffed, levels = char_nrzi_bits(args.message[args.char_index])
        print("Символ:", args.message[args.char_index])
    else:
        raw, stuffed, levels = nrzi_encode_with_stuffing(args.message)
    print("Біти (raw):", raw)
    print("Після bit stuffing:", stuffed)
    print("NRZI рівні:", levels)

    if args.char_index is not None:
        plot_nrzi_char(args.message, args.char_index, show=not args.no_show)
```

```
    else:
        plot_nrzi(args.message, show=not args.no_show)
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Лістинг Б.6. Графічний інтерфейс побудови діаграм host/signal_gui.py

Файл у репозиторії: `https://github.com/dmytro-kushnir/ppid-labs/blob/praktikum-2026/host/signal_gui.py`

```
"""Reference tkinter GUI for UART and NRZI signal visualization (PPID labs 1–2)."""

from __future__ import annotations

import tkinter as tk
from tkinter import filedialog, messagebox, ttk
from typing import Optional

import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
from matplotlib.figure import Figure

from encoding.uart_plot import (
    char_to_frame_bits_from_format,
    message_to_frames,
    plot_to_figure,
    transmission_time,
)
from encoding.usb_nrzi import (
    char_nrzi_bits,
    nrzi_bits_string,
    nrzi_encode_with_stuffing,
    plot_nrzi_char_to_figure,
    plot_nrzi_to_figure,
)
from host.uart_host import open_port, send_message

BAUD_VALUES = (50, 75, 100, 150, 300, 600, 1200, 2400, 4800, 9600, 19200, 38400)
FORMAT_VALUES = ("8N1", "7E1", "8N2")
ALL_CHARS_LABEL = "Усе повідомлення"


class SignalGuiApp:
    def __init__(self, root: tk.Tk) -> None:
        self.root = root
        self.root.title("PPID Lab 1–2 — Signal visualization (reference)")
        self._canvas: Optional[FigureCanvasTkAgg] = None
        self._current_figure: Optional[Figure] = None
        self._build()

    def _build(self) -> None:
        notebook = ttk.Notebook(self.root)
        notebook.pack(fill="both", expand=True, padx=8, pady=8)

        self.uart_tab = ttk.Frame(notebook, padding=8)
        self.nrzi_tab = ttk.Frame(notebook, padding=8)
        notebook.add(self.uart_tab, text="UART")
        notebook.add(self.nrzi_tab, text="NRZI")

        self._build_uart_tab()
        self._build_nrzi_tab()
        self._build_plot_area()

    def _build_plot_area(self) -> None:
        plot_frame = ttk.LabelFrame(self.root, text="Графік", padding=8)
        plot_frame.pack(fill="both", expand=True, padx=8, pady=(0, 8))
        self.plot_container = ttk.Frame(plot_frame)
```

```
        self.plot_container.pack(fill="both", expand=True)

    def _set_figure(self, fig: Figure) -> None:
        if self._canvas is not None:
            self._canvas.get_tk_widget().destroy()
        if self._current_figure is not None:
            plt.close(self._current_figure)
        self._current_figure = fig
        self._canvas = FigureCanvasTkAgg(fig, master=self.plot_container)
        self._canvas.draw()
        self._canvas.get_tk_widget().pack(fill="both", expand=True)

    def _save_figure_png(self) -> None:
        if self._current_figure is None:
            messagebox.showwarning("Увага", "Спочатку побудуйте графік")
            return
        path = filedialog.asksaveasfilename(
            defaultextension=".png",
            filetypes=[("PNG", "*.png")],
        )
        if path:
            self._current_figure.savefig(path, dpi=150, bbox_inches="tight")
            messagebox.showinfo("Готово", f"Збережено: {path}")

    def _message(self, entry: tk.Entry) -> str:
        return entry.get().strip()

    def _char_index_from_combo(self, message: str, combo: ttk.Combobox) -> ↪
        Optional[int]:
        value = combo.get()
        if value == ALL_CHARS_LABEL:
            return None
        return int(value)

    def _refresh_char_combo(self, message: str, combo: ttk.Combobox) -> None:
        values = [ALL_CHARS_LABEL] + [str(i) for i in range(len(message))]
        combo["values"] = values
        if combo.get() not in values:
            combo.set(ALL_CHARS_LABEL)

    def _build_uart_tab(self) -> None:
        frame = self.uart_tab

        ttk.Label(frame, text="Повідомлення:").grid(row=0, column=0, sticky="w")
        self.uart_message = tk.Entry(frame, width=40)
        self.uart_message.insert(0, "IVANOV")
        self.uart_message.grid(row=0, column=1, columnspan=2, sticky="ew", pady=2)
        self.uart_message.bind("<KeyRelease>", lambda _e: ↪
            self._on_uart_message_change())

        ttk.Label(frame, text="Baud:").grid(row=1, column=0, sticky="w")
        self.uart_baud = ttk.Combobox(frame, values=[str(b) for b in BAUD_VALUES], ↪
            width=10)
        self.uart_baud.set("9600")
        self.uart_baud.grid(row=1, column=1, sticky="w", pady=2)

        ttk.Label(frame, text="Формат:").grid(row=2, column=0, sticky="w")
        self.uart_format = ttk.Combobox(frame, values=list(FORMAT_VALUES), width=10)
        self.uart_format.set("8N1")
        self.uart_format.grid(row=2, column=1, sticky="w", pady=2)

        ttk.Label(frame, text="Символ:").grid(row=3, column=0, sticky="w")
        self.uart_char = ttk.Combobox(frame, width=20)
        self.uart_char.grid(row=3, column=1, sticky="w", pady=2)
        self.uart_char.bind("<<ComboboxSelected>>", lambda _e: ↪
            self._update_uart_fields())
        self._refresh_char_combo(self._message(self.uart_message), self.uart_char)
        self.uart_char.set(ALL_CHARS_LABEL)

        ttk.Label(frame, text="Кадр (біти):").grid(row=4, column=0, sticky="nw")
        self.uart_frame_bits = tk.Text(frame, width=50, height=2, state="disabled")
```

```
        self.uart_frame_bits.grid(row=4, column=1, columnspan=2, sticky="ew", pady=2)

        ttk.Label(frame, text="Hex:").grid(row=5, column=0, sticky="w")
        self.uart_hex = tk.Entry(frame, width=20, state="readonly")
        self.uart_hex.grid(row=5, column=1, sticky="w", pady=2)

        ttk.Label(frame, text="Час, с:").grid(row=6, column=0, sticky="w")
        self.uart_time = tk.Entry(frame, width=20, state="readonly")
        self.uart_time.grid(row=6, column=1, sticky="w", pady=2)

        btn_row = ttk.Frame(frame)
        btn_row.grid(row=7, column=0, columnspan=3, pady=8, sticky="w")
        ttk.Button(btn_row, text="Графік UART", ↪
            command=self._plot_uart).pack(side="left", padx=(0, 4))
        ttk.Button(btn_row, text="Зберегти PNG", ↪
            command=self._save_figure_png).pack(side="left", padx=4)
        ttk.Button(btn_row, text="Надіслати (loop://)", ↪
            command=self._send_loopback).pack(side="left", padx=4)

        frame.columnconfigure(1, weight=1)
        self._update_uart_fields()

    def _build_nrzi_tab(self) -> None:
        frame = self.nrzi_tab

        ttk.Label(frame, text="Повідомлення:").grid(row=0, column=0, sticky="w")
        self.nrzi_message = tk.Entry(frame, width=40)
        self.nrzi_message.insert(0, "IVANOV")
        self.nrzi_message.grid(row=0, column=1, columnspan=2, sticky="ew", pady=2)
        self.nrzi_message.bind("<KeyRelease>", lambda _e: ↪
            self._on_nrzi_message_change())

        ttk.Label(frame, text="Символ:").grid(row=1, column=0, sticky="w")
        self.nrzi_char = ttk.Combobox(frame, width=20)
        self.nrzi_char.grid(row=1, column=1, sticky="w", pady=2)
        self.nrzi_char.bind("<<ComboboxSelected>>", lambda _e: ↪
            self._update_nrzi_fields())
        self._refresh_char_combo(self._message(self.nrzi_message), self.nrzi_char)
        if self._message(self.nrzi_message):
            self.nrzi_char.set("0")
        else:
            self.nrzi_char.set(ALL_CHARS_LABEL)

        for row, label, attr in (
            (2, "Raw (8 біт):", "nrzi_raw"),
            (3, "Після bit stuffing:", "nrzi_stuffed"),
            (4, "NRZI (рядок):", "nrzi_levels"),
        ):
            ttk.Label(frame, text=label).grid(row=row, column=0, sticky="nw")
            widget = tk.Text(frame, width=50, height=2, state="disabled")
            widget.grid(row=row, column=1, columnspan=2, sticky="ew", pady=2)
            setattr(self, attr, widget)

        btn_row = ttk.Frame(frame)
        btn_row.grid(row=5, column=0, columnspan=3, pady=8, sticky="w")
        ttk.Button(btn_row, text="Графік NRZI (символ)", ↪
            command=self._plot_nrzi_char).pack(
            side="left", padx=(0, 4)
        )
        ttk.Button(btn_row, text="Графік NRZI (усі)", ↪
            command=self._plot_nrzi_all).pack(
            side="left", padx=4
        )
        ttk.Button(btn_row, text="Зберегти PNG", ↪
            command=self._save_figure_png).pack(side="left", padx=4)

        frame.columnconfigure(1, weight=1)
        self._update_nrzi_fields()

    def _set_text(self, widget: tk.Text, value: str) -> None:
```

```
        widget.configure(state="normal")
        widget.delete("1.0", tk.END)
        widget.insert(tk.END, value)
        widget.configure(state="disabled")

    def _set_readonly_entry(self, widget: tk.Entry, value: str) -> None:
        widget.configure(state="normal")
        widget.delete(0, tk.END)
        widget.insert(0, value)
        widget.configure(state="readonly")

    def _on_uart_message_change(self) -> None:
        message = self._message(self.uart_message)
        self._refresh_char_combo(message, self.uart_char)
        self._update_uart_fields()

    def _on_nrzi_message_change(self) -> None:
        message = self._message(self.nrzi_message)
        self._refresh_char_combo(message, self.nrzi_char)
        if message and self.nrzi_char.get() == ALL_CHARS_LABEL:
            self.nrzi_char.set("0")
        self._update_nrzi_fields()

    def _update_uart_fields(self) -> None:
        message = self._message(self.uart_message)
        fmt = self.uart_format.get()
        if not message:
            self._set_text(self.uart_frame_bits, "")
            self._set_readonly_entry(self.uart_hex, "")
            self._set_readonly_entry(self.uart_time, "")
            return

        idx = self._char_index_from_combo(message, self.uart_char)
        if idx is None:
            frames = message_to_frames(message, fmt)
            frame_bits = " | ".join(frames)
            hex_vals = " ".join(f"0x{ord(c):02X}" for c in message)
        else:
            target = message[idx]
            frame_bits = char_to_frame_bits_from_format(target, fmt)
            hex_vals = f"0x{ord(target):02X}"
        self._set_text(self.uart_frame_bits, frame_bits)
        self._set_readonly_entry(self.uart_hex, hex_vals)

        baud = int(self.uart_baud.get())
        duration = transmission_time(message, baud, fmt)
        self._set_readonly_entry(self.uart_time, f"{duration:.6f}")

    def _update_nrzi_fields(self) -> None:
        message = self._message(self.nrzi_message)
        if not message:
            for widget in (self.nrzi_raw, self.nrzi_stuffed, self.nrzi_levels):
                self._set_text(widget, "")
            return

        value = self.nrzi_char.get()
        if value == ALL_CHARS_LABEL:
            raw, stuffed, _levels = nrzi_encode_with_stuffing(message)
            nrzi_str = nrzi_bits_string(stuffed)
        else:
            raw, stuffed, _levels = char_nrzi_bits(message[int(value)])
            nrzi_str = nrzi_bits_string(stuffed)

        self._set_text(self.nrzi_raw, raw)
        self._set_text(self.nrzi_stuffed, stuffed)
        self._set_text(self.nrzi_levels, nrzi_str)

    def _plot_uart(self) -> None:
        message = self._message(self.uart_message)
        if not message:
            messagebox.showwarning("Увага", "Введіть повідомлення")
```

```
            return
        try:
            baud = int(self.uart_baud.get())
            fmt = self.uart_format.get()
            idx = self._char_index_from_combo(message, self.uart_char)
            fig, duration = plot_to_figure(message, baud, fmt, idx)
            self._set_figure(fig)
            self._set_readonly_entry(self.uart_time, f"{duration:.6f}")
        except (ValueError, IndexError) as exc:
            messagebox.showerror("Помилка", str(exc))

    def _plot_nrzi_char(self) -> None:
        message = self._message(self.nrzi_message)
        if not message:
            messagebox.showwarning("Увага", "Введіть повідомлення")
            return
        value = self.nrzi_char.get()
        if value == ALL_CHARS_LABEL:
            messagebox.showinfo("Підказка", "Оберіть індекс символу (0, 1, …) для ↪
                посимвольного графіка")
            return
        try:
            fig, _, _, _ = plot_nrzi_char_to_figure(message, int(value))
            self._set_figure(fig)
            self._update_nrzi_fields()
        except (ValueError, IndexError) as exc:
            messagebox.showerror("Помилка", str(exc))

    def _plot_nrzi_all(self) -> None:
        message = self._message(self.nrzi_message)
        if not message:
            messagebox.showwarning("Увага", "Введіть повідомлення")
            return
        fig, _, _, _ = plot_nrzi_to_figure(message)
        self._set_figure(fig)

    def _send_loopback(self) -> None:
        message = self._message(self.uart_message)
        if not message:
            messagebox.showwarning("Увага", "Введіть повідомлення")
            return
        try:
            baud = int(self.uart_baud.get())
            with open_port("loop://", baud, timeout=2) as ser:
                nbytes = send_message(ser, message)
            messagebox.showinfo("Готово", f"Надіслано {nbytes} байт на loop:// @ ↪
                {baud} бод")
        except Exception as exc:
            messagebox.showerror("Помилка", str(exc))


def main() -> None:
    root = tk.Tk()
    SignalGuiApp(root)
    root.mainloop()


if __name__ == "__main__":
    main()
```

Б.3. Програми до практичного завдання № 3

Лістинг Б.7. Модель транзакції USB host/usb_transaction.py

Файл у репозиторії: `https://github.com/dmytro-kushnir/ppid-labs/blob/praktikum-2026/host/usb_transaction.py`

```
"""USB 2.0 transaction model for PPID lab 3."""

from __future__ import annotations

import argparse
import struct
from dataclasses import dataclass
from typing import Iterable, List

# USB 2.0 PID values (simplified educational subset)
PID_OUT = 0xE1
PID_IN = 0x69
PID_DATA0 = 0xC3
PID_DATA1 = 0x4B
PID_ACK = 0xD2
PID_NAK = 0x5A


@dataclass
class TokenPacket:
    pid: int
    addr: int
    endp: int

    def to_bytes(self) -> bytes:
        # Packed 11-bit ADDR (7) + ENDP (4) field. A real USB token packet also
        # appends a 5-bit CRC5 over this field; omitted here as an educational
        # simplification (see §Теорія USB).
        addr_endp = ((self.addr & 0x7F) | ((self.endp & 0x0F) << 7)) & 0xFFFF
        return bytes([self.pid, addr_endp & 0xFF, (addr_endp >> 8) & 0xFF])


@dataclass
class DataPacket:
    pid: int
    payload: bytes

    def to_bytes(self) -> bytes:
        if len(self.payload) > 64:
            raise ValueError("Full-speed max data payload is 64 bytes")
        frame = bytes([self.pid]) + self.payload
        crc = _crc16(frame[1:])
        return frame + struct.pack("<H", crc)


@dataclass
class HandshakePacket:
    pid: int

    def to_bytes(self) -> bytes:
        return bytes([self.pid])


def _crc16(data: bytes) -> int:
    """Simplified educational CRC-16 (0x8005, reflected), used as the DATA
    packet checksum. NOTE: a real USB CRC-16 also inverts the final remainder
    (``^ 0xFFFF``); that step is intentionally omitted here so the produced
    bytes stay stable for the lab example — this is not a wire-accurate USB CRC.
    """
    crc = 0xFFFF
    for byte in data:
        crc ^= byte
```

```
        for _ in range(8):
            if crc & 1:
                crc = (crc >> 1) ^ 0xA001
            else:
                crc >>= 1
    return crc & 0xFFFF


def build_out_transaction(message: str, encoding: str = "cp1251") -> List[bytes]:
    payload = message.encode(encoding)
    if len(payload) < 8:
        payload = payload + b"\x00" * (8 - len(payload))
    return [
        TokenPacket(PID_OUT, addr=1, endp=1).to_bytes(),
        DataPacket(PID_DATA0, payload).to_bytes(),
        HandshakePacket(PID_ACK).to_bytes(),
    ]


def transaction_phases(packets: Iterable[bytes]) -> List[str]:
    phases = []
    for pkt in packets:
        pid = pkt[0]
        if pid in (PID_OUT, PID_IN):
            phases.append("TOKEN")
        elif pid in (PID_DATA0, PID_DATA1):
            phases.append("DATA")
        else:
            phases.append("HANDSHAKE")
    return phases


def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(description="Build USB OUT transaction")
    parser.add_argument("--message", default="IVANOV")
    parser.add_argument("--encoding", default="cp1251")
    args = parser.parse_args(argv)

    packets = build_out_transaction(args.message, args.encoding)
    phases = transaction_phases(packets)
    print("Фази:", " → ".join(phases))
    for i, pkt in enumerate(packets, 1):
        print(f"Пакет {i}: {pkt.hex(' ')}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Лістинг Б.8. Сканування mock-переліку USB-пристроїв host/usb_scan.py

Файл у репозиторії: https://github.com/dmytro-kushnir/ppid-labs/blob/praktikum-2026/host/usb_scan.py

```
"""Mock USB device enumeration for PPID lab 3."""

from __future__ import annotations

import argparse
import json
from pathlib import Path
from typing import Any, Optional

FIXTURES_DIR = Path(__file__).resolve().parent.parent / "fixtures"
DEFAULT_FIXTURE = FIXTURES_DIR / "usb_devices.json"


def load_devices(path: Path = DEFAULT_FIXTURE) -> list[dict[str, Any]]:
    with path.open(encoding="utf-8") as f:
        data = json.load(f)
```

```
    return data["devices"]


def get_device_by_name(name: str, path: Path = DEFAULT_FIXTURE) -> ↪
    Optional[dict[str, Any]]:
    for dev in load_devices(path):
        if dev["name"] == name:
            return dev
    return None


def format_device(dev: dict[str, Any]) -> str:
    return (
        f"{dev['vid']}:{dev['pid']} "
        f"{dev['class']} @ {dev['speed']} — {dev['name']}"
    )


def _format_optional(value: Any, suffix: str = "") -> str:
    if value is None:
        return "—"
    if isinstance(value, float):
        return f"{value:g}{suffix}"
    return f"{value}{suffix}"


def format_device_details(dev: dict[str, Any]) -> str:
    lines = [
        f"VID:PID: {dev['vid']}:{dev['pid']}",
        f"Клас: {dev['class']}",
        f"Швидкість: {dev['speed']}",
        f"Назва: {dev['name']}",
        f"Мітка тому: {_format_optional(dev.get('volume_label'))}",
        f"Файлова система: {_format_optional(dev.get('drive_format'))}",
        f"Обсяг: {_format_optional(dev.get('total_gb'), ' ГБ')}",
        f"Вільно: {_format_optional(dev.get('free_gb'), ' ГБ')}",
    ]
    return "\n".join(lines)


def list_usb_devices(path: Path = DEFAULT_FIXTURE) -> list[str]:
    return [format_device(d) for d in load_devices(path)]


def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(description="List mock USB devices")
    parser.add_argument(
        "--fixture",
        type=Path,
        default=DEFAULT_FIXTURE,
        help="Path to usb_devices.json",
    )
    args = parser.parse_args(argv)

    print("Mock USB enumeration (fixtures/usb_devices.json):")
    print("У продакшені: lsusb (Linux) або Get-PnpDevice (Windows)\n")
    for line in list_usb_devices(args.fixture):
        print(line)
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Лістинг Б.9. Mock-перелік USB-пристроїв fixtures/usb_devices.json

Файл у репозиторії: `https://github.com/dmytro-kushnir/ppid-labs/blob/praktikum-2026/fixtures/usb_devices.json`

```
{
  "devices": [
    {
      "vid": "046d",
      "pid": "c52b",
      "name": "Logitech USB Receiver",
      "class": "HID",
      "speed": "12 Mbps (Full Speed)",
      "volume_label": null,
      "drive_format": null,
      "total_gb": null,
      "free_gb": null
    },
    {
      "vid": "0781",
      "pid": "5567",
      "name": "SanDisk Cruzer (Mass Storage)",
      "class": "Mass Storage",
      "speed": "480 Mbps (High Speed)",
      "volume_label": "PPID_USB",
      "drive_format": "FAT32",
      "total_gb": 16.0,
      "free_gb": 15.2
    },
    {
      "vid": "303a",
      "pid": "1001",
      "name": "Espressif USB JTAG/serial (CDC)",
      "class": "CDC",
      "speed": "12 Mbps (Full Speed)",
      "volume_label": null,
      "drive_format": null,
      "total_gb": null,
      "free_gb": null
    },
    {
      "vid": "1d6b",
      "pid": "0002",
      "name": "Linux Foundation 2.0 root hub",
      "class": "Hub",
      "speed": "480 Mbps (High Speed)",
      "volume_label": null,
      "drive_format": null,
      "total_gb": null,
      "free_gb": null
    },
    {
      "vid": "8087",
      "pid": "0026",
      "name": "Intel Bluetooth (USB)",
      "class": "Wireless",
      "speed": "12 Mbps (Full Speed)",
      "volume_label": null,
      "drive_format": null,
      "total_gb": null,
      "free_gb": null
    }
  ]
}
```

Лістинг Б.10. Діалоговий драйвер передавача host/usb_gui.py

Файл у репозиторії: `https://github.com/dmytro-kushnir/ppid-labs/blob/praktikum-2026/host/usb_gui.py`

```
"""Tkinter GUI for mock USB mass-storage transfer (PPID lab 3)."""

from __future__ import annotations

import tempfile
import tkinter as tk
from pathlib import Path
from tkinter import filedialog, messagebox, ttk
from typing import Any, Optional

from host.usb_scan import format_device_details, get_device_by_name, load_devices


class UsbGuiApp:
    def __init__(self, root: tk.Tk) -> None:
        self.root = root
        self.root.title("PPID Lab 3 — Mock USB transfer")
        self.devices: list[dict[str, Any]] = []
        self.storage_dir = Path(tempfile.mkdtemp(prefix="ppid_usb_"))
        self._build()
        self._scan_devices()

    def _build(self) -> None:
        frame = ttk.Frame(self.root, padding=10)
        frame.grid(row=0, column=0, sticky="nsew")
        self.root.columnconfigure(0, weight=1)
        self.root.rowconfigure(0, weight=1)

        top_row = ttk.Frame(frame)
        top_row.grid(row=0, column=0, columnspan=3, sticky="ew", pady=(0, 4))
        ttk.Button(top_row, text="Сканувати", ↪
            command=self._scan_devices).pack(side="left")

        ttk.Label(frame, text="Mock USB-пристрій:").grid(row=1, column=0, sticky="w")
        self.device_var = tk.StringVar()
        self.device_combo = ttk.Combobox(
            frame, textvariable=self.device_var, values=[], width=40, state="readonly"
        )
        self.device_combo.grid(row=1, column=1, columnspan=2, sticky="ew", pady=4)
        self.device_combo.bind("<<ComboboxSelected>>", lambda _e: ↪
            self._show_device_details())

        ttk.Label(frame, text="Властивості:").grid(row=2, column=0, sticky="nw")
        self.details = tk.Text(frame, width=50, height=8, state="disabled")
        self.details.grid(row=2, column=1, columnspan=2, sticky="ew", pady=4)

        ttk.Button(frame, text="Відкрити файл", command=self._open_file).grid(
            row=3, column=0, pady=4, sticky="w"
        )
        ttk.Label(
            frame,
            text="Повідомлення (прізвище латиницею, літери A-Z) — введіть або ↪
                відкрити .txt",
            wraplength=420,
        ).grid(row=3, column=1, columnspan=2, sticky="w", padx=(8, 0))
        self.text = tk.Text(frame, width=50, height=10)
        self.text.grid(row=4, column=0, columnspan=3, pady=4, sticky="nsew")

        ttk.Button(frame, text="Записати на mock-накопичувач", ↪
            command=self._save).grid(
            row=5, column=0, columnspan=3, pady=4
        )

        ttk.Label(
            frame,
```

```
            text=f"Каталог накопичувача: {self.storage_dir}",
            wraplength=420,
        ).grid(row=6, column=0, columnspan=3, sticky="w")

        frame.columnconfigure(1, weight=1)
        frame.rowconfigure(4, weight=1)

    def _selected_device(self) -> Optional[dict[str, Any]]:
        name = self.device_var.get()
        if not name:
            return None
        return get_device_by_name(name) or next(
            (d for d in self.devices if d["name"] == name), None
        )

    def _set_details(self, text: str) -> None:
        self.details.configure(state="normal")
        self.details.delete("1.0", tk.END)
        self.details.insert(tk.END, text)
        self.details.configure(state="disabled")

    def _scan_devices(self) -> None:
        self.devices = load_devices()
        names = [d["name"] for d in self.devices]
        self.device_combo["values"] = names
        if names:
            current = self.device_var.get()
            self.device_var.set(current if current in names else names[0])
            self._show_device_details()
        else:
            self.device_var.set("")
            self._set_details("Пристрої не знайдено")

    def _show_device_details(self) -> None:
        dev = self._selected_device()
        if dev is None:
            self._set_details("")
            return
        self._set_details(format_device_details(dev))

    def _open_file(self) -> None:
        path = filedialog.askopenfilename(
            title="Відкрити текстовий файл",
            filetypes=[
                ("Текстові файли", "*.txt"),
                ("Markdown", "*.md"),
                ("Усі файли", "*.*"),
            ],
        )
        if not path:
            return
        raw = Path(path).read_bytes()
        if b"\x00" in raw[:8192]:
            messagebox.showerror(
                "Помилка",
                "Обрано бінарний файл (зображення, PDF тощо).\n"
                "Для лаб. 3 — текстовий .txt із прізвищем латиницею (напр. IVANOV).",
            )
            return
        content: str | None = None
        for encoding in ("utf-8", "cp1251"):
            try:
                content = raw.decode(encoding)
                break
            except UnicodeDecodeError:
                continue
        if content is None:
            messagebox.showerror(
                "Помилка",
                "Не вдалося прочитати файл як текст (UTF-8 або cp1251).",
            )
```

```
            return
        self.text.delete("1.0", tk.END)
        self.text.insert(tk.END, content)

    def _save(self) -> None:
        content = self.text.get("1.0", tk.END).strip()
        if not content:
            messagebox.showwarning("Увага", "Немає даних для запису")
            return
        dev = self._selected_device()
        label = dev["name"] if dev else "unknown"
        target = self.storage_dir / "message.txt"
        try:
            target.write_text(content, encoding="cp1251")
        except UnicodeEncodeError:
            messagebox.showerror(
                "Помилка",
                "Текст містить символи, яких немає в cp1251.\n"
                "Записуйте прізвище латиницею (літери A-Z), не PDF чи зображення.",
            )
            return
        messagebox.showinfo("Готово", f"Записано на {label}:\n{target}")


def main() -> None:
    root = tk.Tk()
    UsbGuiApp(root)
    root.mainloop()


if __name__ == "__main__":
    main()
```

Б.4. Програми до практичного завдання № 4

Лістинг Б.11. Драйвер давача BMP180 bmp180.py (практичні завдання № 4, 5)

Файл у репозиторії: `https://github.com/dmytro-kushnir/ppid-labs/blob/praktikum-2026/wokwi/lib/bmp180.py`

```
# BMP180 driver for MicroPython (based on robert-hh/BMP085_BMP180, MIT)
# Wokwi: board-bmp180 (I2C 0x77). Real hardware course variants: BME280.

from ustruct import unpack as unp
import time


class BMP180:
    def __init__(self, i2c):
        self._i2c = i2c
        self._addr = 0x77
        self._delays = (7, 8, 14, 28)
        self._sign = time.ticks_diff(1, 0)
        cal = unp(
            ">hhhHHHhhhhh",
            self._i2c.readfrom_mem(self._addr, 0xAA, 22),
        )
        (self._AC1, self._AC2, self._AC3, self._AC4, self._AC5, self._AC6,
         self._B1, self._B2, self._MB, self._MC, self._MD) = cal
        self._oversample = 2
        self._ut = bytearray(2)
        self._mlx = bytearray(3)
        self._cmd = bytearray(1)
        self._B5 = 0
        self._gauge = self._make_gauge()
        for _ in range(128):
            next(self._gauge)
            time.sleep_ms(1)

    def _make_gauge(self):
        while True:
            self._cmd[0] = 0x2E
            self._i2c.writeto_mem(self._addr, 0xF4, self._cmd)
            t0 = time.ticks_ms()
            while time.ticks_diff(time.ticks_ms(), t0) * self._sign <= 5:
                yield None
            self._i2c.readfrom_mem_into(self._addr, 0xF6, self._ut)

            self._cmd[0] = 0x34 | (self._oversample << 6)
            self._i2c.writeto_mem(self._addr, 0xF4, self._cmd)
            t0 = time.ticks_ms()
            while time.ticks_diff(time.ticks_ms(), t0) * self._sign <= ↪
                self._delays[self._oversample]:
                yield None
            self._i2c.readfrom_mem_into(self._addr, 0xF6, self._mlx)
            yield True

    def blocking_read(self):
        while next(self._gauge) is None:
            pass

    @property
    def temperature(self):
        next(self._gauge)
        x1 = ((unp(">H", self._ut)[0] - self._AC6) * self._AC5) >> 15
        x2 = (self._MC << 11) // (x1 + self._MD)
        self._B5 = x1 + x2
        return ((self._B5 + 8) >> 4) / 10.0

    @property
```

```
    def pressure(self):
        self.temperature
        up = (((self._mlx[0] << 16) + (self._mlx[1] << 8) + self._mlx[2])
              >> (8 - self._oversample))
        b6 = self._B5 - 4000
        x1 = (self._B2 * ((b6 * b6) >> 12)) >> 11
        x2 = (self._AC2 * b6) >> 11
        b3 = (((self._AC1 * 4 + x1 + x2) << self._oversample) + 2) >> 2
        x1 = (self._AC3 * b6) >> 13
        x2 = (self._B1 * ((b6 * b6) >> 12)) >> 16
        x3 = ((x1 + x2) + 2) >> 2
        b4 = (self._AC4 * (x3 + 32768)) >> 15
        b7 = (up - b3) * (50000 >> self._oversample)
        p = (b7 * 2) // b4
        x1 = (((p >> 8) * (p >> 8)) * 3038) >> 16
        x2 = (-7357 * p) // 65536
        return (p + (x1 + x2 + 3791) // 16) / 100
```

Лістинг Б.12. Прикладна програма main.py (варіанти з давачем)

Файл у репозиторії: `https://github.com/dmytro-kushnir/ppid-labs/blob/praktikum-2026/wokwi/lab04-i2c-sensor/main.py`

```
# PPID lab 4 — I2C environmental sensor (Wokwi)
# Wokwi files: this main.py + diagram.json (here) + bmp180.py (wokwi/lib/)
# Sensor: board-bmp180 (I2C 0x77). Course variants on real hardware: BME280.

from machine import I2C, Pin
import time

from bmp180 import BMP180

i2c = I2C(0, scl=Pin(22), sda=Pin(21), freq=100000)

print("PPID Lab 4 — I2C sensor")
addrs = i2c.scan()
print("I2C scan:", [hex(a) for a in addrs])

sensor = BMP180(i2c)

while True:
    try:
        sensor.blocking_read()
        print("TEMP={:.1f} PRESS={:.1f}".format(sensor.temperature, sensor.pressure))
    except OSError as e:
        print("I2C error:", e)
    time.sleep(1)
```

Лістинг Б.13. Опис схеми diagram.json (варіанти з давачем)

Файл у репозиторії: `https://github.com/dmytro-kushnir/ppid-labs/blob/praktikum-2026/wokwi/lab04-i2c-sensor/diagram.json`

```
{
  "version": 1,
  "author": "PPID LPNU",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 0, "left": 0, "attrs": ↪
        {} },
    {
      "type": "board-bmp180",
      "id": "bmp1",
      "top": 96,
      "left": 220,
      "attrs": { "temperature": "24", "pressure": "101325" }
    },
    {
      "type": "wokwi-logic-analyzer",
```

```
      "id": "logic1",
      "top": -120,
      "left": 80,
      "attrs": { "channelNames": "SDA,SCL" }
    }
  ],
  "connections": [
    [ "esp:TX", "$serialMonitor:RX", "", [] ],
    [ "esp:RX", "$serialMonitor:TX", "", [] ],
    [ "bmp1:VCC", "esp:3V3", "red", [ "v0" ] ],
    [ "bmp1:GND", "esp:GND.1", "black", [ "v0" ] ],
    [ "bmp1:SDA", "esp:21", "green", [ "v0" ] ],
    [ "bmp1:SCL", "esp:22", "yellow", [ "v0" ] ],
    [ "logic1:D0", "esp:21", "green", [ "v0" ] ],
    [ "logic1:D1", "esp:22", "yellow", [ "v0" ] ]
  ],
  "serialMonitor": { "display": "auto", "newline": "lf" }
}
```

Лістинг Б.14. Драйвер OLED-індикатора SSD1306 ssd1306.py

Файл у репозиторії: `https://github.com/dmytro-kushnir/ppid-labs/blob/praktikum-2026/wokwi/lib/ssd1306.py`

```
# SSD1306 OLED driver for MicroPython (from micropython/micropython-lib, MIT).
# Wokwi: wokwi-ssd1306 (I2C 0x3C). Used by OLED variants (4, 6, 9) in labs 4-5.

from micropython import const
import framebuf

# register definitions
SET_CONTRAST = const(0x81)
SET_ENTIRE_ON = const(0xA4)
SET_NORM_INV = const(0xA6)
SET_DISP = const(0xAE)
SET_MEM_ADDR = const(0x20)
SET_COL_ADDR = const(0x21)
SET_PAGE_ADDR = const(0x22)
SET_DISP_START_LINE = const(0x40)
SET_SEG_REMAP = const(0xA0)
SET_MUX_RATIO = const(0xA8)
SET_COM_OUT_DIR = const(0xC0)
SET_DISP_OFFSET = const(0xD3)
SET_COM_PIN_CFG = const(0xDA)
SET_DISP_CLK_DIV = const(0xD5)
SET_PRECHARGE = const(0xD9)
SET_VCOM_DESEL = const(0xDB)
SET_CHARGE_PUMP = const(0x8D)


class SSD1306(framebuf.FrameBuffer):
    def __init__(self, width, height, external_vcc):
        self.width = width
        self.height = height
        self.external_vcc = external_vcc
        self.pages = self.height // 8
        self.buffer = bytearray(self.pages * self.width)
        super().__init__(self.buffer, self.width, self.height, framebuf.MONO_VLSB)
        self.init_display()

    def init_display(self):
        for cmd in (
            SET_DISP | 0x00,  # off
            SET_MEM_ADDR, 0x00,  # horizontal
            SET_DISP_START_LINE | 0x00,
            SET_SEG_REMAP | 0x01,  # column addr 127 mapped to SEG0
            SET_MUX_RATIO, self.height - 1,
            SET_COM_OUT_DIR | 0x08,  # scan from COM[N] to COM0
            SET_DISP_OFFSET, 0x00,
            SET_COM_PIN_CFG, 0x02 if self.height == 32 else 0x12,
```

```
            SET_DISP_CLK_DIV, 0x80,
            SET_PRECHARGE, 0x22 if self.external_vcc else 0xF1,
            SET_VCOM_DESEL, 0x30,  # 0.83*Vcc
            SET_CONTRAST, 0xFF,  # maximum
            SET_ENTIRE_ON,  # output follows RAM contents
            SET_NORM_INV,  # not inverted
            SET_CHARGE_PUMP, 0x10 if self.external_vcc else 0x14,
            SET_DISP | 0x01,  # on
        ):
            self.write_cmd(cmd)
        self.fill(0)
        self.show()

    def poweroff(self):
        self.write_cmd(SET_DISP | 0x00)

    def poweron(self):
        self.write_cmd(SET_DISP | 0x01)

    def contrast(self, contrast):
        self.write_cmd(SET_CONTRAST)
        self.write_cmd(contrast)

    def invert(self, invert):
        self.write_cmd(SET_NORM_INV | (invert & 1))

    def show(self):
        x0 = 0
        x1 = self.width - 1
        if self.width == 64:
            # displays with width of 64 pixels are shifted by 32
            x0 += 32
            x1 += 32
        self.write_cmd(SET_COL_ADDR)
        self.write_cmd(x0)
        self.write_cmd(x1)
        self.write_cmd(SET_PAGE_ADDR)
        self.write_cmd(0)
        self.write_cmd(self.pages - 1)
        self.write_data(self.buffer)


class SSD1306_I2C(SSD1306):
    def __init__(self, width, height, i2c, addr=0x3C, external_vcc=False):
        self.i2c = i2c
        self.addr = addr
        self.temp = bytearray(2)
        self.write_list = [b"\x40", None]  # Co=0, D/C#=1
        super().__init__(width, height, external_vcc)

    def write_cmd(self, cmd):
        self.temp[0] = 0x80  # Co=1, D/C#=0
        self.temp[1] = cmd
        self.i2c.writeto(self.addr, self.temp)

    def write_data(self, buf):
        self.write_list[1] = buf
        self.i2c.writevto(self.addr, self.write_list)
```

Лістинг Б.15. Прикладна програма main.py (варіанти з індикатором)

Файл у репозиторії: `https://github.com/dmytro-kushnir/ppid-labs/blob/praktikum-2026/wokwi/lab04-i2c-oled/main.py`

```
# PPID lab 4 (OLED variants 4, 6, 9) — I2C SSD1306 display (Wokwi)
# Wokwi files: this main.py + diagram.json (here) + ssd1306.py (wokwi/lib/)
# Display: wokwi-ssd1306 (I2C 0x3C). Task: show your surname (Latin A-Z).

from machine import I2C, Pin
import time
```

```
from ssd1306 import SSD1306_I2C

# TODO: replace with your surname in Latin letters (variants.json message rule).
SURNAME = "IVANOV"

i2c = I2C(0, scl=Pin(22), sda=Pin(21), freq=100000)

print("PPID Lab 4 — I2C OLED")
addrs = i2c.scan()
print("I2C scan:", [hex(a) for a in addrs])

oled = SSD1306_I2C(128, 64, i2c, addr=0x3C)

oled.fill(0)
oled.text("PPID Lab 4", 0, 0)
oled.text("I2C OLED 0x3C", 0, 16)
oled.text(SURNAME, 0, 40)
oled.show()
print("OLED: shown", SURNAME)

while True:
    time.sleep(1)
```

Лістинг Б.16. Опис схеми diagram.json (варіанти з індикатором)

Файл у репозиторії: `https://github.com/dmytro-kushnir/ppid-labs/blob/praktikum-2026/wokwi/lab04-i2c-oled/diagram.json`

```
{
  "version": 1,
  "author": "PPID LPNU",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 0, "left": 0, "attrs": ↪
        {} },
    {
      "type": "wokwi-ssd1306",
      "id": "oled1",
      "top": 96,
      "left": 220,
      "attrs": { "i2cAddress": "0x3c" }
    },
    {
      "type": "wokwi-logic-analyzer",
      "id": "logic1",
      "top": -120,
      "left": 80,
      "attrs": { "channelNames": "SDA,SCL" }
    }
  ],
  "connections": [
    [ "esp:TX", "$serialMonitor:RX", "", [] ],
    [ "esp:RX", "$serialMonitor:TX", "", [] ],
    [ "oled1:VCC", "esp:3V3", "red", [ "v0" ] ],
    [ "oled1:GND", "esp:GND.1", "black", [ "v0" ] ],
    [ "oled1:SDA", "esp:21", "green", [ "v0" ] ],
    [ "oled1:SCL", "esp:22", "yellow", [ "v0" ] ],
    [ "logic1:D0", "esp:21", "green", [ "v0" ] ],
    [ "logic1:D1", "esp:22", "yellow", [ "v0" ] ]
  ],
  "serialMonitor": { "display": "auto", "newline": "lf" }
}
```

Б.5. Програми до практичного завдання № 5

Лістинг Б.17. Прошивка вузла моніторингу main.py

Файл у репозиторії: `https://github.com/dmytro-kushnir/ppid-labs/blob/praktikum-2026/wokwi/lab05-capstone/main.py`

```
# PPID lab 5 — Capstone: I2C sensor + UART telemetry (Wokwi)
# Wokwi files: this main.py + diagram.json (here) + bmp180.py (wokwi/lib/)

from machine import I2C, Pin, UART
import time

from bmp180 import BMP180

i2c = I2C(0, scl=Pin(22), sda=Pin(21), freq=100000)
uart = UART(1, baudrate=9600, tx=17, rx=16)
INTERVAL_MS = 500

sensor = BMP180(i2c)

print("PPID Lab 5 — Capstone node")
print("I2C:", [hex(a) for a in i2c.scan()])

while True:
    try:
        sensor.blocking_read()
        line = "TEMP={:.1f}\r\n".format(sensor.temperature)
        uart.write(line.encode("utf-8"))
        print(line.strip())
    except OSError as e:
        print("ERR:", e)
    time.sleep_ms(INTERVAL_MS)
```

Лістинг Б.18. Опис схеми вузла моніторингу diagram.json

Файл у репозиторії: `https://github.com/dmytro-kushnir/ppid-labs/blob/praktikum-2026/wokwi/lab05-capstone/diagram.json`

```
{
  "version": 1,
  "author": "PPID LPNU",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 0, "left": 0, "attrs": ↪
        {} },
    {
      "type": "board-bmp180",
      "id": "bmp1",
      "top": 96,
      "left": 220,
      "attrs": { "temperature": "24", "pressure": "101325" }
    },
    {
      "type": "wokwi-logic-analyzer",
      "id": "logic1",
      "top": -120,
      "left": 80,
      "attrs": { "channelNames": "SDA,SCL" }
    }
  ],
  "connections": [
    [ "esp:TX", "$serialMonitor:RX", "", [] ],
    [ "esp:RX", "$serialMonitor:TX", "", [] ],
    [ "bmp1:VCC", "esp:3V3", "red", [ "v0" ] ],
    [ "bmp1:GND", "esp:GND.1", "black", [ "v0" ] ],
    [ "bmp1:SDA", "esp:21", "green", [ "v0" ] ],
    [ "bmp1:SCL", "esp:22", "yellow", [ "v0" ] ],
```

```
    [ "logic1:D0", "esp:21", "green", [ "v0" ] ],
    [ "logic1:D1", "esp:22", "yellow", [ "v0" ] ],
    [ "logic1:D2", "esp:17", "blue", [ "v0" ] ]
  ],
  "serialMonitor": { "display": "auto", "newline": "lf" }
}
```

Лістинг Б.19. Прикладна програма опрацювання телеметрії host/capstone_host.py

Файл у репозиторії: `https://github.com/dmytro-kushnir/ppid-labs/blob/praktikum-2026/host/capstone_host.py`

```
"""Capstone host: parse sensor log and plot (PPID lab 5)."""

from __future__ import annotations

import argparse
import csv
import re
import tempfile
from pathlib import Path

import matplotlib.pyplot as plt

TEMP_RE = re.compile(r"TEMP=(-?\d+(?:\.\d+)?)")


def parse_log_lines(lines: list[str]) -> list[tuple[int, float]]:
    readings: list[tuple[int, float]] = []
    for i, line in enumerate(lines):
        match = TEMP_RE.search(line)
        if not match:
            continue
        try:
            value = float(match.group(1))
        except ValueError:
            # Skip malformed readings (e.g. noisy serial line) instead of crashing.
            continue
        readings.append((i, value))
    return readings


def parse_log_file(path: Path) -> list[tuple[int, float]]:
    if not path.is_file():
        repo_sample = Path(__file__).resolve().parent.parent / "sample_log.txt"
        hint = (
            f"Файл не знайдено: {path}\n"
            f"Для швидкого тесту: python3 -m host.capstone_host\n"
            f"(за замовчуванням — {repo_sample})\n"
            f"Або з Wokwi збережіть Serial Monitor у my_log.txt і передайте --input ↪
                my_log.txt"
        )
        raise FileNotFoundError(hint)
    text = path.read_text(encoding="utf-8", errors="replace")
    return parse_log_lines(text.splitlines())


def write_csv(readings: list[tuple[int, float]], path: Path) -> None:
    with path.open("w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["sample", "temp_c"])
        writer.writerows(readings)


def plot_readings(readings: list[tuple[int, float]], out_path: Path) -> None:
    if not readings:
        raise ValueError("No TEMP= readings found in log")
    xs = [r[0] for r in readings]
```

```
    ys = [r[1] for r in readings]
    plt.figure(figsize=(8, 4))
    plt.plot(xs, ys, marker="o")
    plt.xlabel("Зразок")
    plt.ylabel("Температура, °C")
    plt.title("PPID Capstone — моніторинг")
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(out_path)
    plt.close()


def export_to_mock_usb(csv_path: Path, usb_dir: Path) -> Path:
    usb_dir.mkdir(parents=True, exist_ok=True)
    target = usb_dir / csv_path.name
    target.write_bytes(csv_path.read_bytes())
    return target


def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(description="Capstone host log processor")
    parser.add_argument(
        "--input",
        type=Path,
        default=Path(__file__).resolve().parent.parent / "sample_log.txt",
        help="Serial log with TEMP=... lines (default: sample_log.txt in repo)",
    )
    parser.add_argument("--plot", type=Path, default=Path("capstone_plot.png"))
    parser.add_argument(
        "--csv",
        type=Path,
        default=Path("readings.csv"),
        help="Write parsed temperatures to this CSV",
    )
    parser.add_argument("--export-usb", action="store_true")
    args = parser.parse_args(argv)

    readings = parse_log_file(args.input)
    print(f"Знайдено зчитувань: {len(readings)}")

    write_csv(readings, args.csv)
    plot_readings(readings, args.plot)
    print(f"CSV: {args.csv}")
    print(f"Графік: {args.plot}")

    if args.export_usb:
        with tempfile.TemporaryDirectory(prefix="ppid_usb_export_") as tmp:
            exported = export_to_mock_usb(args.csv, Path(tmp) / "mock_usb")
            print(f"Експорт на mock USB: {exported}")

    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

ДОДАТОК В. ВІДПОВІДІ ДО ТЕСТОВИХ ЗАВДАНЬ

Таблиця В.1

Відповіді до тестових завдань

Розділ	1	2	3	4	5	6	7	8
1	б	б	б	г	а	в	г	б
2	а	а	г	а	г	г	а	б
3	в	г	г	а	б	б	г	в
4	в	г	г	б	а	а	г	а

ПРЕДМЕТНИЙ ПОКАЖЧИК

адреса пристрою, 36, 55, 59, 61, 69, 79, 83, 89
амплітудно-часова діаграма, 38, 44–47, 50, 76, 83
арбітраж шини, 10, 16, 22, 23, 68
асинхронний інтерфейс, 25, 26, 41, 44, 67, 71, 77, 84

бібліотека HAL, 20
бібліотека libusb, 18, 20, 63
бібліотека pySerial, 18, 31, 41, 88
біт парності, 26, 27, 33, 38, 40, 41, 44, 67
бодова швидкість (baud rate), 38, 50

віртуальна пара портів, 42, 88
віртуальний COM-порт, 19, 35, 80, 85
вставлення біта (bit stuffing), 16, 39, 40, 46, 48, 50
вузол моніторингу, 77, 78, 84–86, 89

давач BMP180, 20, 68, 80, 81, 85, 86, 89, 91
дескриптор USB, 20, 53, 54, 63
диференційна пара, 8, 15, 16, 23, 33, 53, 56, 57
драйвер пристрою, 18, 22, 54, 58, 77, 78, 80, 85, 90
дуплексний режим, 8, 9, 22, 26, 33, 53, 74, 75

з’єднувачі DB9, DB25, 26–28, 33, 49
з’єднувач USB Type-C, 57, 65

індикатор SSD1306, 68, 79, 81
інтерфейс 1-Wire, 16, 22, 23
інтерфейс RS-232C, 8, 26, 33, 41, 49, 67
інтерфейс RS-485, 8, 10, 11, 16, 22, 23, 33–36, 49
інтерфейс SPI, 8, 19, 20, 72–74
інтерфейс TTL-UART, 24, 33–35, 49

кадр UART, 39–41, 44, 47, 48, 87
квитування (handshake), 12, 22, 55, 59, 61, 65
кінцева точка (endpoint), 54, 55, 59, 61, 64
класовий драйвер, 19, 20, 22
кодування NRZI, 38–40, 44–48, 50, 51, 56
контрольна сума CRC, 36, 54, 55, 61

логічний аналізатор, 37, 76, 79–81, 83, 85, 87, 89, 90

мікроконтролер ESP32, 13, 16, 19, 20, 31, 33–35, 68, 79–81, 85, 86
мікросхема 16550A, 31
мікросхема MAX485, 34–37, 51
мікросхеми MAX232, MAX3232, 33–35, 37, 49, 51
модель OSI, 36, 37, 49

напівдуплексний режим, 8, 9, 22, 23, 33, 49, 53, 67, 75, 89

опитування шини (enumeration), 53, 54, 62

паралельна шина Intel 8080, Motorola 6800, 13, 15, 22, 23
паралельний інтерфейс, 12, 13, 15, 22, 23
переривання, 19, 22, 30, 31, 54, 65
перетворювач рівнів, 33–35, 49
підтвердження ACK, NACK, 12, 41–43, 49, 55, 61, 65, 69, 79, 81–83, 89, 91
підтягувальний резистор, 53, 65, 68, 74, 79, 89
повторна умова СТАРТ, 69, 70, 79, 81, 89, 91
порт Centronics (LPT), 8, 12, 13, 23, 56
програма PulseView, 83, 87
простір ядра і простір користувача, 19, 20, 22, 78, 90
протокол Modbus RTU, 10, 11, 16, 22, 35, 36, 38, 49
прямий доступ до пам’яті (DMA), 19, 74

режими SPI (CPOL, CPHA), 73, 74, 89
рівні програмного забезпечення, 77, 78, 84, 85, 89, 90
розтягування такту (clock stretching), 71, 89, 91

сигнали RTS, CTS, 28, 31, 51
сигнал DE/RE, 33, 34, 36, 51
символи XON, XOFF, 31, 51
симулятор Wokwi, 35, 68, 76, 79–83, 86, 87
стартовий біт, 26, 33, 38, 40
стоповий біт, 26, 29, 31, 33, 34, 38, 40, 41, 50, 51, 67, 86
схема з відкритим стоком, 68, 79, 89

телеметричне повідомлення, 77, 78, 84–87, 90
термінал Serial Monitor, 35, 80–86
топологія, 8–11, 22, 23, 26, 33, 49, 53, 64, 75, 81, 89
транзакція, 52, 54, 55, 59, 61, 64, 67, 69, 70, 76, 79, 81, 87, 89
тривалість такту, 29, 38, 47

умови СТАРТ і СТОП, 69, 70, 76, 79, 81, 83, 89, 91
універсальна послідовна шина USB, 8, 11, 19, 20, 39, 40, 44, 50, 53, 54, 56–65

файл пристрою, 18, 78, 90
флеш-пам’ять, 53, 56, 58
формат кадру, 25, 31, 37, 38, 41, 42, 44, 49, 50, 84, 86, 87

хост USB, 8, 9, 18, 23, 53–57, 61, 64

часовий перекіс (skew), 15, 23

шина CAN, 8, 10, 11, 16, 20, 22, 23
шина I²C, 8, 11, 20, 67, 74, 87, 89, 91

UART, 11, 20, 31, 33, 35, 36, 38, 41, 44, 49, 84–87, 90

USB On-The-Go (OTG), 53

ІМЕННИЙ ПОКАЖЧИК

До покажчика включено прізвища авторів праць, на які є посилання в тексті посібника; зазначено номери сторінок, на яких ці праці цитуються.

Бочкарьов О. Ю., 10, 18, 77

Голембо В. А., 10, 18, 77
Гребеняк А. В., 10

Мельник А. О., 10, 18, 77
Миц А. М., 26, 31
Міюшкович Є. Г., 10, 11, 34

Парамуд Я. С., 8, 10–12, 16, 18, 26, 31, 34, 53, 58, 72, 77

Яцук В. О., 10, 18, 77

Corbet J., 19
Kroah-Hartman G., 19
Rubini A., 19

НАВЧАЛЬНЕ ВИДАННЯ

Кушнір Дмитро Олександрович

ПЕРИФЕРІЙНІ ПРИСТРОЇ, ІНТЕРФЕЙСИ ТА ДРАЙВЕРИ

Навчальний посібник

Редактор _____
Комп’ютерне верстання _____
Художник-дизайнер _____

Підписано до друку __.__.2026. Формат 60 × 84/16.
Папір офсетний. Друк на різографі.
Умовн. друк. арк. __. Обл.-вид. арк. __.
Наклад __ прим. Зам. __.

Видавець і виготівник: Видавництво Львівської політехніки
Свідоцтво суб’єкта видавничої справи ДК № 4459 від 27.12.2012 р.
вул. Ф. Колесси, 4, Львів, 79013
тел. +380 32 2584103, факс +380 32 2584101
vlp.com.ua, ел. пошта: vmr@vlp.com.ua